PRoNTo v3.0

J. SCHROUFF

Table of Contents

Introduction

This version of PRoNTo is provided by J. Schrouff, as a potential precursor for an official release of PRoNTo v3.0. It can be found on https://github.com/JessicaSchrouff (note: this version is NOT available at the usual download link at UCL). Hence, it is not supported as other versions have been. Please read the official PRoNTo documentation (v2.1 manual) as well as the provided documents for v3.0 (this document, with tutorials) before use.

This development version comes 'as is' under the GNU-GPL license without warranty. As the number of possible analysis combinations is growing exponentially when adding new features, not all options could be tested. Please note that this document was written while finalizing the code, hence there might be slight discrepancies between the figures and the actual interfaces. This however does not affect the information in this document in a major way.

Although this has been quite a solo effort in terms of development, I would like to thank Janaina Mourao-Miranda, Maria Joao Rosa, Joao Matos Monteiro, Liana Lima Portugal, Tong Wu, Fabio Ferreira, Agoston Mihalik, Josef Parvizi, Amy Daitch, Stephan Bickel and Christophe Phillips for their advice and help with testing/developing v3.0. This work was funded by a Marie Sklodowska-Curie global fellowship to J. Schrouff (project DecoMP-ECoG, #654038).

How to cite

Please refer to:

- Schrouff J, Rosa MJ, Rondina JM, Marquand AF, Chu C, Ashburner J, Phillips C, Richiardi J, Mourao-Miranda J. PRoNTo: Pattern Recognition for Neuroimaging Toolbox. Neuroinformatics, 2013.
- Schrouff J, Mourao-Miranda J, Phillips C, Parvizi J. Decoding intracranial EEG data with multiple kernel learning method. Journal of Neuroscience Methods, 2016.

When using this version of PRoNTo. Please also refer to:

- Schrouff J, Monteiro, JM, Portugal L, Rosa MJ, Phillips C, Mourao-Miranda J. Embedding Anatomical or Functional Knowledge in Whole-Brain Multiple Kernel Learning Models Neuroinformatics, 2018.
- Schrouff J, Cremers J, Garraux G, Baldassarre L, Mourao-Miranda J, Phillips C. Localizing and Comparing Weight Maps Generated from Linear Kernel Machine Learning Models. International Workshop on Pattern Recognition in Neuroimaging (PRNI), 2013.

When using Multiple Kernel Learning to combine signals from different Regions of Interest (ROIs) or modalities, or when using a posteriori weight summarization (respectively).

Nomenclature

This version of PRoNTo broadens the spectrum of potential applications. Hence some terms that have been used in the toolbox and documentation so far need to be revised. Here is a list of terms and their meaning to PRoNTo:

Data format:	Format the recording files were saved in. PRoNTo accepts 3 data formats: nifti, .mat and MEEG objects (SPM .mat/.dat pairs).
Experimental design:	Refers to within-subject experimental design, i.e. when multiple samples are generated for a single subject.
Features:	Variables recorded and on which to perform the modelling. For brain images saved as nifti, the features typically correspond to voxels. For EEG/MEG traces, features correspond to time points on channels per frequency bin. For .mat, features are the variables provided.
Fold:	Cross-validation split. Each fold defines some training samples and some test samples, from which one model is built. Results in PRoNTo are presented in average (with std) across folds.
Machine:	Machine learning algorithm, e.g. Support Vector Machine or Binary Gaussian Processes classification.
Modality:	Recording of brain or behavioural data, with or without experimental design. Modalities can have different or the same data formats.
Regions of Interest (ROI):	A region of interest is a subset of features to build a kernel from. The ROIs are identified by an integer number from an atlas file. For brain images saved as nifti, ROIs correspond to anatomical or functional parcellations (e.g. AAL atlas). For EEG/MEG, ROIs can correspond to any combination of channel/ time points and frequency bins. The atlas is built implicitly. For .mat, the atlas needs to be built by the user and ROIs can correspond to any (exclusive) grouping of variables.
Run:	When multiple recordings of the same data are performed, these can be entered as separate 'modalities' in the Data and Design and then combined (i.e. concatenated in samples) during the building of the feature set.
Sample:	A data point in our multi-dimensional space, where the number of dimensions corresponds to the number of features. These data points are used to build the regression/classification model. Samples can hence correspond to fMRI scans, to beta images, to sMRI scans or to single or averaged trials of an EEG/MEG recording. Typically, there will be one sample per subject (Select by Samples) or one sample per condition or trial (Select by Subject).

New functionalities

File formats

PRoNTo now accepts multiple 'Data formats', i.e. nifti (as before), .mat and MEEG SPM objects:

MEEG

EEG, MEG, LFP or ECoG files converted to the MEEG SPM data format can be entered when choosing 'Select by Subject'. PRoNTo assumes that this file format represents a within-subject design that it will read automatically. The data has to contain epoched trials, either one per stimulus or one per condition. Only one MEEG file can be entered per subject and modality.

.mat

Any set of variables can be saved in a .mat and used in PRoNTo. The variables need to be numerical (no categorical inputs) and saved as an array or vector (no restriction on dimension) in the first variable of the file. One .mat needs to be provided per sample. The dimensionality of the data needs to be consistent across samples (i.e. same number of dimensions and variables in the same order).

The model weights can be displayed for 1D and 2D MEEG or .mat inputs. The display has not been specialized for MEEG as there are many much better tools to explore such files (either directly in SPM or converting back to your favourite software).

Combine multiple formats

In previous versions of PRoNTo, modalities could only be combined if they had the same features. The framework has been modified to allow the combination of modalities with different feature spaces. First, only modalities of the same data format can be combined at the feature set level. The option for 'building one kernel per modality' has been removed. This means that, at the feature set step, only *runs* of a same data format can be concatenated in samples. At the specify model step, multiple feature sets can now be chosen in a model. These will be combined in features (either concatenated or used in multiple kernel settings depending on the choice of the machine) during model estimation.

Important note: when combining multiple feature sets, their samples need to be identical across feature sets. This means that if there are 3 conditions in the MEEG object saved as 'condB', 'condA', 'condC', it is not enough that the equivalent beta images for fMRI include conditions 'condA', 'condB' and 'condC', these need to be entered in the exact same order! It might not throw an error but will lead to poor performance of the model.

Subsampling classes

PRoNTo now allows to subsample classes to the least represented class (or as close as possible) while taking into account the stratified structure of the data (e.g. blocks of scans in fMRI) as well as potential pooling of multiple conditions (i.e. it will subsample equally from all pooled conditions). Related to this option, a new module has been implemented: 'Specify from'. This module (batch and GUI) allows to choose a model that was previously specified as a 'basis' for a new model. Some fields of this 'basis' model will be copied into the new model, including class or regression sample selection and outer cross-validation. This ensures that, when performing subsampling, the exact same samples are considered in the 'basis' model and in the new model to ease performance comparison.

Machines

Kernel machines

PRoNTo interfaces LIBSVM. However, only the SVM algorithm was used so far. PRoNTo now interfaces LIBLINEAR as well and uses more algorithms from both libraries. In addition, the 'custom' machine can now be optimized given hyper-parameters. PRoNTo also allows the optimization of more than one parameter (entered as a cell array of values that is then transformed into a grid).

Non-kernel machines

The non-kernel route has also been enabled, i.e. the 'Use kernel' radio button can be ticked off to use primal formulations instead of kernel machines. This is intended for samples with a low number of features. Please note that using these algorithms on typical neuroimaging datasets (i.e. # features >> # samples), will likely lead to time-consuming estimations and poor performance.

Regression targets

Per subject

Multiple regression targets can be specified at the Data and Design level. When choosing which samples to use for the regression model, the user can also specify which target to use. This avoids the need for multiple PRT files when multiple regression targets might be of interest.

Per trial

Regression targets can be input along the conditions of an experimental design to perform within-subject regression.

Cross-validation

A 'Leave-One-Block-per-Class-Out' cross-validation scheme was added with its k-folds counterpart to allow balanced train and test sets when performing within subject cross-validation.

Other important changes

Model performance estimation

Model performance was typically concatenated across folds (in the confusion matrix or ROC curve). The concatenation can however lead to over-optimistic model performance estimations and further reflects a model that was not estimated (instead, we estimated multiple models). We have hence modified the code to compute the average of performance across folds. This is also reflected in the 'Display results' window where some plots have been replaced at the model level (e.g. replacing the overall confusion matrix by a 'balanced accuracy distribution' across folds violin plot). In general, the results will reflect more the average but also the deviation of the results across folds. We encourage the users to report both values.

Hyper-parameter optimization

This change only applies when multiple values of the hyper-parameter lead to the same maximum value of model performance. In previous versions, PRoNTo was choosing the median value. In v3.0, PRoNTo identifies the largest stable region (if the 'Image Processing' toolbox from Matlab is present) and chooses the center of gravity of this region in the hyper-parameter grid as the best hyper-parameter. This change should have limited impact.

With great power…

Many checks and errors have simplified or discarded to allow more flexible analyses. It however comes with the downside that it is easier to make mistakes in an analysis. Please make sure you double-check every step and regularly load the PRT to verify the input information.

Input files

In some cases, files need to be input by the user. These typically need to include a specific variable or satisfy certain conditions. Here is a summary of input files and their requirements:

Data and Design

.mat inputs

For .mat modalities, PRoNTo expects one file per sample, as for nifti. This means that, no matter the dimensionality of the data, one .mat file needs to be saved per subject for 'Select by Scans' or per trial

'Select by subject'. The first variable of this file will then be extracted as the data for that sample. The data array can be of any dimensionality but needs to be consistent across samples (no missing values).

A script is provided (in Appendix and in PRoNTo\utils) to convert a matrix that would contain all the data for the different samples into separate .mat files. The script reads each row of the matrix and saves it as a .mat file in a separate subdirectory.

Regression target file – across subjects

A regression target file needs 2 variables:

- 'names': a cell array with the names of the regression targets, the first name corresponding to the first column of the matrix, and so on. Example: {'age','score'}
- 'rt_subj': a matrix of size # of subject x # of regression targets. For 3 subjects and the 2 regression targets mentioned above, we'd have:

```
>> rt_subj = [25, 6; 26, 8; 30, 7]

rt_subj =

   25    6
   26    8
   30    7
```

Covariate file – across subjects

The covariate file at the subject level should contain a matrix R, of size # subjects x # covariates. It cannot contain missing or NaN values and cannot be a cell array. For categorical variables, the values need to be one-hot encoded if no ordinal ranking is expected. For the 3 subjects above, a one-hot encoding of gender would be (2 males, one female):

```
>> R = [1, 0; 1, 0; 0, 1]

R =

   1   0
   1   0
   0   1
```

The same number of covariates should be provided across subjects.

Conditions file – within subject

To specify the design of a subject using a .mat file, 5 variables can be provided:

- names: cell array of conditions names, e.g. {'A','B'}
- onsets: cell array of vectors on onsets for each condition, e.g. {[1,4,8],[2,5,7]}
- durations: cell array of vectors or single values for each condition, e.g. {1,1}. In the case of a single value, this value will be repeated for all trials.
- (optional) rt_trial: cell array of vectors containing one target per trial (including bad trials for MEEG), e.g. {[1.2, 1.3, 1.1],[2.1, 2.5, 2.3]}.

- (optional) R: cell array of matrices or vectors containing covariates per trial (including bad trials for MEEG). Each matrix should have the size # trials in condition x # covariates. The same number of covariates should be provided across conditions and subjects. E.g. {[1,0;0,1;1,0],[0,1;1,0;1,0]}.

Feature set

Atlas for .mat

When loading an atlas for a .mat modality, PRoNTo will extract the first variable of that file. In this first variable (that hence can have any name), an array should be found. This array should have the exact same dimensions as the dimensions of the data from the first (and other) subjects. It should include values between 1 and the number of ROIs. 0 and NaN values will be excluded from the feature set (as the atlas is also forced as 2nd level mask).

To include labels of ROIs with the atlas, a separate file called 'Labels_*atlasname*.mat' needs to be found along the atlas '*atlasname*.mat'. This file should contain the variable 'ROI_names', a cell array of ROI labels. The structure of this file is the same as for nifti ROI labels. Alternatively, ROI labels can be loaded at the 'Display weights' step.

A script that builds an atlas and its labels automatically from a list of networks for connectivity matrices is provided in appendix and along the PRoNTo distribution (in \utils). If we assume that 2D connectivity matrices are provided as input, estimated from a certain number of time series (e.g. 200). Among those time series, some are thought to pertain to a same 'network' (e.g. 'Default Mode' or 'Visual'). The script will build a ROI for each 'network' within and between interaction. For example, 'Visual-Visual' will be ROI_1, 'Visual-Default Mode' will be ROI_2 and 'Default Mode-Default Mode' will be ROI_3.

Channel selection file for MEEG in batch

The 'channel selection' module in the batch comes from SPM directly. They accept a file containing a cell array called 'label', with each cell being a channel name to include.

Model

Custom cross-validation

Custom cross-validation allows to load a matrix. The name of the variable in the .mat file should be 'CV' and the matrix should be of size #selected samples x #folds. The values in the matrix need to be either 0 (sample not selected in fold), 1 (sample used for training) or 2 (sample used for testing).

Display weights

Labels for ROIs

For nifti and .mat, ROI labels can be loaded when displaying the weights (MEEG comes with its own labelling). The loaded .mat file should contain a variable 'ROI_names', a cell array of ROI labels. The number of labels has to match the number of regions as defined in the atlas.

Tutorial: SPM multimodal face dataset

Data & pre-processing

This dataset is a visual experiment where pictures of famous faces (*Famous*), unfamiliar faces (*Unfamiliar*) and scrambled faces (*Scrambled*) are presented to a subject. 16 subjects were recorded, in fMRI, EEG and MEG. The data is available for research purposes from ftp://ftp.mrc-cbu.cam.ac.uk/personal/rik.henson/wakemandg_hensonrn/ and is fully detailed in the publication:

'A multi-subject, multi-modal human neuroimaging dataset', Wakeman, D. G. and R. N. Henson, Scientific Data, vol. 2, 150001, 2015.

A within-subject version of the dataset can be found, along with pre-processing batches, on the SPM website (https://www.fil.ion.ucl.ac.uk/spm/data/mmfaces/).

In this tutorial, the EEG and MEG data was used to provide 6 modalities:

- EEG: average voltage trace for each channel, in the time window [-100 800]ms around onset. There are 70 EEG channels. There is one file with the 3 traces per subject.
- MEG: average trace for each channel, in the considered time window. There are 102 MEG channels. There is one file with the 3 traces per subject.
- Interpolated EEG: nifti images from spatial interpolation of average trace per condition on the scalp, for EEG. We thus obtain one image per condition and per subject.
- Interpolated MEG: nifti images from spatial interpolation of average trace per condition on the scalp, for MEG.
- Connectivity EEG: for each pair of channels, we computed the correlation between the average traces of a condition. The output is a 70*70 matrix per condition and per subject. Only the upper triangular part of the matrix was extracted.
- Connectivity MEG: for each pair of channels, we computed the correlation between the average traces of a condition. The output is a 102*102 matrix per condition and per subject. Only the upper triangular part of the matrix was extracted.

The EEG, MEG, interpolated EEG and MEG files were obtained from following the batches provided with the corresponding chapter in the SPM manual. The connectivity was built from the obtained EEG and MEG files, based on in-house scripts (in appendix).

GUI

Data and Design

There are 16 subject to input. For each subject, there are 6 modalities to specify. The easiest is to fully specify subject 1, then use the entered modalities and designs in further subjects. First we choose a directory to save the resulting PRT.mat. We then add a group (e.g. 'G1'), and one subject ('S1').

Modalities of subject 1

We then enter the first modality, call it 'EEG' and choose its type as 'MEEG'. When the file is selected, PRoNTo will automatically read the design from it. The design can be reviewed be clicking on Events in file'.

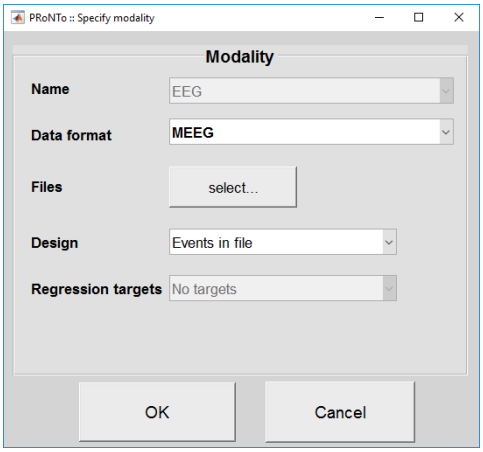


Figure 1: MEEG modality specification.

We see 3 conditions: Unfamiliar, Famous and Scrambled. The units are set to seconds and the TR is the sampling interval (in seconds) in the file. In this example, the TR of 0.005 corresponds to a sampling rate of 200Hz.

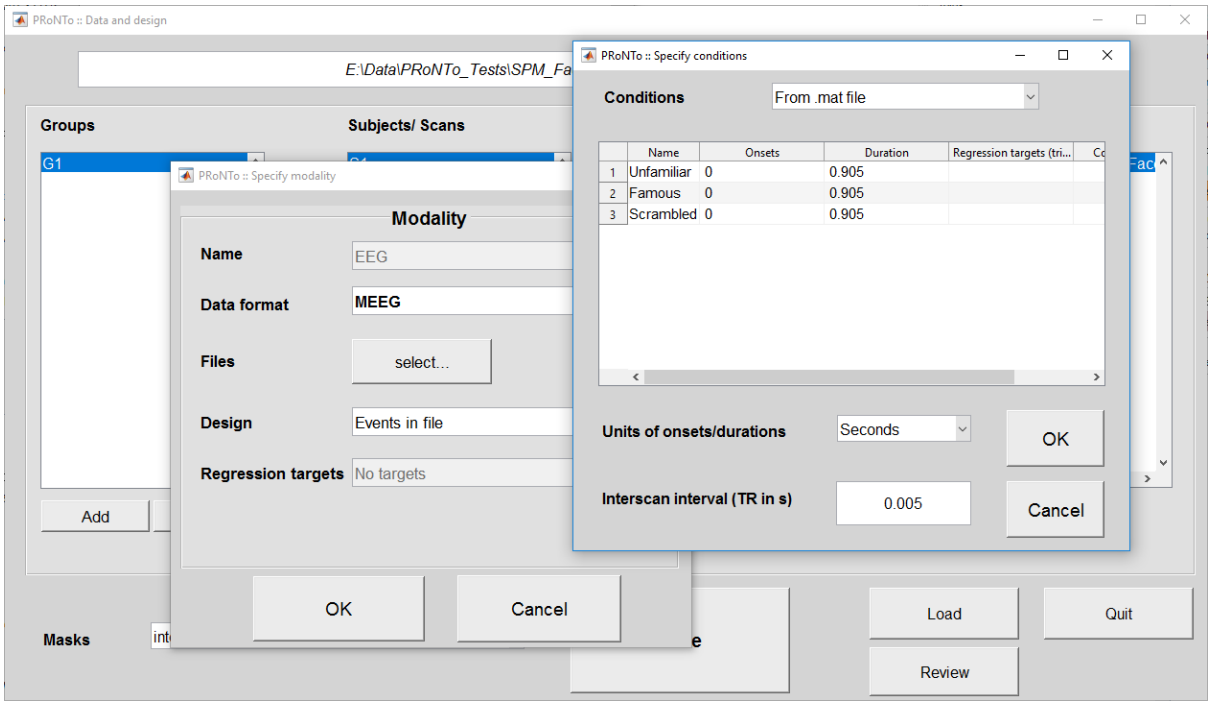


Figure 2: MEEG design review.

The same can be done for the MEG modality.

The interpolated EEG consists of nifti files. This is similar to entering beta images from a GLM analysis in previous versions of PRoNTo. The only difference is that the type needs to be specified (it is nifti by default).

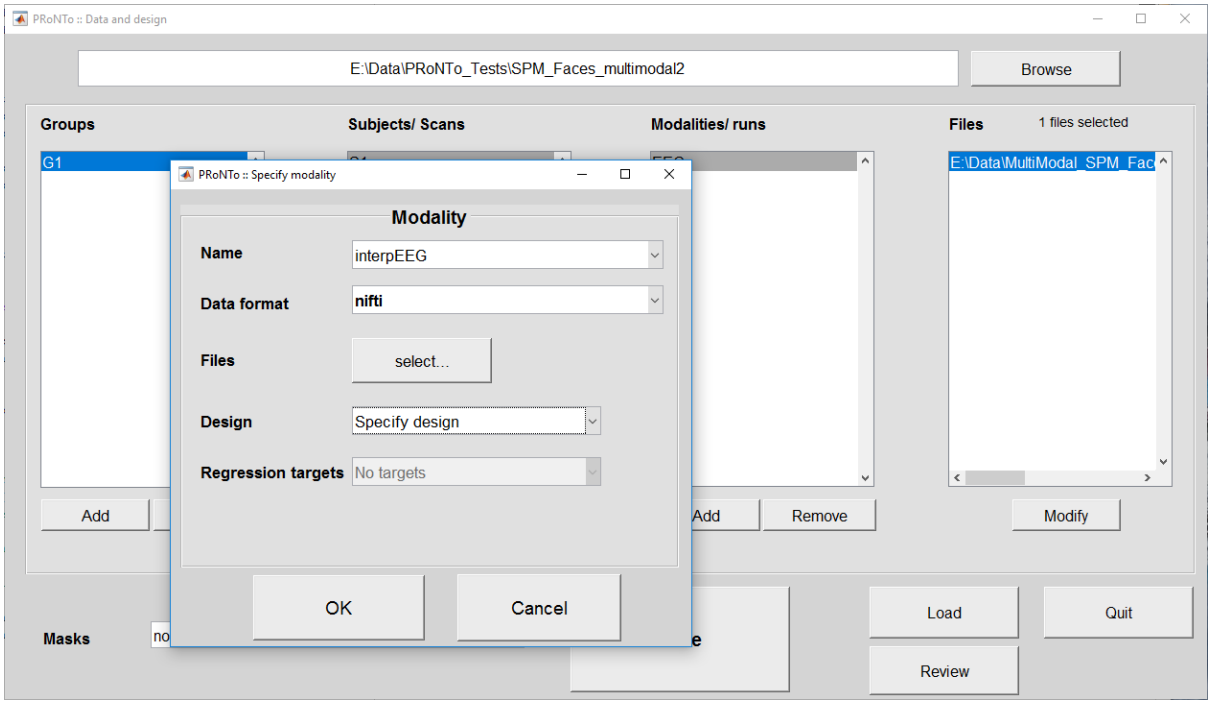


Figure 3: nifti modality specification.

Important note: the order of the conditions should be the same across all modalities that will be used in a single model. This means that we should use the same order (Unfamiliar, Famous, Scrambled) that was input with the EEG and MEG files.

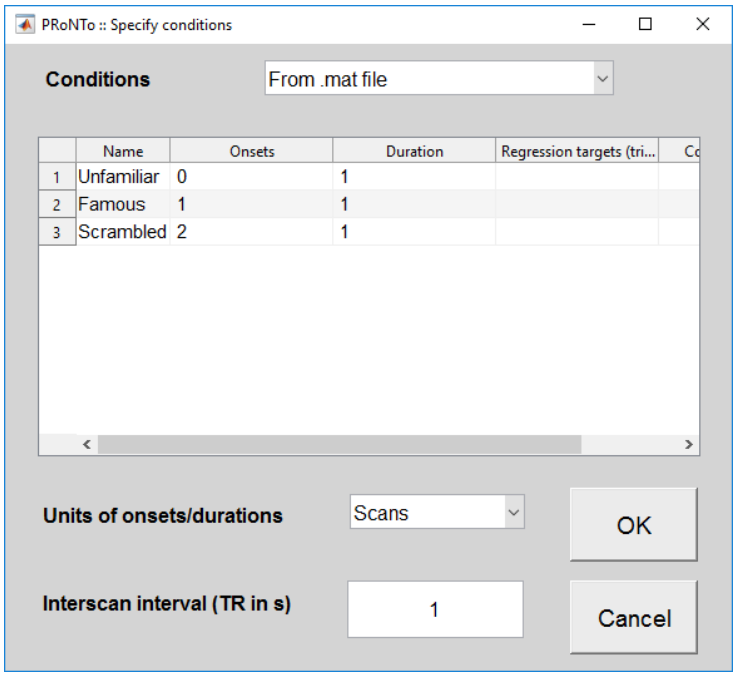


Figure 4: nifti modality design, to match the EEG/MEG design.

As when using beta images, the onsets should correspond to the index of the image in the file selection, with the smallest index (i.e. first selected image) being 0. The duration should be set to 1 for all images, and the units should be set to 'Scans', with a TR of 1.

The interpolated MEG is entered in the same way.

The connectivity derived EEG and MEG matrices can then be entered, in a similar way as for images. The 'Data format' needs to be set to '.mat'. The design options are then 'Specify design' or 'No design'.

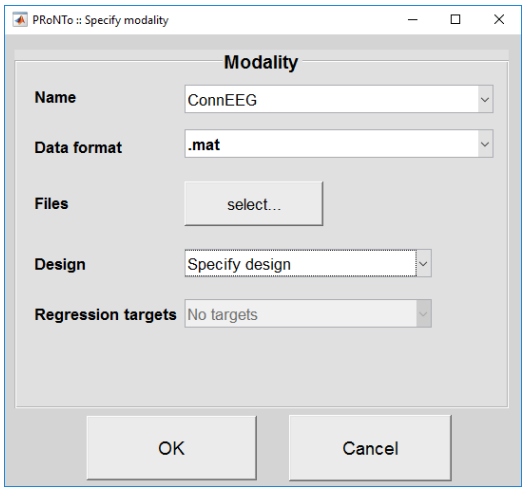


Figure 5: .mat modality specification.

Specifying the design is identical to that of images. As for images, PRoNTo expects one .mat file per trial/sample. **Important note**: When loading the file, it will read the first variable only (independently of the variable's name). You hence need to ensure that the variable of interest is saved as the first one, or in a specific .mat.

Modalities of other subjects

The second subject can then be entered by selecting the already specified modalities. For EEG and MEG, the design is read automatically. For .mat and nifti types, the design menu will have the 'Replicate design from subject 1' option that is very convenient in this application.

Masks

Mask files then need to be specified for nifti images. For .mat and MEEG, we indeed assume that the users provide the useful information only, as this can be easily performed during pre-processing. Please ensure that your data (.mat or MEEG) do not contain NaNs. If the user wants to use a mask, it can still be done at the feature set level (for connectivity matrices, we'll see in another tutorial that another option could be to enter the full matrix and specify a 2nd level mask).

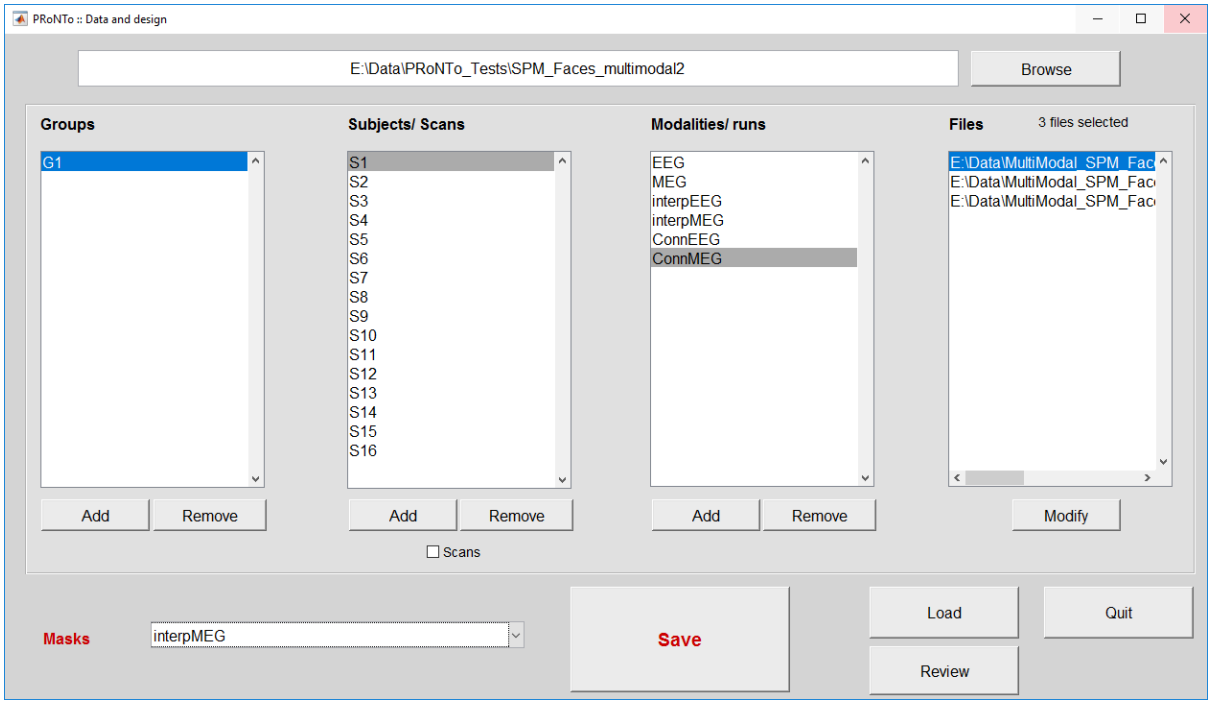


Figure 6: masks, only for nifti modalities.

A simple 'Check Reg' in SPM will show that the interpolated EEG (MEG) images have NaNs at different pixels, for different subjects. We hence need to create a common mask to discard those NaNs. This can easily be performed using SPM ImCalc batch (select all images from the EEG interpolated modality, select 'read as matrix', operation can be: ~isnan(sum(X))) and should be done for EEG and MEG separately.

Review

The data and design can be reviewed. It will display one group of 16 subjects, with 6 modalities. For nifti modalities, this is where you'd specify HRF parameters if you were using a design specified in seconds. This option is (obviously) not available for .mat or MEEG. In the present case, our design is in terms of images/scans. The default values of 0 and 0 should not be modified.

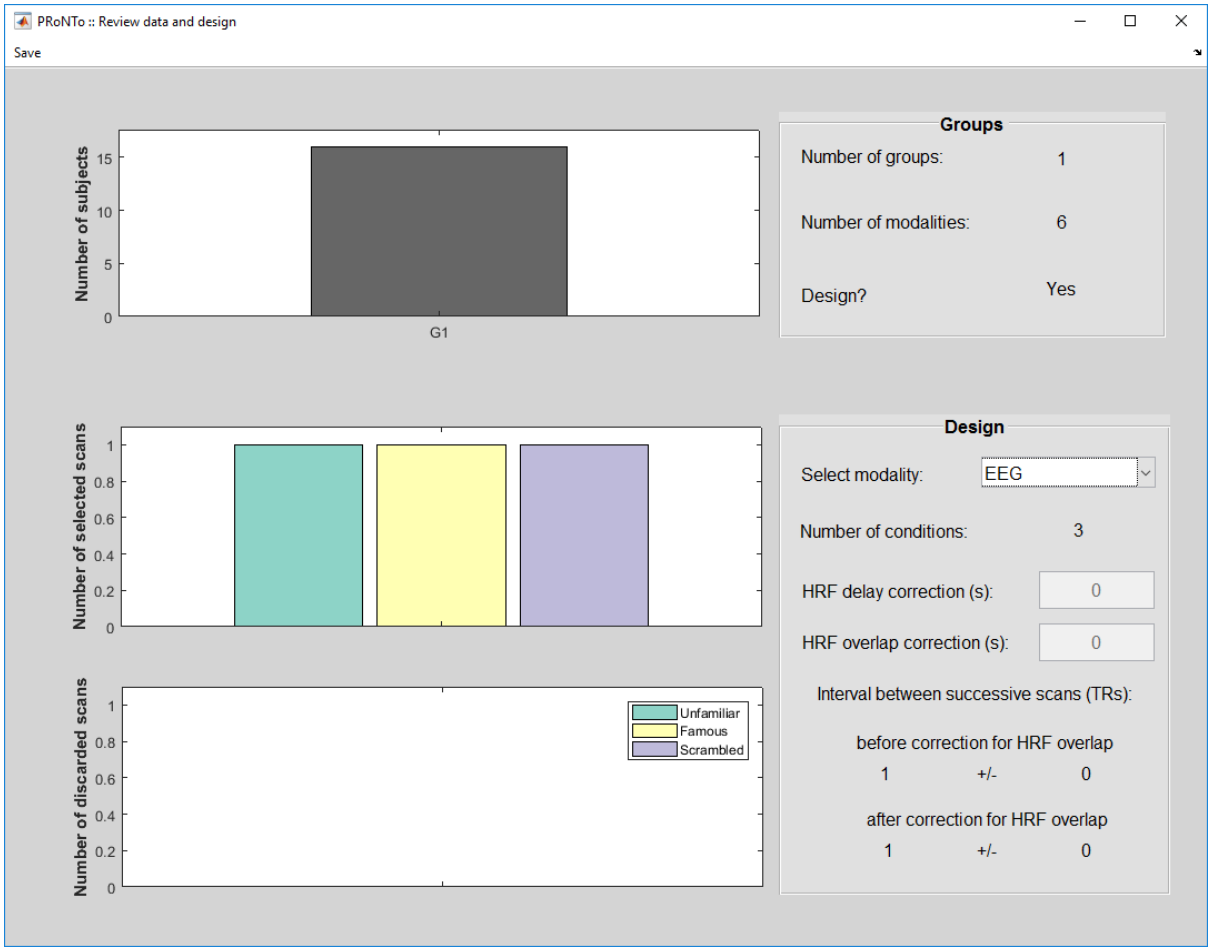


Figure 7: Review data and design

Feature sets

In version 2, modalities could be combined in 2 ways at this stage: either concatenated in samples (e.g. multiple runs of a same experiment), or by computing one kernel per modality and storing them together. This had the limitation that combined modalities (in either way) had to have the exact same number of features. We have now decoupled those operations for more flexibility. At the feature set stage, modalities can only be concatenated in samples. They then still need to have the exact same number of features.

As we want to use our modalities as different 'features' for the same model and not as different samples, we hence need to build one feature set per modality. The different data types are handled in different ways: images and .mat in a way similar to v2, MEEG with a new window.

This means that, after loading the PRT, a new window will appear if multiple data formats are contained in the PRT. The window asks which data format the specified feature set will have.

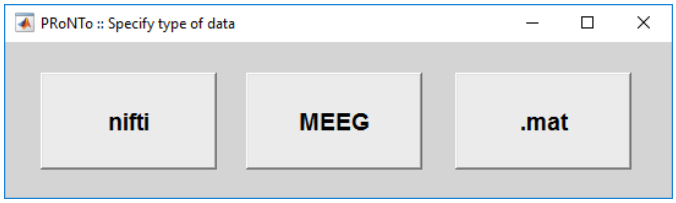


Figure 8: Feature sets are built for only one data format at a time. The buttons available on this interface will reflect the data formats present in your PRT. If only one type is present, this step is skipped.

We first choose 'nifti' and will build the feature set for interpolated EEG.

Once the data format has been chosen, the process is similar to that of v2: if there are multiple modalities of that data format in the PRT, the following window will appear, asking how many modalities to concatenate. Here, there is only one modality to consider.

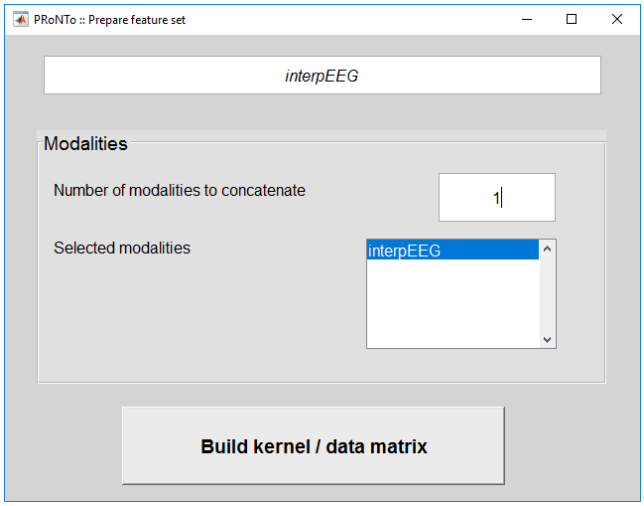


Figure 9: Specify modalities to concatenate as well as feature set name. This window is the same as in v1.

After specifying '1' and 'Enter/Return', the appropriate window will then open to specify parameters for this specific modality. If there were multiple modalities to concatenate, the following windows would open as many times as the number entered.

nifti data format

For nifti, nothing changes. All options previously available are included in v3, with no addition. We select all default options. We repeat for the MEGinterp modality to create a second feature set.

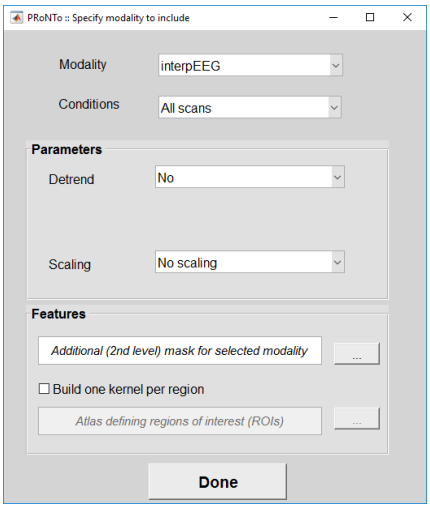


Fig 10: nifti modality specification, as in v2.

.mat data format

To create the connectivity MEG feature set, we repeat the first step but select '.mat'. As we have 2 .mat modalities, window in Fig.9 appears first. Again, we enter 'ConnMEG' as the feature set name and enter '1' as the number of modalities to include. The next window is the same as for images. The only option not available is the detrend. 2nd level masks and atlases can be entered. An example is provided in another tutorial. In the present case, we use all default options.

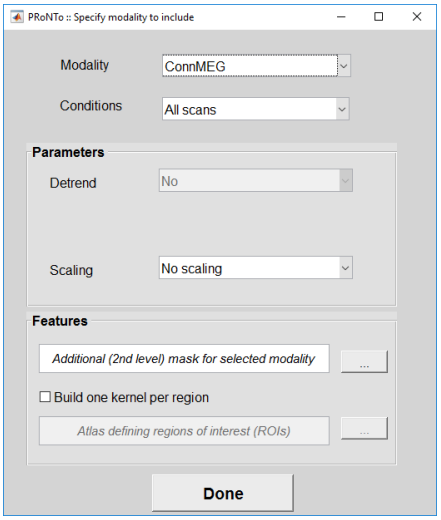


Figure 10: specify .mat modality. Similar to nifti, only detrending is disabled.

MEEG modality

To build an MEEG feature set, we select 'MEEG' in the first window and specify 'EEG' as the feature set name and '1' as the number of modalities to include in the second window. Then, a new window appears. It allows to 'play' along the different dimensions of the file, averaging across dimensions or building multiple kernels. This last option will be equivalent in the code to creating an atlas for the MEEG file. The atlas is not saved but the features selected along each dimension are stored and further used to enable building the weights per kernel.

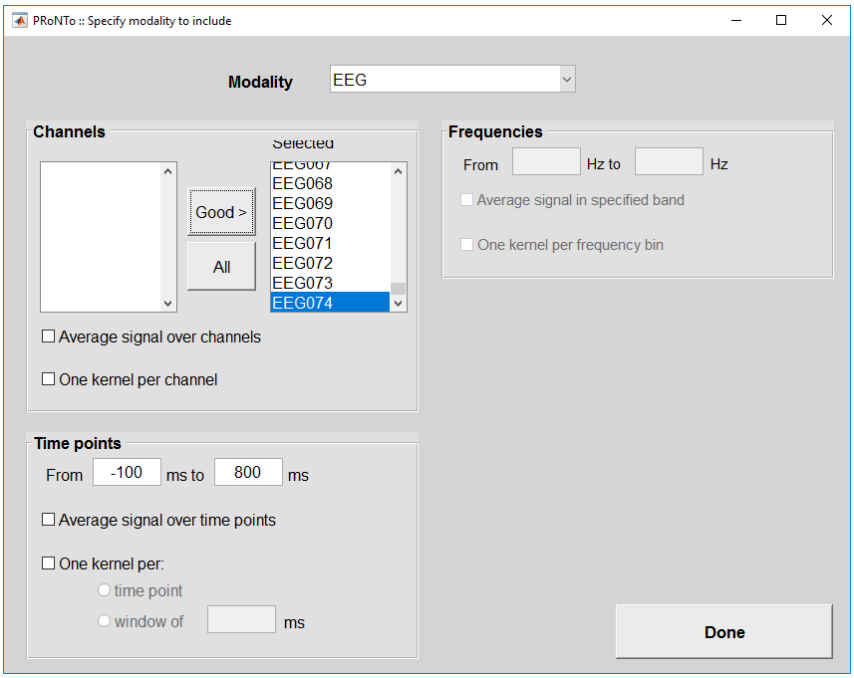


Figure 11: specify MEEG modality.

In the present case, we choose all 'good' channels (which is equivalent to all channels as this is a multi-subject study), use the whole time window for consistency with the other modalities and do not specify kernels along a dimension as we did not use an atlas for nifti or .mat. The 'Frequencies' panel is disabled as the signal is in voltage and was not decomposed in time-frequency. Files with TF information enables this panel.

Important note: For .mat and nifti, we assume that feature 1 in sample 1 is equivalent to feature 1 of sample 2. For MEEG, the assumption is the same: the first time point in the first channel and frequency bin should be equivalent across subjects/runs. This means that if the channel montage was different or epoching was different, preprocessing steps need to be performed to ensure compatibility across files.

After we build the 6 feature sets, we can see 12 new files along the PRT.mat: 6 binary file arrays (.dat) that collect the information across all subjects and samples in a modality, and 6 linear kernels (.mat) that store the pair-wise similarity between all selected samples in a feature set.

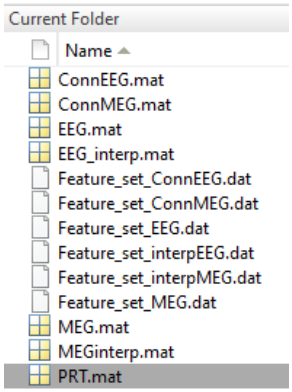


Figure 12: output files after creating 6 feature sets.

Model specification

Our goal is now to discriminate between Famous faces and Scrambled faces. We will first build one model for each modality separately, to see how they perform, then we'll build a model that combines all the information and look at the modality contributions.

The main PRoNTo window has been amended to allow specifying new models (similar to 'Specify' in v2), as well as specifying models from previously specified models from the same PRT ('Specify from').

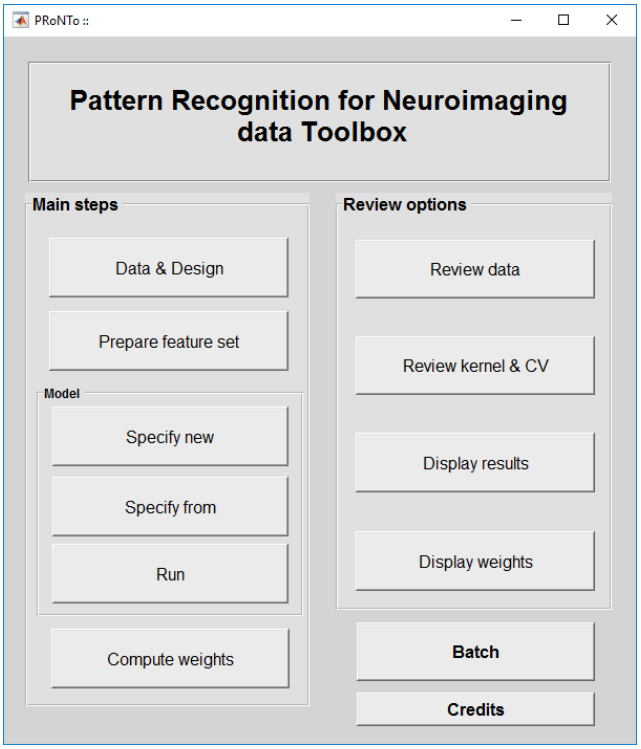


Figure 13: Main PRoNTo window displaying the new option to Specify from.

Interpolated EEG with SVM

First we select 'Specify new' and load the PRT. The main change in this window is that it now has lists of unselected (left) and selected (right) feature sets to include in the model. When more than one feature set is selected, multiple kernel machines will be added to the 'machine' popup menu.

We first add the 'EEGinterp' feature set by clicking on it in the left list. It then passes on the right and will be included in the model.

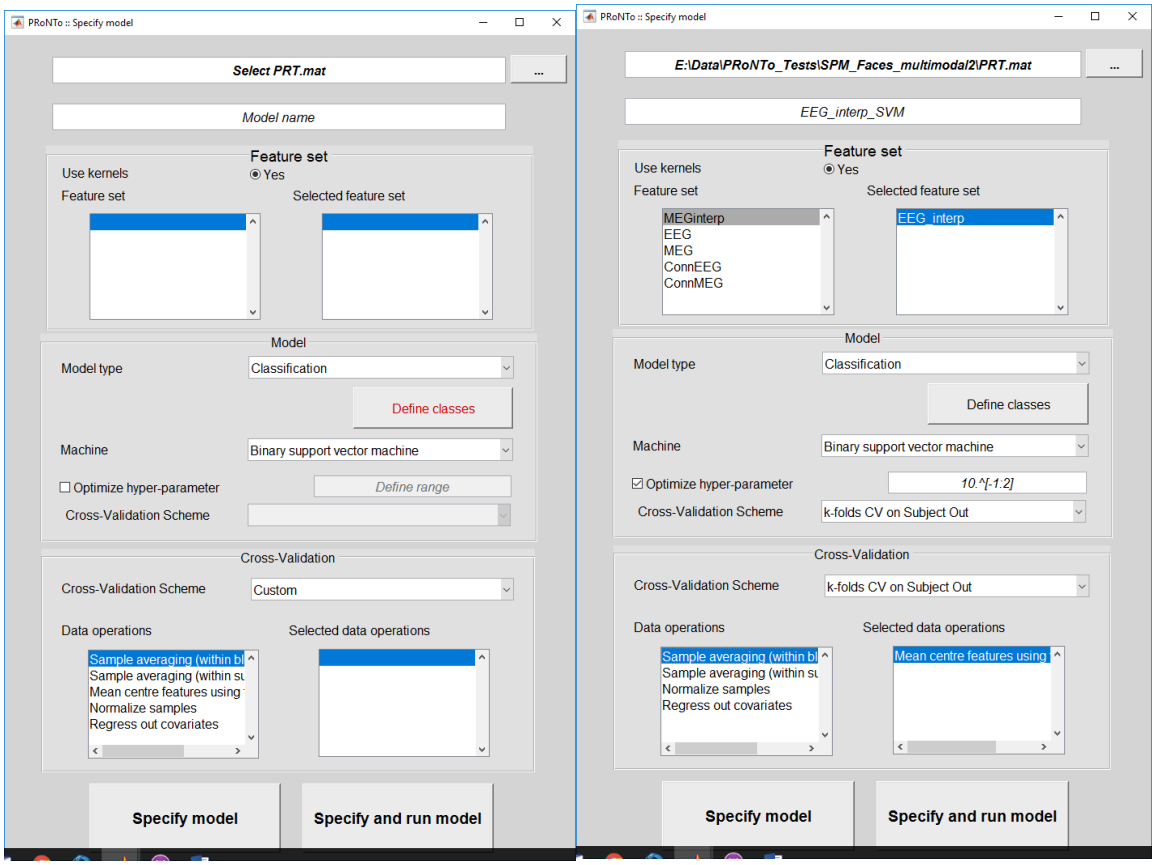


Figure 14: Model specification with feature set list (left). Example modelling parameters selected for the EEG_interp_SVM model discriminating Famous versus Scrambled faces.

We then choose simple modelling parameters, specifying class 1 as 'Famous' using all subjects and class 2 as 'Scrambled' using all subjects. A novelty in the class specification is the possibility to randomly subsample the over-represented class (to match as close as possible the under-represented class). However, we use only condition per class and one image per subject, so our dataset is balanced.

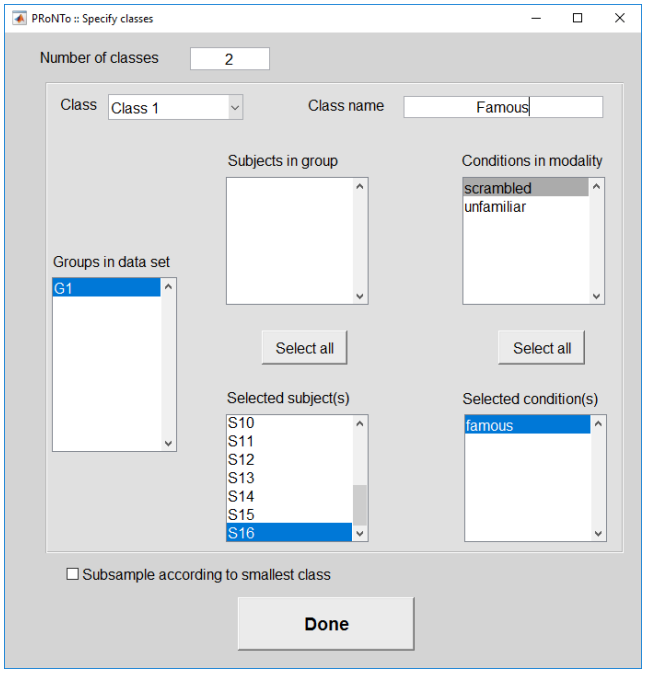


Figure 15: Class selection now includes a 'Subsample' option, that is not used in the present model.

We then choose a SVM binary classifier, with hyper-parameter optimization (C is '10.^[-1:2]' and nested CV is 'k-folds on Subject Out' with k=4). The outer CV is also 'k-folds on Subject Out', with k=4. **Note**: 'Leave One Subject per Class Out' or its k-folds variant is not appropriate here as we need to leave out all images from a single subject. Using the 'per Class Out' CV will throw an error and ask to select 'Leave Subject Out' instead. Finally, we add the 'mean centering' operation and then 'Specify and Run' the model.

This operation could be repeated across the different modalities. However, this is cumbersome. In addition, if we had used the subsampling option, the different models would very likely select different samples, which would make them harder to compare. Instead, we can now copy the model we just defined.

Other models via 'Specify from'

The main interface has been modified to add a 'Specify from' model option. This will launch a window that is very similar to the 'Specify model' window, but with a couple of changes: there is an extra popup menu that allows to select which model to copy from. Some fields have also been disabled to ensure comparability of the models: only the feature sets, the machine (with its hyper-parameter optimization) and the operations can be modified. Classes/Regression selection and outer CV options cannot be modified.

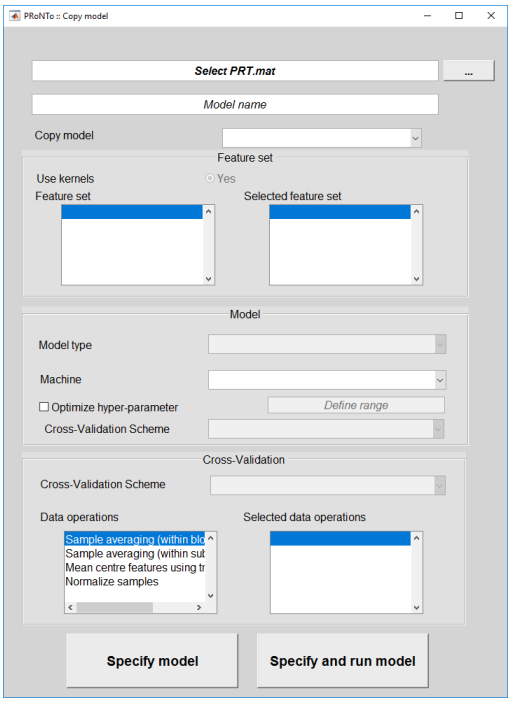


Figure 16: Specify from, new in v3. Allows to copy most parameters from a previously defined model.

To create the other models based on each modality, we simply need to load our PRT and select the 'EEG_interp_SVM' model to copy from (it is by default the 1st model in the PRT). We need to specify a model name for this new model. In the present case, we only want to change which feature set is used, so we click on 'EEG_interp' to deselect it and then click on the modality we'd like to add. We leave other parameters intact.

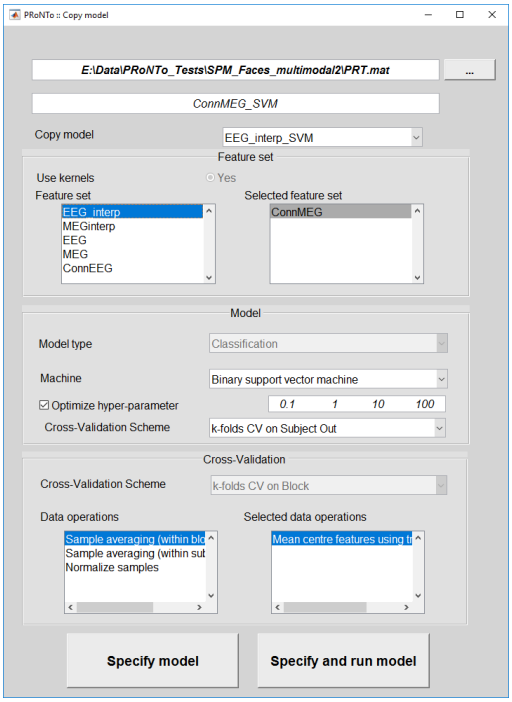


Figure 17: Copy model example for the ConnMEG feature set.

We can then Specify and Run this model, and repeat for each modality.

MKL model on modalities

We would now like to know which modality contains most information for the 'Faces' versus 'Scrambled' comparison, and see if using different features improves performance when compared to the single modalities. We can use the 'Copy' model option again, but this time we add all feature sets to the model and change the machine to the 'L1-Multiple Kernel Learning'. We also need to add the 'Normalize samples' operation to give a fair chance to each modality as they do not have the same number of features.

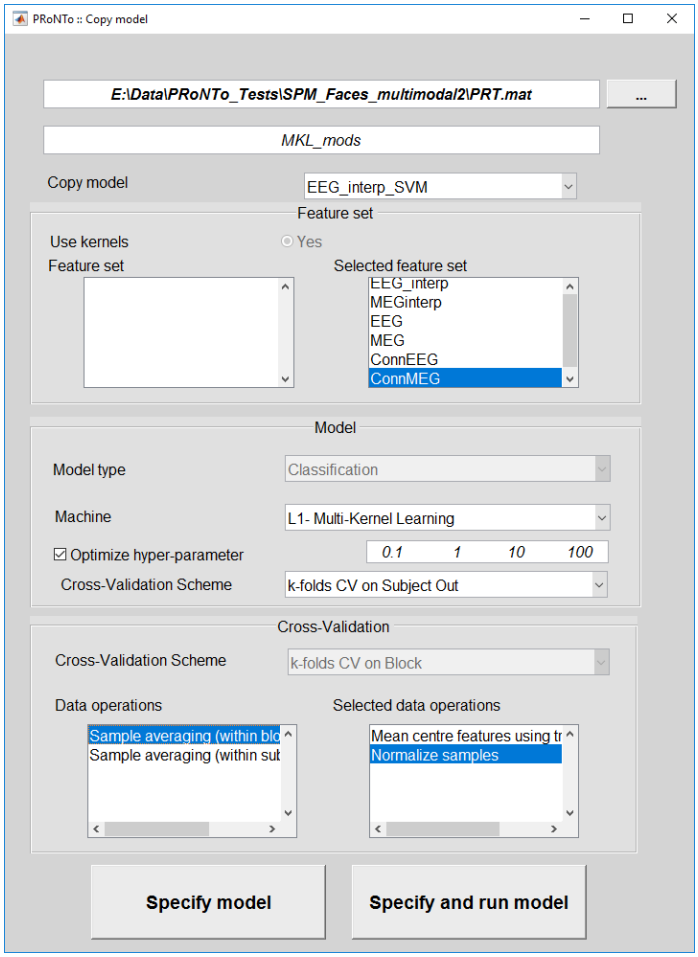


Figure 18: MKL model on all modalities.

Model results

A very important change in v3 is that model performance is computed within folds, and then the stats are averaged across all folds. In v2, stats were computed within folds but then predictions were concatenated across folds to produce the model-level stats. This has been advised against as being over-optimistic. It also provides stats for a concatenated model that was not estimated, by concatenating predictions from different models (one model per fold).

This change is reflected in the 'Display results' window: the list of plots available across folds is not the same as the list of plots available within folds.

Across folds, i.e. the 'Average', the user can display:

- 'Accuracy distribution': the balanced accuracy in each fold plotted as a violin plot, with its mean displayed in red. If permutations were estimated, the left side of the violin corresponds to the accuracy distribution (with additional histogram), while the right side corresponds to the distribution of average balanced accuracy with permuted labels.
- 'Predictions': same as v2.
- 'ROC': in v2, the ROC was built from the concatenated function values. In v3, we now build the average ROC curve and display its standard deviation across folds as shaded area.
- 'Influence of the hyper-parameters': similar to v2, but added scatter plot representing the balanced accuracy within each fold for each value of the hyper-parameter.

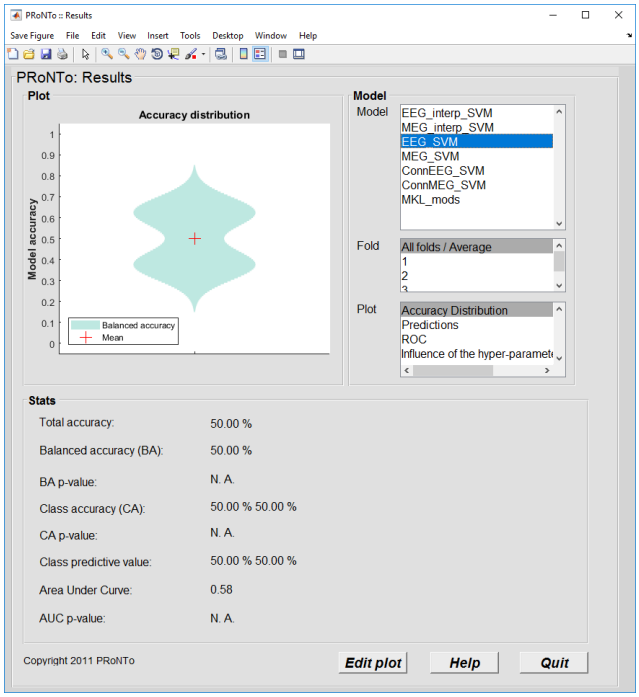
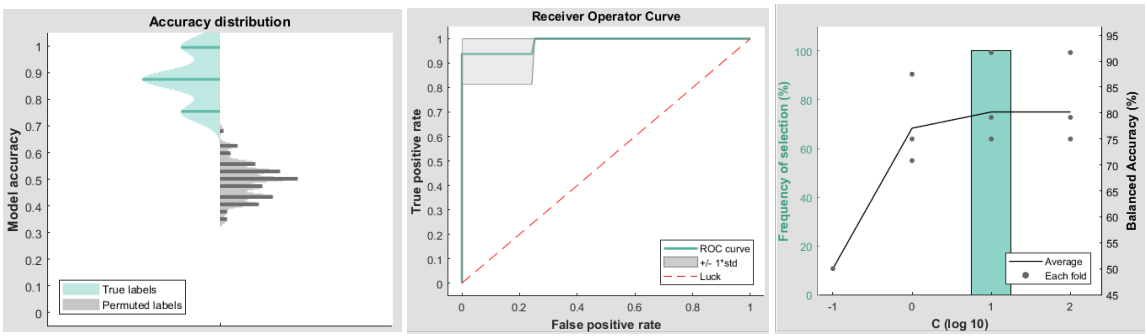


Figure 19: New plots to review classification performance. Left: overall display, showing the 'Accuracy distribution' of the EEG_SVM model when no premutations were estimated. Bottom left: same for EEG_interp_SVM when permutations were estimated. Bottom center: ROC average curve with standard deviation. Bottom right: Scatter plot of fold performance for hyper-parameter optimization.



Within folds, the plots are the same as in v2. Minor changes were performed to improve their appearance.

Run model

One change in v3 is that the permutation computation has been moved from the 'Display Results' window to the 'Run Model' window. This is more consistent with the batch and allows to run permutations for multiple models automatically from the GUI. There is also the option to save all the parameters from the permutations, as in the batch.

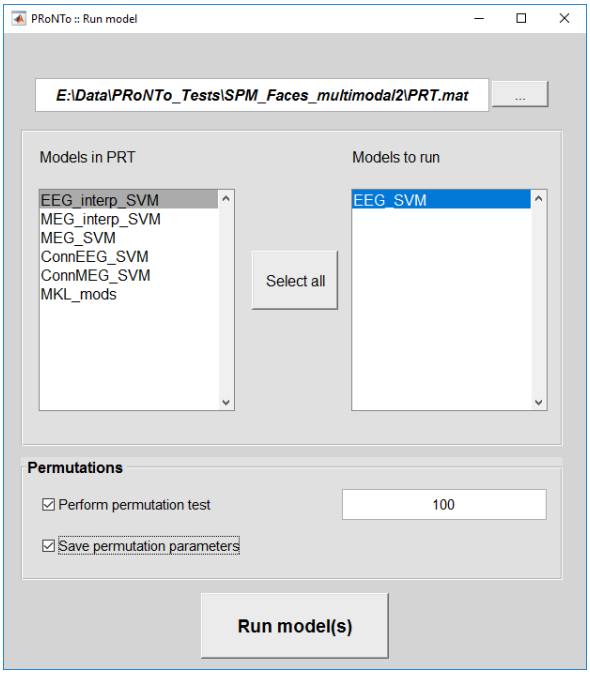


Figure 20: Run model window with added 'Permutations' panel.

Weight computation

Accordingly, the weight computation window includes a tick box to allow building the average weights for each permutation. As in the v2 batch, these are saved in another folder.

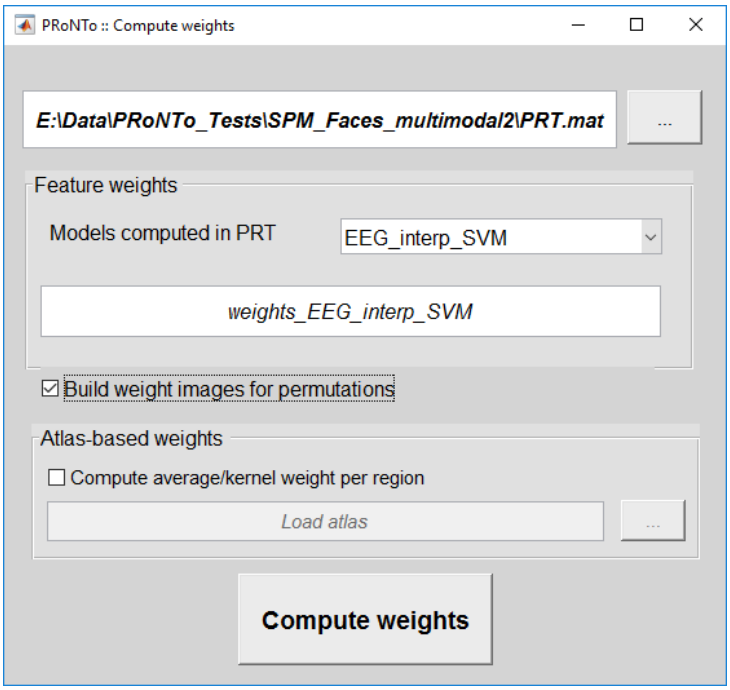


Figure 21: Compute weights allows for building weight images from permutations.

The window will also allow to select one atlas per feature set included in the model. To this end, simply select multiple images or .mat, in the correct order. If the 'Computer average/kernel weight per region' option is selected, feature sets that were built using an atlas will automatically use this atlas. If this option is selected and no atlas is specified for some feature sets, only the weights will be built, not the weights per region/kernel.

Please note that loading an atlas for weight summarization is not available for MEEG objects (only nifti and .mat).

Display weights

Weights can be displayed for all modality types. For MEEG and .mat, weights are displayed as bar graphs (with the color of the bar representing the amplitude of the weight) for 1D objects (e.g. the connectivity vector used for .mat).

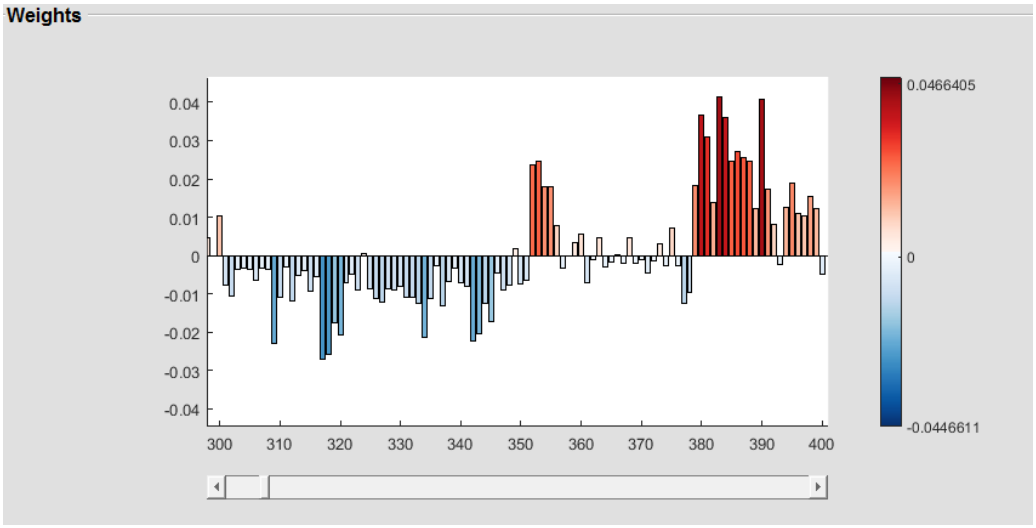


Figure 22: Weights for .mat 1D features, example from the MEG connectivity features used in the MKLmods model.

For 2D objects, the display shows the matrix, with y-axis as the 1st dimension and x-axis as the 2nd dimension (after squeezing out dimensions of size 1), with the color of a (x,y) pair displaying the magnitude of its weight.

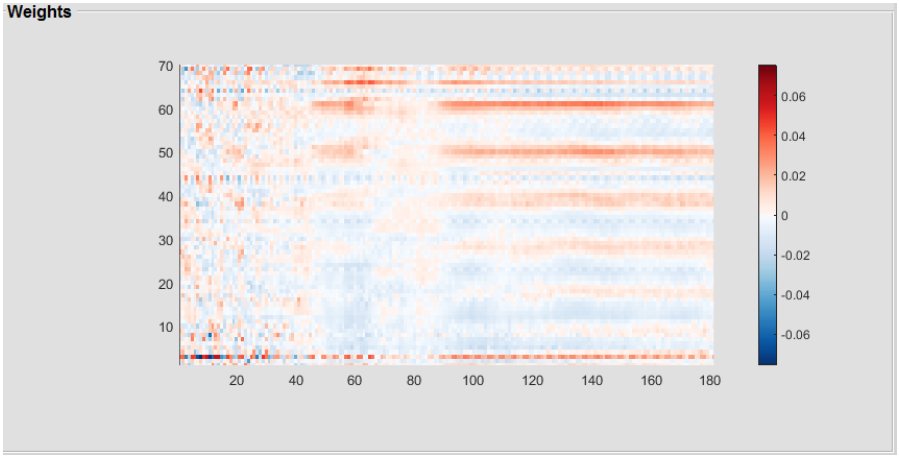


Figure 23: Weights for the EEG features, with channels on the x-axis and time points on the y-axis.

Nifit images are displayed as before, except a small change in the color map. When building the weights from the MKLmods model, we can also look at the feature set contributions. It looks like the interpolated EEG has the largest contribution to the model, followed by the EEG and MEG traces. This can be due to the interpolated EEG smoothing some of the anatomical variance in electrode placement, while EEG and MEG keep subject-specific information.

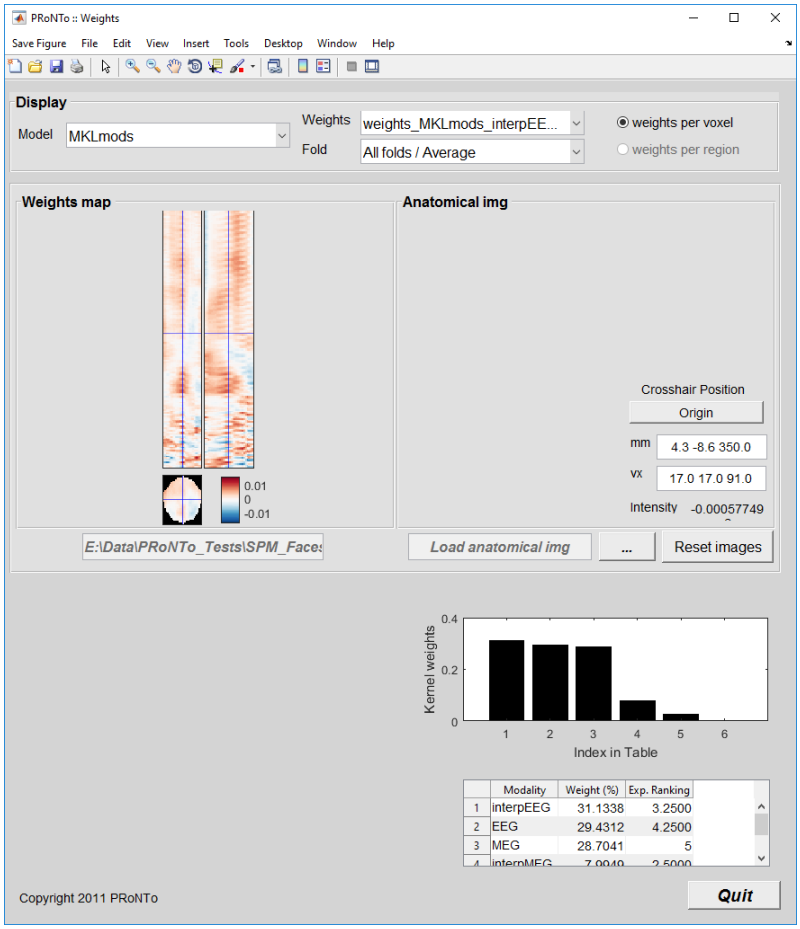


Figure 24: Weights for interpolated EEG and table of kernel contributions.

Using an atlas with .mat

As for nifit, an atlas can be used with .mat and MEEG files. For MEEG files, the atlas is implicit, resulting from the selection of channels, time points and kernels on which to build the kernels on. However, for .mat, PRoNTo is agnostic of the type of features entered, so the atlas needs to be built by the user. As an example, we built an atlas for the EEG connectivity derived modality. We defined 11 'networks' that the channels could belong to based on their anatomical position (e.g. 'orbitofrontal', 'occipital', 'left-parietal', 'central-parietal', …). The atlas then builds ROIs from pair-wise network interactions (i.e. 'orbitofrontal-occipital', 'orbitofrontal – left-parietal', …) leading to 66 ROIs (11*10/2 between networks interactions + 11 within-network interactions). The atlas looks as follows (2D version but only the upper triangular part is saved):

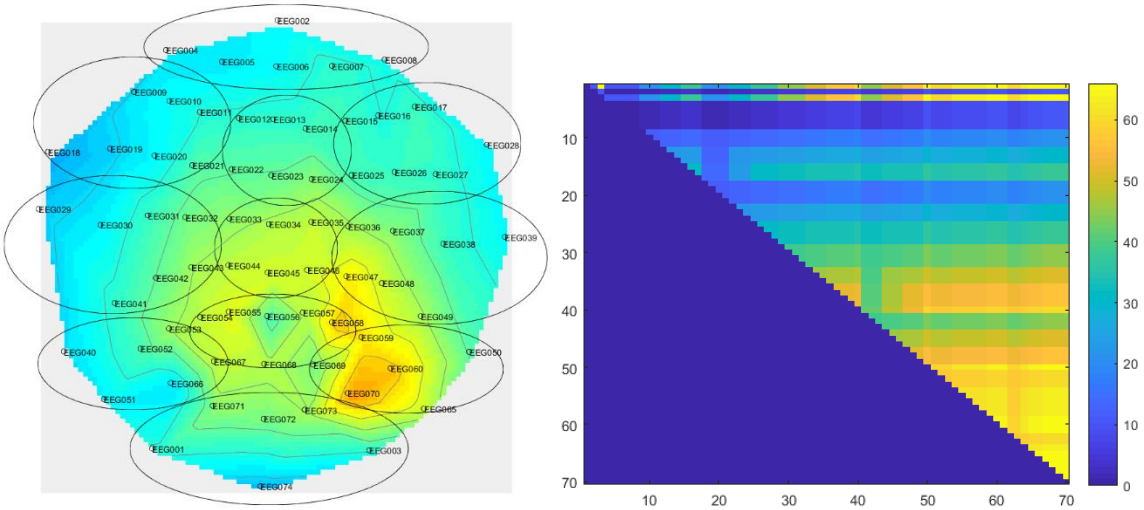


Figure 25: Atlas for EEG connectivity modality, defining interactions between and within 11 'networks' (identified based on their anatomical position as displayed on the left).

Going back to the feature set step, we can build another .mat feature set (called ConnEEGMK) including the ConnEEG modality and ticking the 'Build one kernel per ROI' box. We can load the designed atlas and PRoNTo will automatically gather ROI labels if a label file (called 'Labels_*atlasname*.mat') is present in the same folder and contains a 'ROI_names' variable.

Model specification can then be performed as in v2: select the ConnEEGMK feature set and define the classes, choose the L1- Multiple Kernel Learning machine and use 4-folds CV on subjects out for inner and outer CV. Add the mean centring and normalize samples operations before specifying and running.

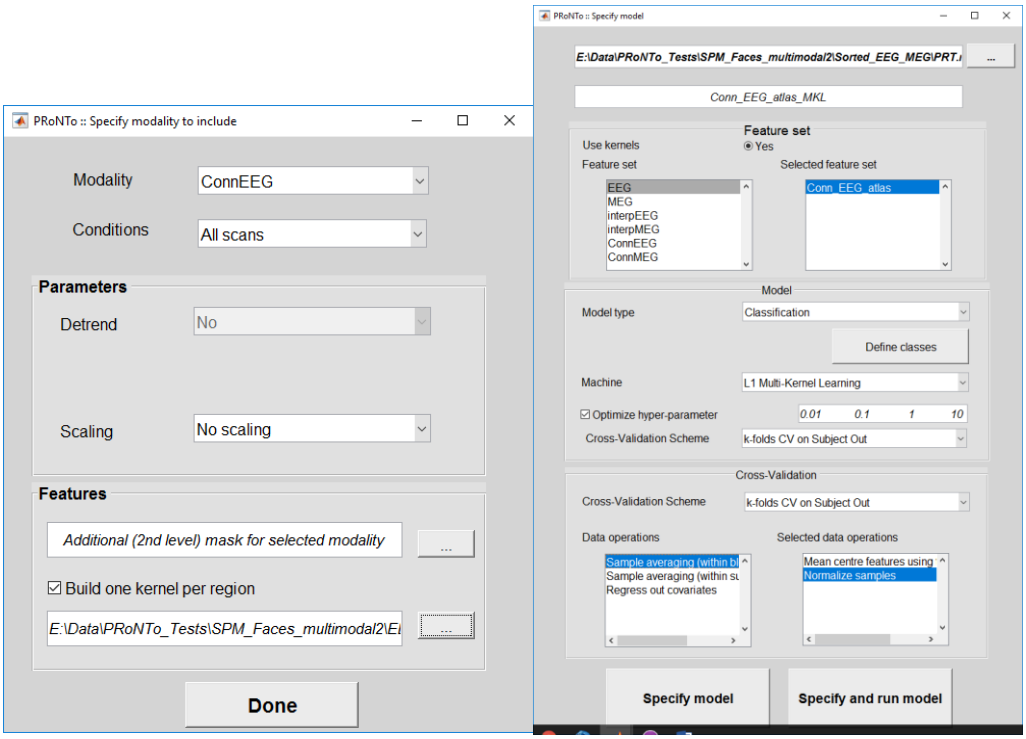


Figure 26: Build feature set and specify model for MKL on ROIs from .mat.

The model does not perform very well (56.25% balanced accuracy) for this modality taken on its own. For information, the same model using SVM (i.e. discarding the atlas information) leads to 50%

balanced accuracy. Both models display a large variance across folds and would not be considered as significantly discriminating between Famous faces and scrambled faces.

We can compute the weights for this model and select the option to build the weights per region. This will build 2 .mat files: one including the weights per feature, one including the weights per ROI. The latter gives the same value (i.e. its kernel contribution) to each feature within a ROI. The display results window shows the table of kernel contributions.

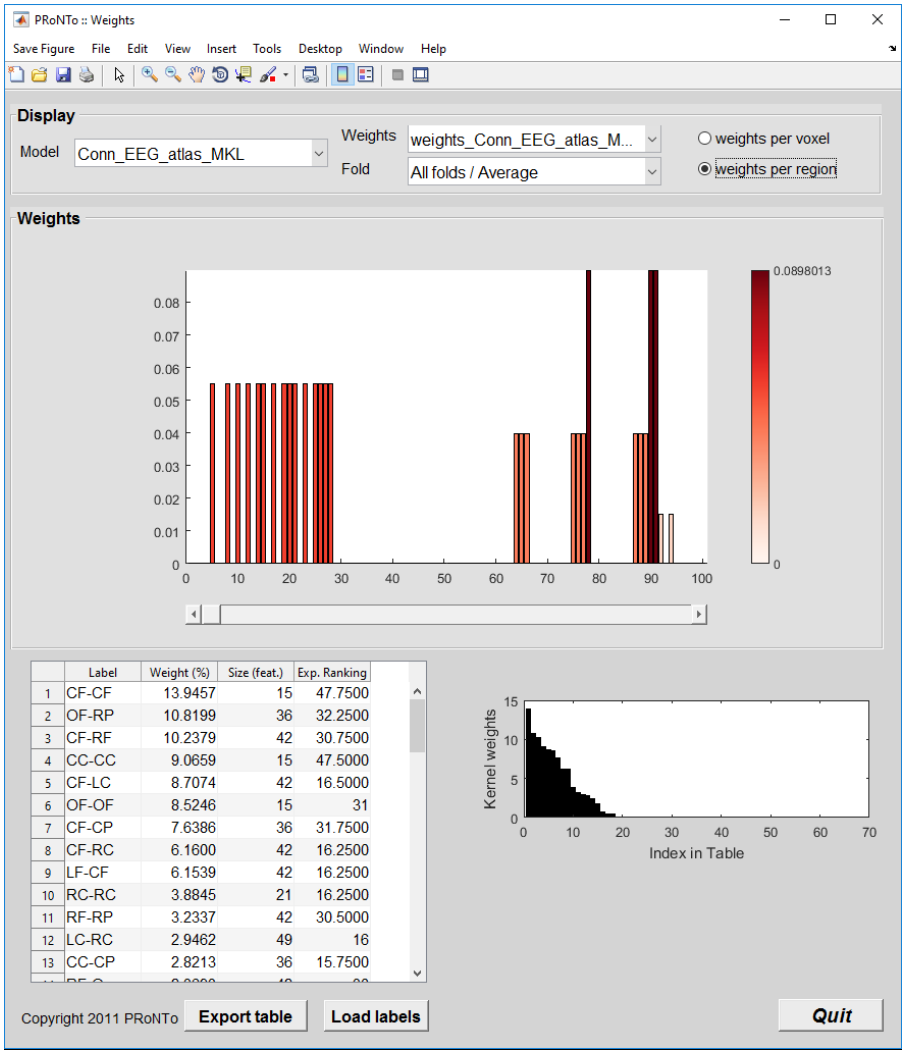


Figure 27: Display weights per ROI from MKL model on network interactions.

The weight file can then be postprocessed for better visualization. This has to be performed outside of PRoNTo as PRoNTo does not know what type of data the .mat represents (i.e. it doesn't know this is connectivity data).

As an example, we have summarized the weights per region in a 11*11 matrix instead of the 70*70 matrix. Furthermore, a schemaball plot (code modified from Matlab's file exchange schemaball submission, in appendix) was derived. It depicts the contribution of each between-network interaction to the classification as a colored curve and the within-network contributions as node color and size.

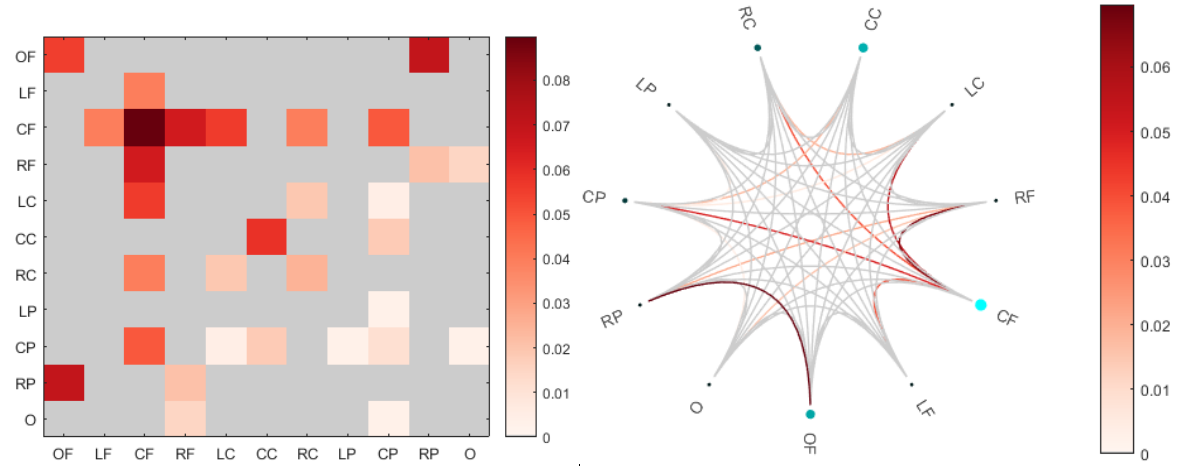


Figure 28: Summarized ROI weights across networks, displayed as a matrix (left) and as a schemaball (right). The size and brightness of the network marker displays the within-network contribution while the curves display the between-network contributions. Grey lines or matrix entries represent 0 contribution.

Batch

The new functionalities have been reflected in the batch.

Data and design

When adding a modality, the user now has to specify its 'Data format' as either nifti, MEEG or .mat. As the batch is agnostic of the user's choices, all design options are available for each type. Please make sure to use the appropriate option (i.e. loading an SPM.mat is only for nifti, and 'Events in file' is only for MEEG).

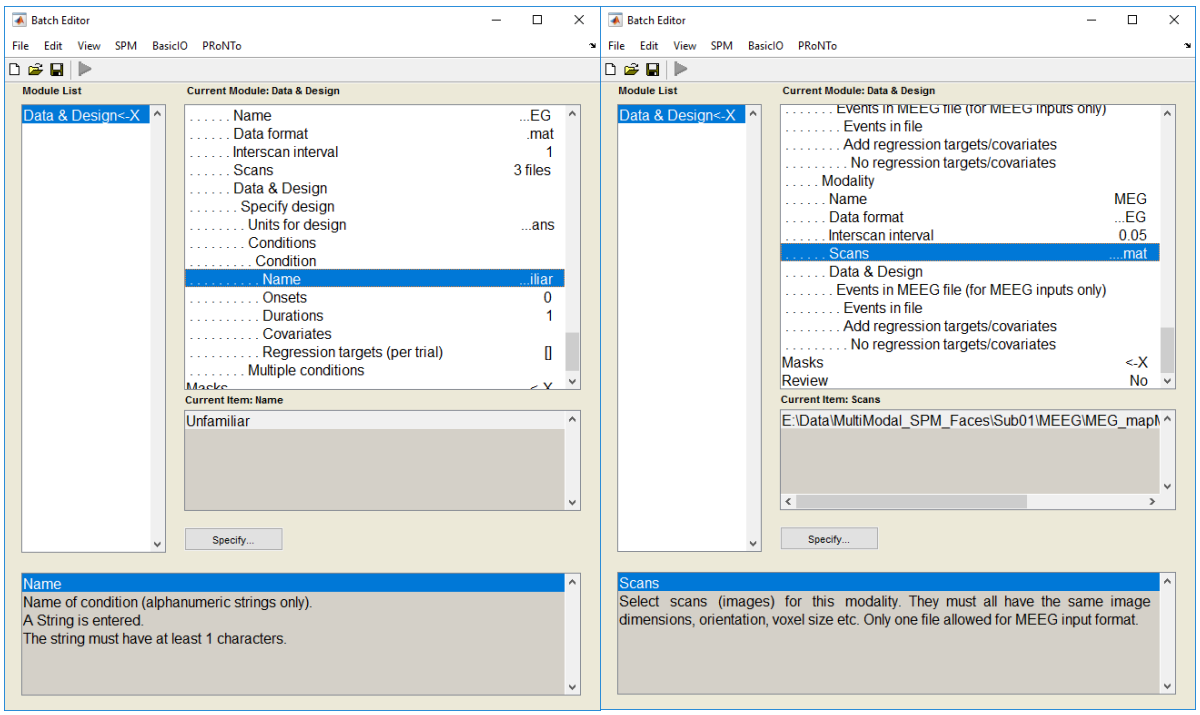


Figure 25: batch for .mat modality, specifying conditions and for MEEG modality, getting design from file.

For the same reason, one mask has to be entered per modality, independent of its type. Only ‘nifti’ format requires a file, .mat and MEEG just need a modality name. This is in contrast with the GUI that only requires masks for nifti files (as it derives the masks for the MEEG and .mat automatically).

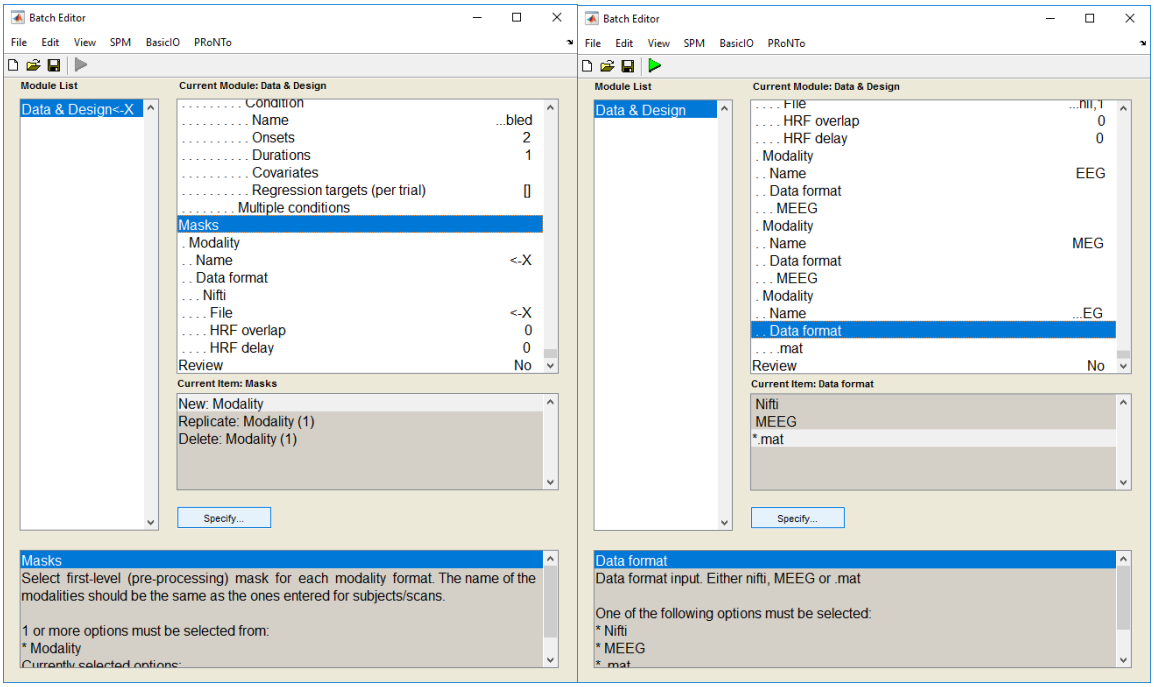


Figure 26: mask specification for nifti (left) and .mat (identical for MEEG).

Feature set

At the feature set step, the user needs to select which feature type to build. The nifti modality is identical to v2, while .mat is very similar to it (only detrend is not an option). The MEEG modality is different as it allows to play with the dimensions of the data (channels, time points or frequency bins), with multiple kernels or averaging. It contains the same options as the GUI and has the same channel specification options as SPM (including a channel file with variable called ‘Labels’).

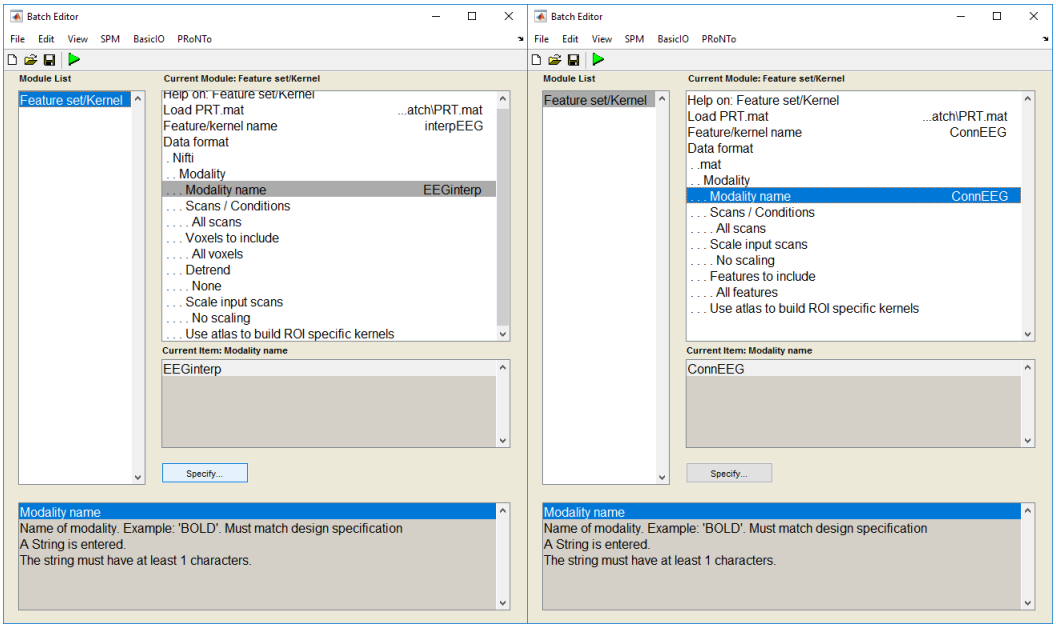


Figure 27: nifti (left) and .mat (right) feature set builder.

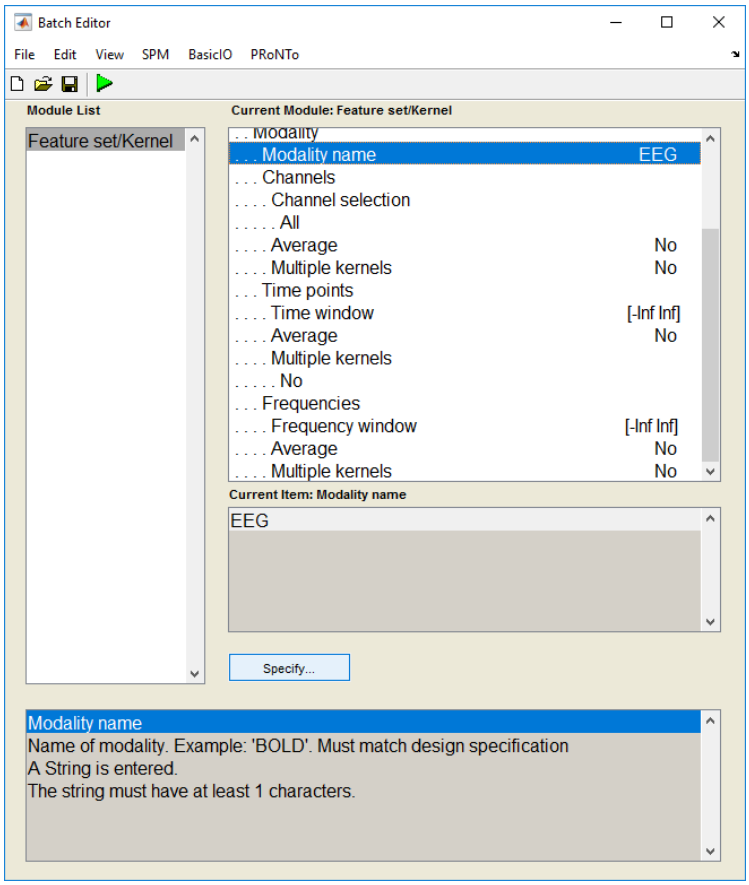


Figure 28: MEEG modality builder.

Model

At the Specify model, two minor changes can be observed. First, Feature set is now a branch and multiple names can be added to be combined (with MKL for example). Second, at the class definition, there is an option to subsample the most represented class.

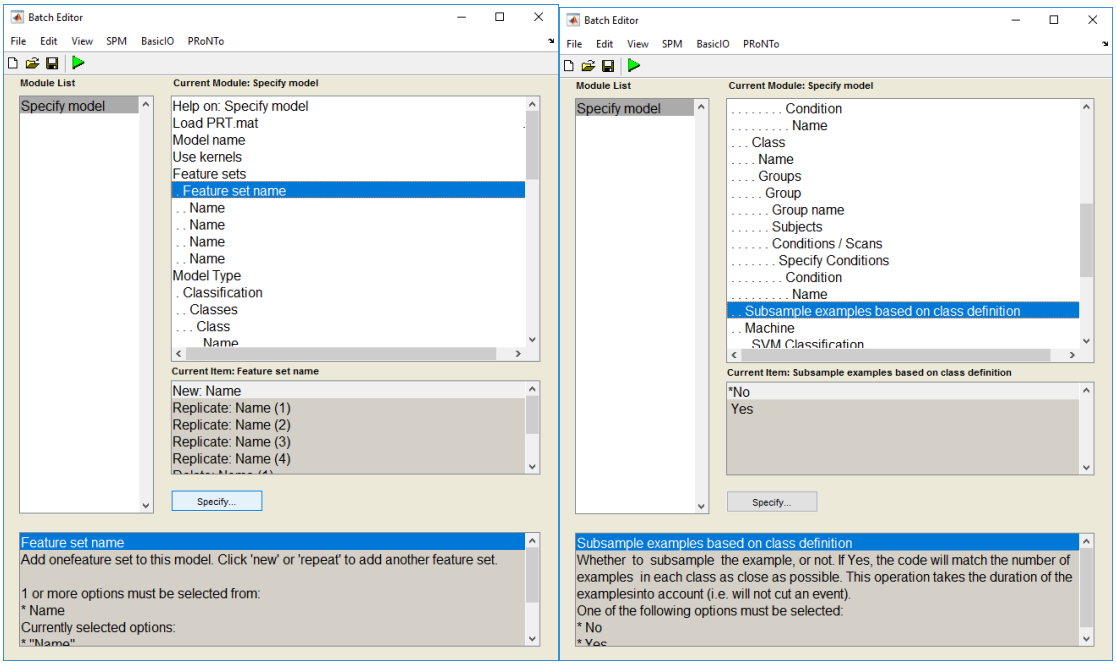


Figure 29: new model functionalities in batch.

Weights

The options are the same as previously. In addition, it is possible to load multiple atlases (should be one per feature set), if desired.

Tutorial: Classification of semi-simulated ECoG data

Data

The data consists of a semi-simulated data set from a single subject in electrocorticography (ECoG, a.k.a. intracranial EEG). It was built from a 5 minutes rest session to which a fake design was added: 2 conditions, 'A' and 'B', randomly interspersed. The pre-processing discarded noisy and pathological channels, leaving 38 'good' channels. A rectangular signal window was added to events from 'A', after extraction of the power in the High Frequency Broadband (HFB). The magnitude of the signal to add was calculated based on the A vs B contrast. The chosen signal (A) to noise (B) ratio varied in the original experiment but is fixed to 3 in the present case. Similarly, the simulated signal was not added to all channels but to roughly half of them. After artefact rejection, 60 events of 'A' and 56 of 'B' remain for classification. This dataset was published in:

J. Schrouff and J. Mourão-Miranda, "Interpreting weight maps in terms of cognitive or clinical neuroscience: nonsense?," 2018 International Workshop on Pattern Recognition in Neuroimaging (PRNI), Singapore, 2018, pp. 1-4. doi: 10.1109/PRNI.2018.8423944

And the code to generate this data is available at: https://github.com/JessicaSchrouff/Simulated_ECoG

GUI

Data and Design

The data set needs to be entered into PRoNTo using the Data and Design window. As this is a single-subject analysis, enter one group (e.g. 'G1') and one subject. Add a new modality and give it a new name (e.g. 'ECoG'). Set the type of the modality as 'MEEG' and select the .mat file corresponding to the ECoG simulated recording.

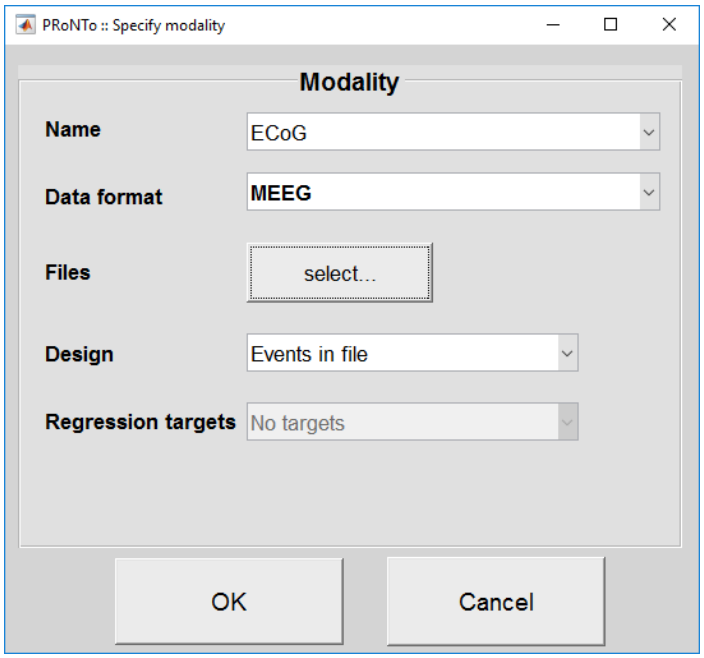


Figure 1: Specify MEEG modality

This will automatically load the experimental design from the file. It can be reviewed by clicking the 'Events in file' option:

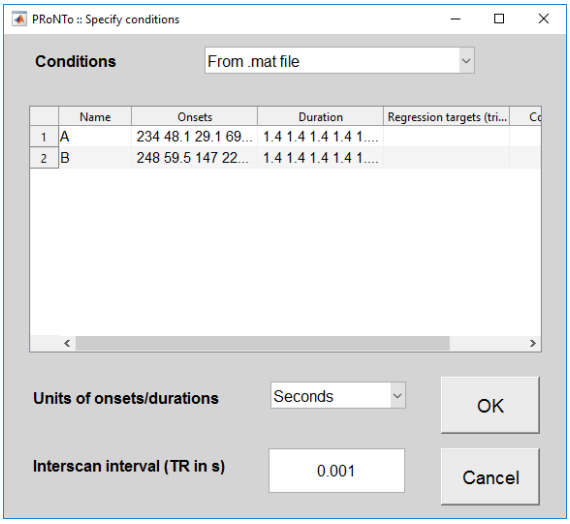


Figure 2: Events are automatically loaded form the file.

After clicking 'OK' on both windows, the data can be saved into a PRT.mat and the main window looks like this:

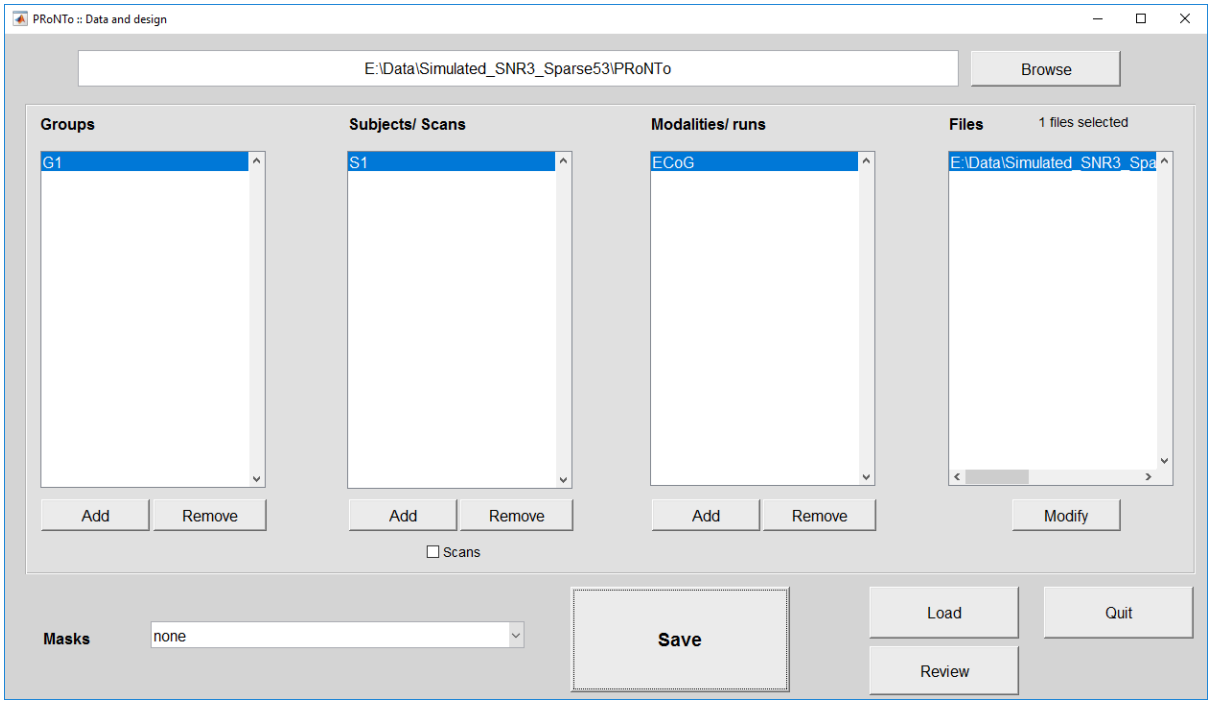


Figure 3: Data and Design window after subject was entered and saved.

Please note that there is no need to specify a mask file for the MEEG modality type. The information entered can then be reviewed:

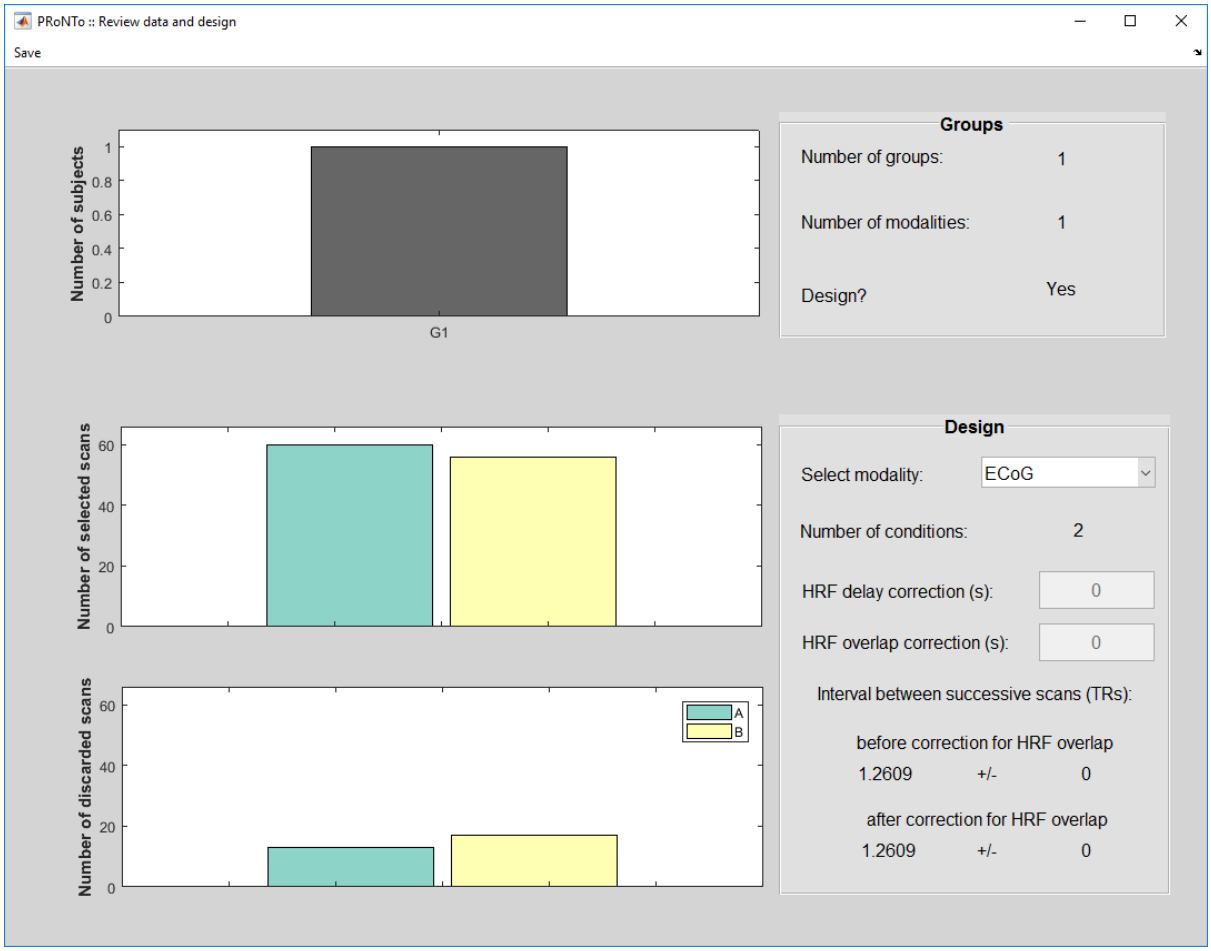


Figure 4: Review Data and Design window.

As we can see, we have one subject in one group that contains one modality (ECoG). There are 2 conditions (A and B). The 'selected scans' reflect the 'good' epochs, while the 'discarded scans' reflect the epochs marked as 'bad' in the SPM file.

Feature set

This step is probably the most different from all other new functionalities in v3. It reflects the different dimensions of the data and allows to 'play' on them.

Click on 'Prepare feature set' in the main window and load the PRT. As there is only one modality of type MEEG, the corresponding MEEG modality window will be launched directly:

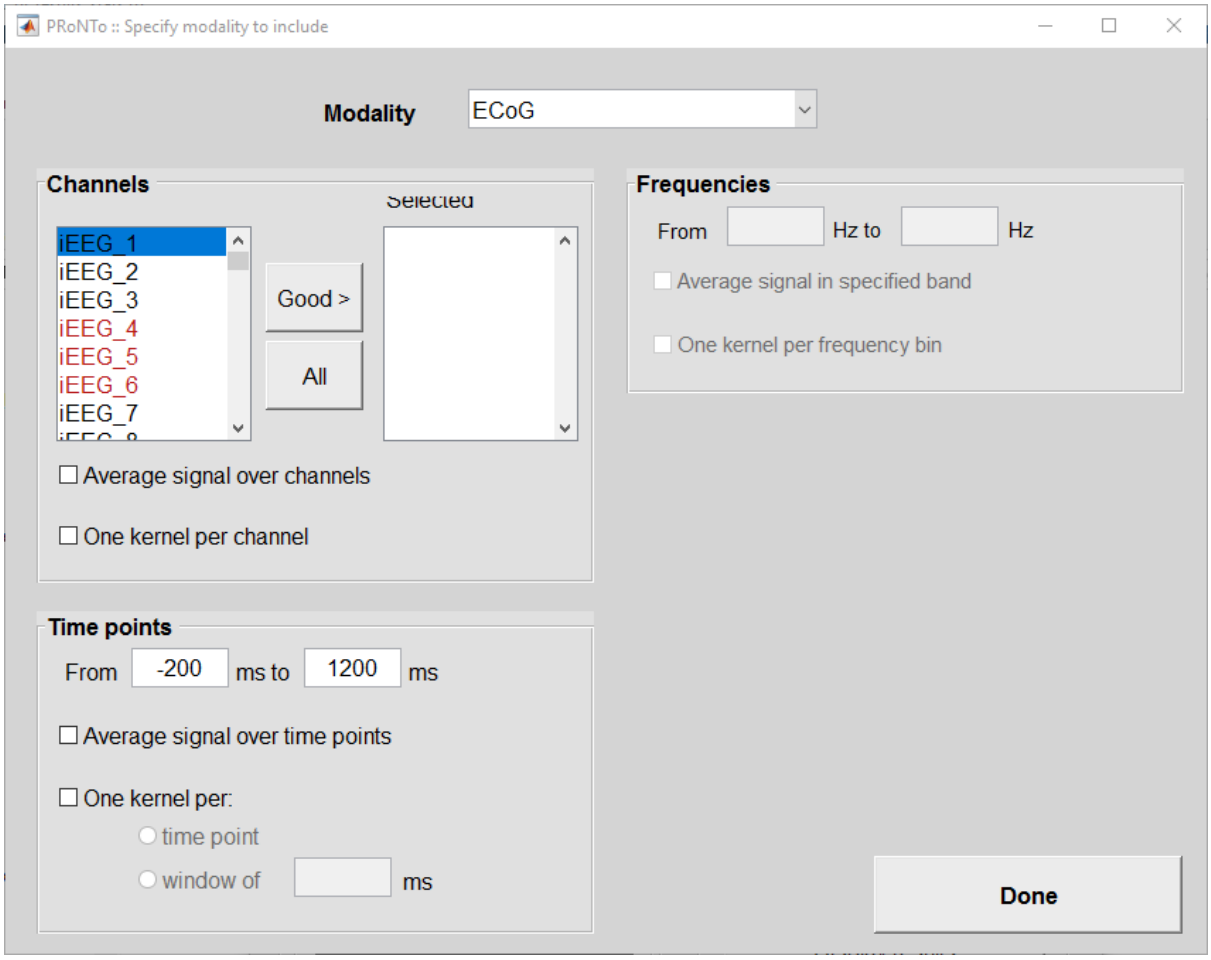


Figure 5: MEEG modality window in feature set step.

There are 3 main panels, corresponding to the 3 dimensions an MEEG file can have:

- Channels:
    - The list of channels is displayed on the left panel, as unselected. To add a channel to the feature set, click on it. It will then appear on the right panel. Alternatively, you can select 'All' channels or all 'Good' channels. On Windows OS, the 'bad' channels appear in red in the list.
    - Average signal over channels: this option will average the signal over all selected channels. This might be useful to obtain an average trace across a specific region of interest.
    - One kernel per channel: whether to build one kernel per channel (ticked) or not. This option corresponds to building an atlas with each channel as a 'ROI'.
- Time points:
    - Similarly, the time window is displayed and can be amended. When entering a value in ms, PRoNTo will identify the closest corresponding time point.
    - The signal can be averaged across all selected time points.
    - One kernel can also be built, either per time point or per sub-windows of xx ms (user-specified). PRoNTo does not discard kernels with less than a few time points, so to avoid big imbalances in the feature numbers across kernels you should ensure you provide right window end (e.g. 100ms subwindows between -100 and 1100ms, the end time point should be 1099ms, otherwise a kernel with only one time point will be created, see batch tutorial below for example).

- Frequencies:
    - Similar to time points, the range of frequency bins included in the data can be viewed and specific sub-ranges selected.
    - The data can be averaged over a frequency band, as specified above. This is useful to test multiple frequency bands without having to manually perform the averages as a pre-processing step.
    - A kernel can also be built on each frequency bin.

Dimensions that are not present in the 1st file are greyed out and the corresponding panel cannot be accessed (e.g. frequencies in the present case).

It is not possible to use 'average' and 'multiple kernels' on the same dimension simultaneously.

In the present case, let's first build a simple feature set, using all 'Good' channels and the time points between -100 and 1100ms (note: the simulated window was added between 0 and 1000ms).

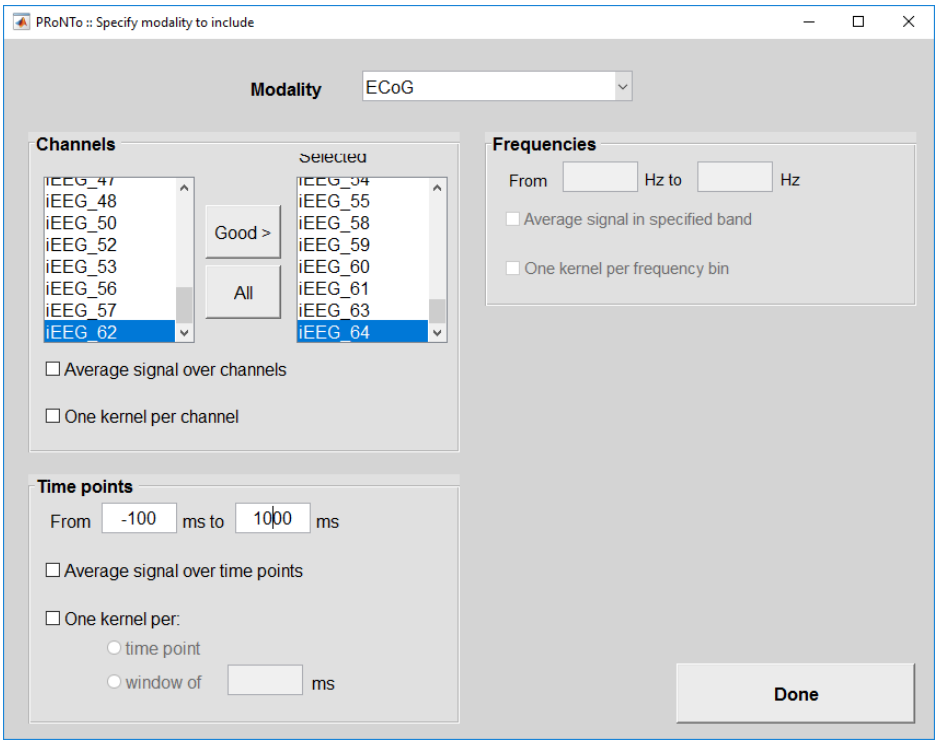


Figure 6: MEEG specification for feature set 'All'.

Clicking 'Done' sends us to the main feature set window, where we specify the feature set name (e.g. 'All').

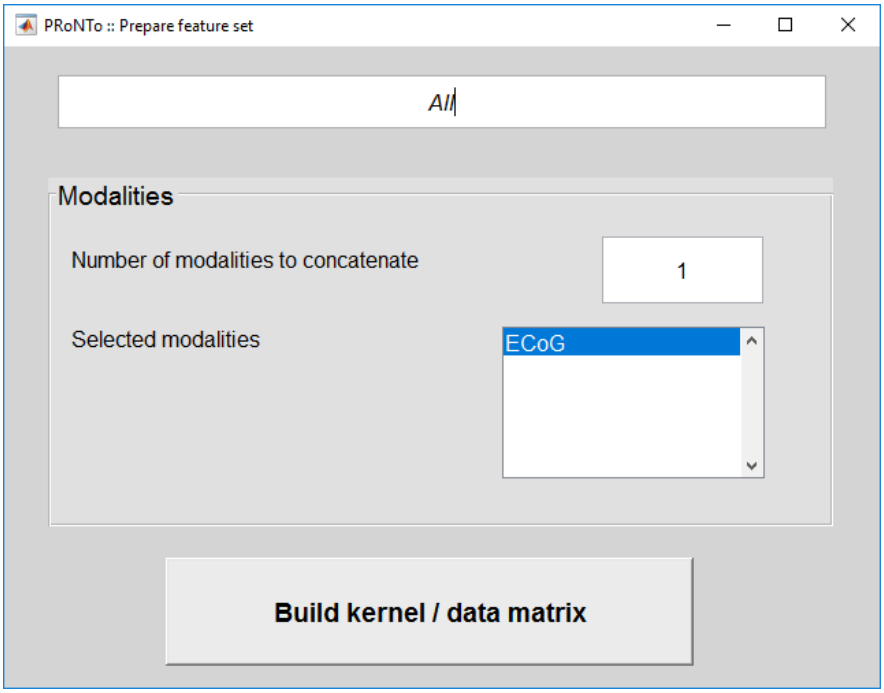


Figure 7: Main feature set window after ECoG specification.

Clicking on 'Build kernel/data matrix' builds the binary file array (as for nifti) with all the features, as well as the kernel(s) containing only the selected features. Hence, the MEEG modality window builds a second-level mask but implicitly (instead of the user specifying a file).

We will then repeat this operation but building one kernel per channel, for later use with L1-MKL machine, and name it 'MKChans' (for Multiple Kernels on channels). As displayed in the main window, 38 kernels were built, one for each channel containing all the time points between -100 and 1100ms around onset.

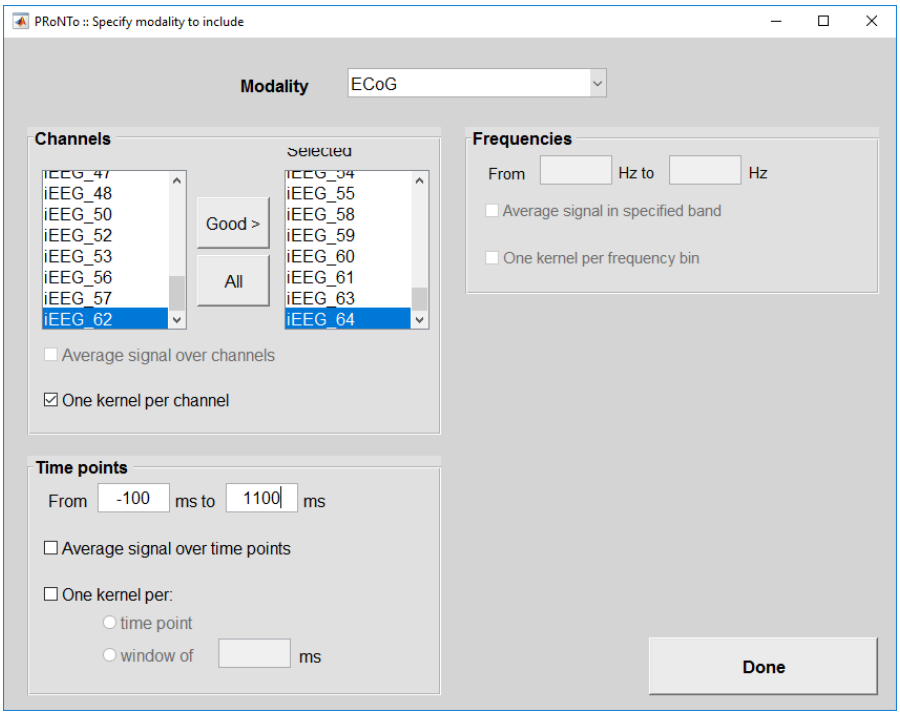


Figure 8: Specify MKChans feature set.

Specify model

We can now build two models discriminating A from B: one using SVM and the other using MKL on the channels. The model specification is very similar to PRoNTo v2.1 and there is nothing specific to MEEG at this stage.

For the SVM model, we chose the 'All' feature set and defined the classes as class 1 is A and class 2 is B. Although the imbalanced is small, we also select the 'Subsample according to smallest class' option to randomly subsample 56 epochs from A to match the number of B epochs.

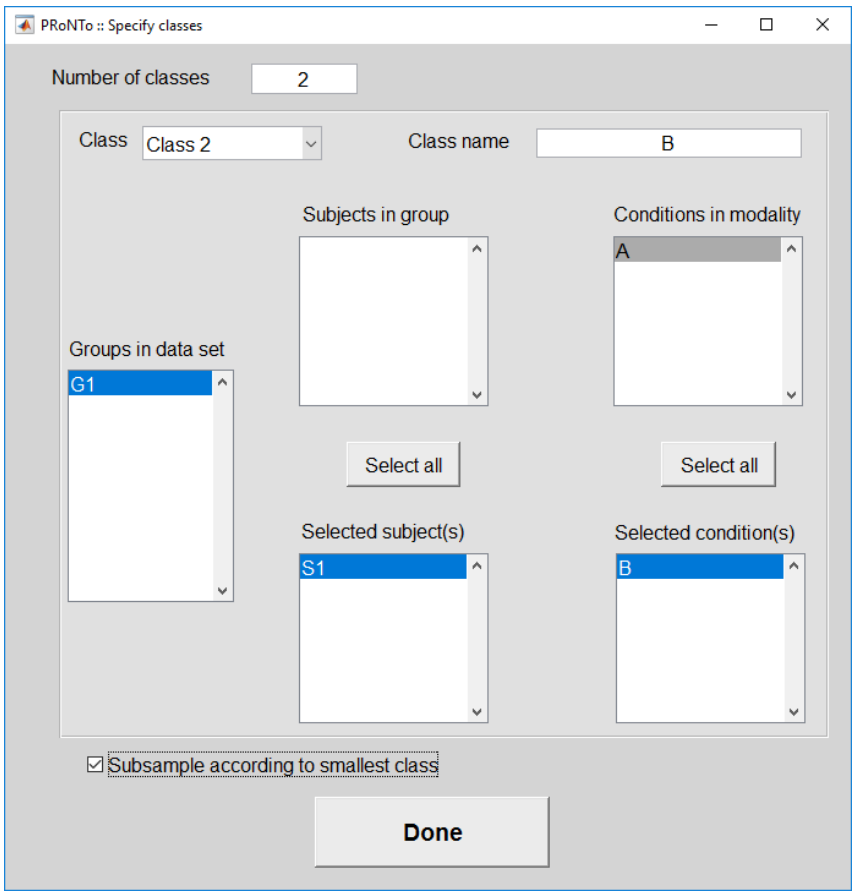


Figure 9: Specify classes.

We choose the SVM machine and optimize the hyper-parameter C (range: 10.^[-2:3]). The nested cross-validation scheme is 'k-folds on blocks per class out' with k=4, meaning that we will leave 25% of A and 25% of B epochs out for testing in each fold (approximate to modulo). We choose the same scheme for the outer CV with k=5. Finally, we mean centre the kernel and specify and run the model. The window looks like:

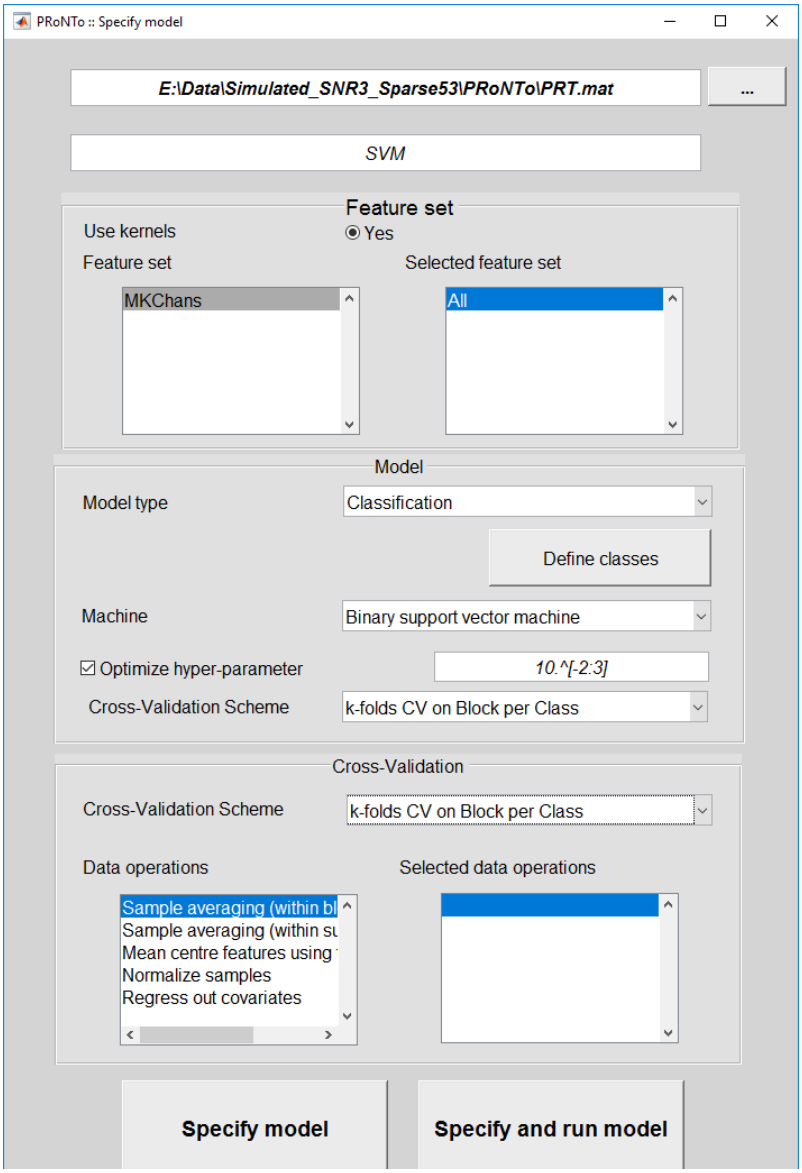


Figure 10: SVM model specification.

To specify the MKL model, we need to 'copy' the SVM model if we want to use the same samples as the random subsample option was selected. In this case, we click 'Specify from' and load our PRT.mat. The SVM model is loaded by default (as the first) and we can only modify the feature set, the machine (with hyper-parameter optimization) and/or the kernel operations.

We will use the MKChans feature set and change the machine to L1-Multiple Kernel Learning. We use the same hyper-parameter optimization as before, but need to add the 'Normalize samples' operation. Note: as the kernels all include the same number of features (i.e. the number of time points considered), this operation could also be left out.

The 'Specify from' window looks like:

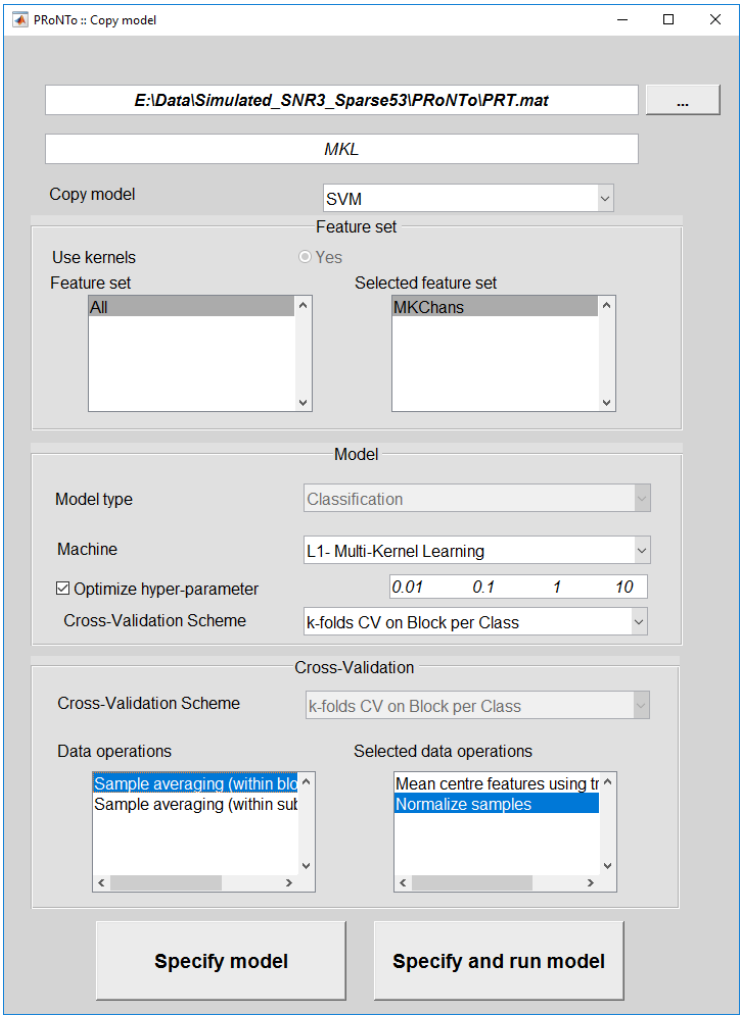


Figure 11: MKL model on channels.

Building weights

We can now build the weights for the SVM model. In this case, select the PRT, the SVM model and leave the other options as default. Please note that weight summarization is not available for MEEG data. A new file, in MEEG format, will be built containing the weights per feature. It will have the same dimensions as the input file but contain NaN where the second-level masks did not overlap with the data (i.e. the features not selected at the feature step stage).

Repeat the operation for the MKL model but this time tick the 'Compute average/kernel weight per region'. Two images are then built: one with weights per feature, one with weight per kernel. Again, the dimensions are identical to the dimensions of the input file.

Review performance

The SVM model leads to 93.86% balanced accuracy, while the MKL model leads to 97.35%. Their AUC is however the same (0.99).

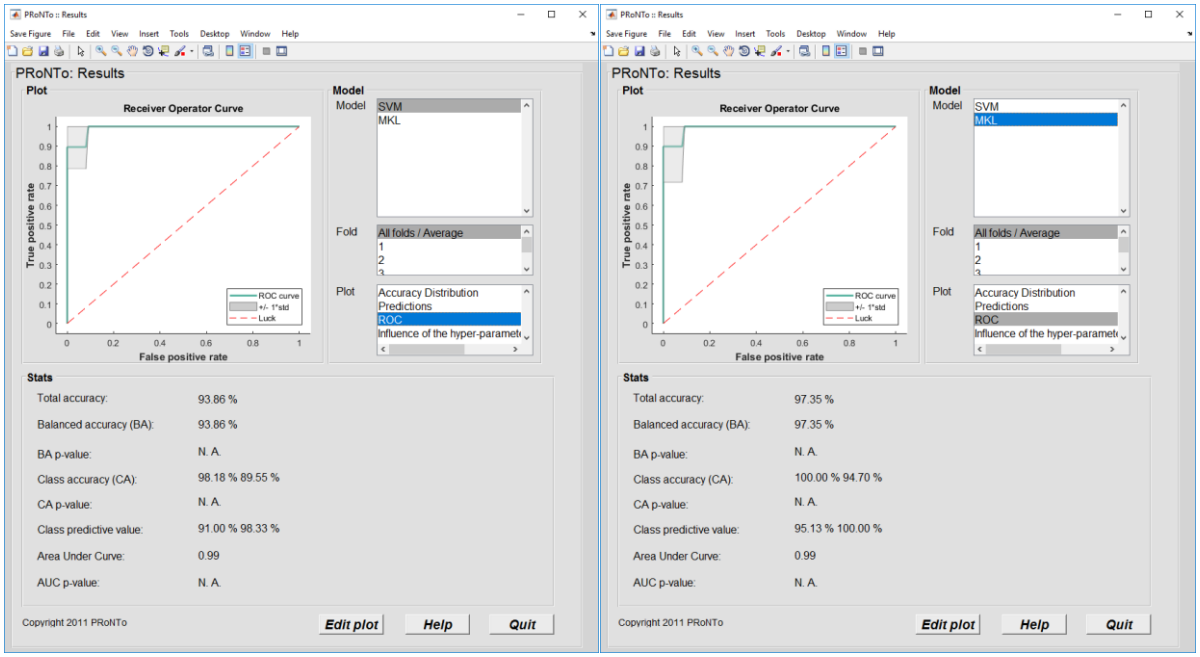


Figure 12: Model performance.

It is expected that MKL performs better as only ½ of the good channels contain signal of interest while the other half only contains noise.

Display weights

The weight display for MEEG is very basic, just a color-coded 2D matrix if there are 2 dimensions in the file (3D files are not displayed). We refer the user to SPM (or other software after conversion) for more advanced plotting.

The weights for the SVM model are displayed in the following figure. As we can see, channels are quite consistently either positive or negative.

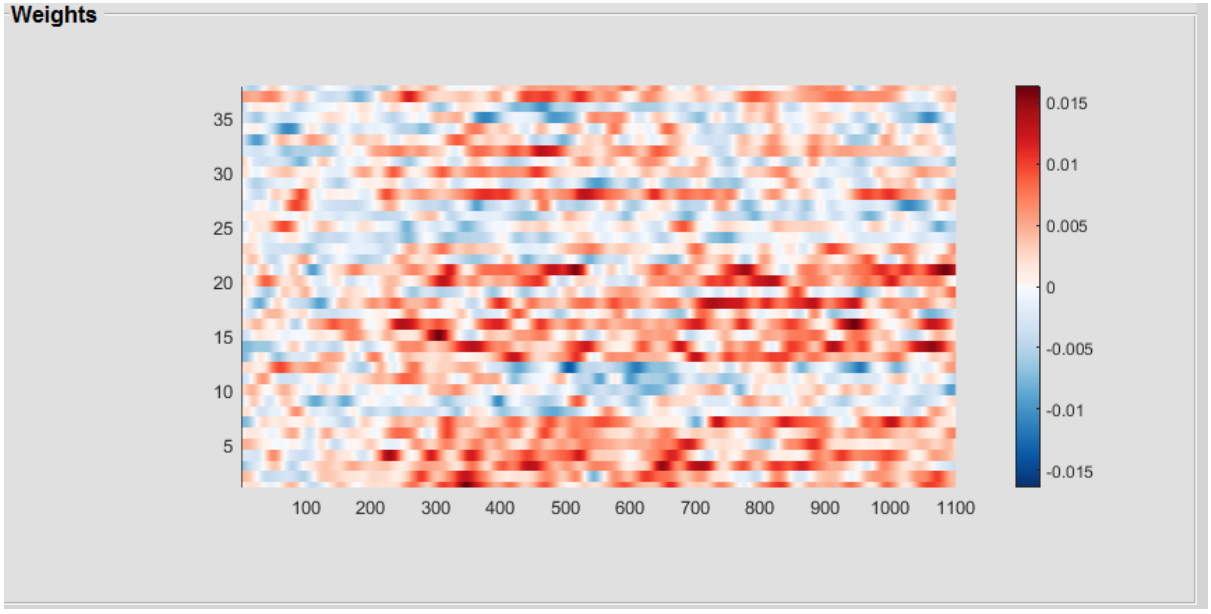


Figure 13: Weights for each channel (y-axis) and time point (x-axis).

On the other hand, for the MKL model some channels have no contribution:

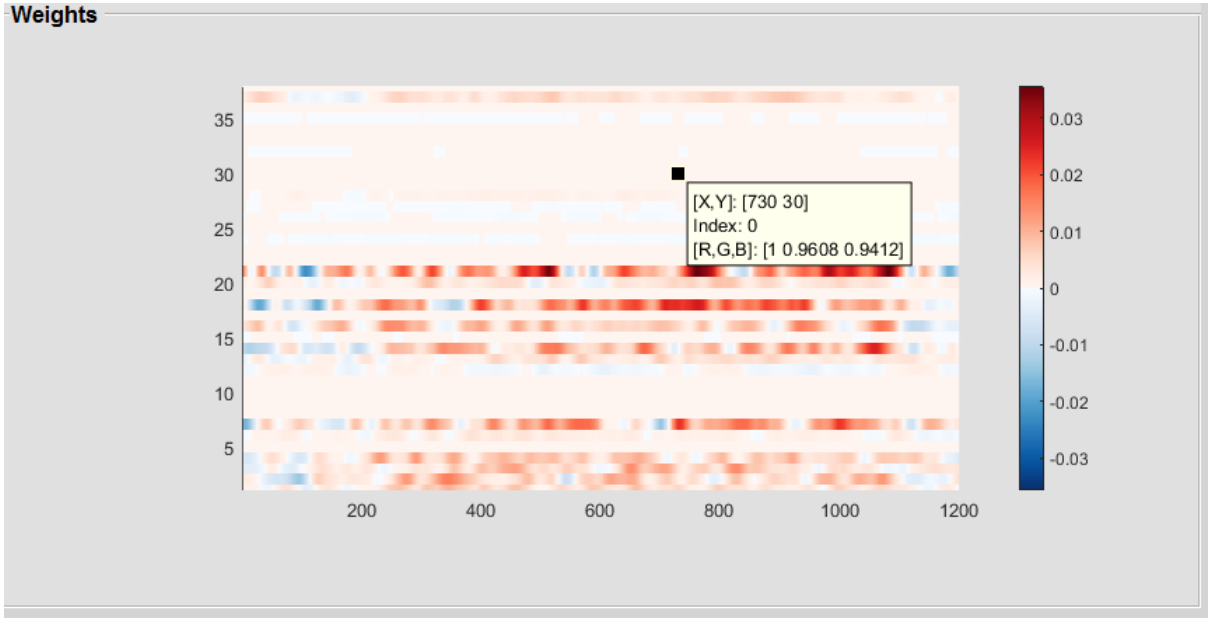


Figure 14: Weights for each channel (y-axis) and each time point (x-axis) for the MKL model.

The weights per regions can also be displayed and the table of 'region' (here channels) contributions is shown along a histogram depicting the sparsity of the solution. Interestingly, the histogram shows that around half of the channels have a contribution to the model (21 channels out of 38 on average across all folds).

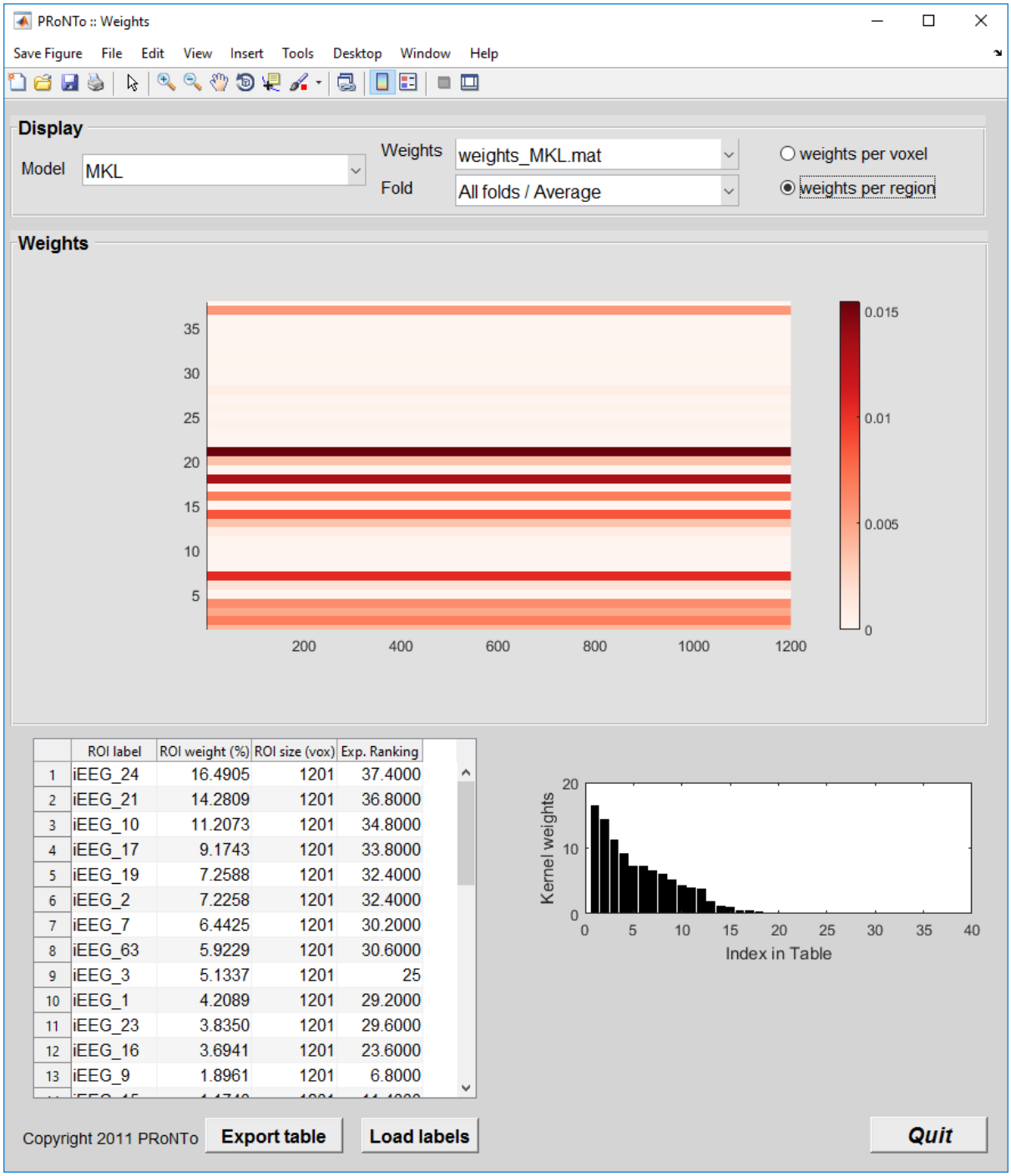


Figure 15: Kernel (here channel) contributions to the classification.

Batch

Data and Design

In the batch, we can specify our experiment by:

- Select a 'batch' subfolder to write the results
- Add a group, 'G1'
- Add a subject, 'S1'

- Add a modality 'ECoG'
    - Specify the modality type as 'MEEG'
    - The interscan interval as the sampling interval (here 0.001s)
    - Select the file
    - Under Data & Design, select 'Events in MEEG file'. Note: this option is not selected by default as in the GUI, because the batch is agnostic of the user choices.
    - No regression targets/covariates
- In 'Masks' add a modality and specify 'ECoG', set the data format to 'MEEG'

The batch for the modality and mask are displayed below:

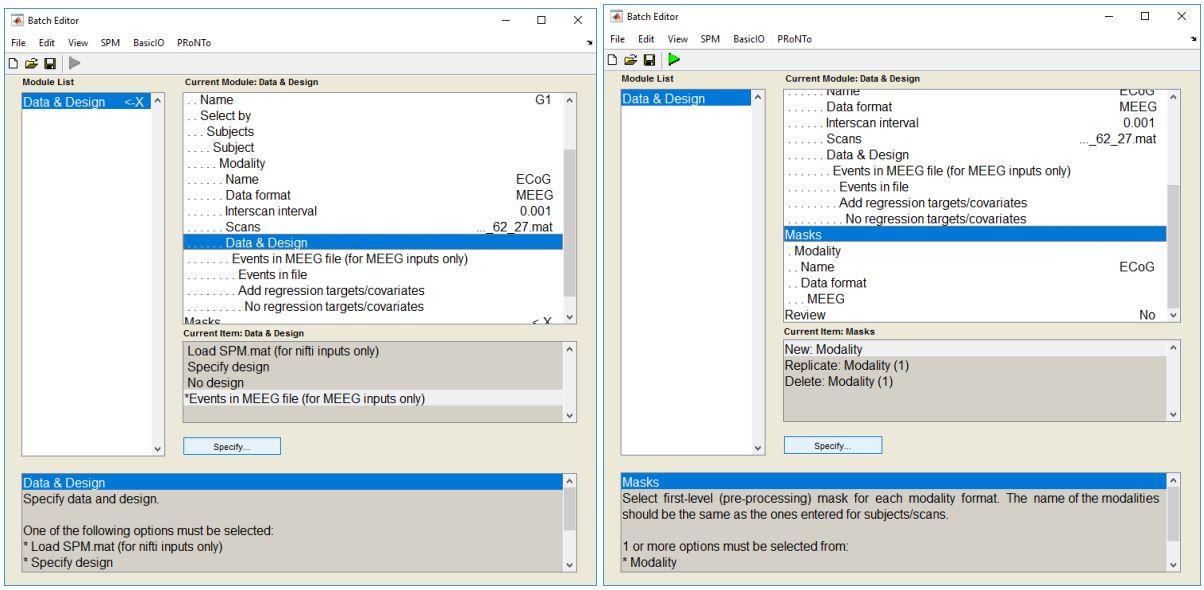


Figure 16: Data and design batch, specifying modality (left) and corresponding mask (right)

Feature set

In this step, we will specify a feature set for our ECoG modality. This time however we will specify multiple kernels to be built at the channel level, as well as for subwindows of 100ms in our window of interest.

The different fields should be filled as:

- Dependency on PRT.mat created at Data and Design step, or loading the PRT
- Naming the feature set, here MKLChanstp
- Choosing the Data format as 'MEEG' and adding a modality
    - Naming the modality 'ECoG' as defined in II.1
    - Channels: delete the 'All' selected by default and replace it with a New: Channel file. Choose the file 'Good_channels.mat' provided with the data set. This file contains the labels (in a cell array) of all the channels marked as 'good' during preprocessing. The channel selector is the same as SPM. Please refer to SPM's help for more details.
    - Tick the 'Multiple kernels' option under channels
    - Specify the time window as [-100 1100]
    - Select 'Multiple kernels' under time points and select 'One kernel per time window', entering 100ms as the time window to consider for each kernel. Please note: this will create 13 kernels, from -100 to -1, 0 to 99, 100 to 199, from 200 to 299, … plus one kernel comprising the last 1100ms time point. For balanced kernels, please specify the end of

your global time window (here 1100) as the end desired minus your sampling interval in ms (here 1).

- Then leave all the defaults

The batches for the channel selection and for the kernel specification are displayed below:

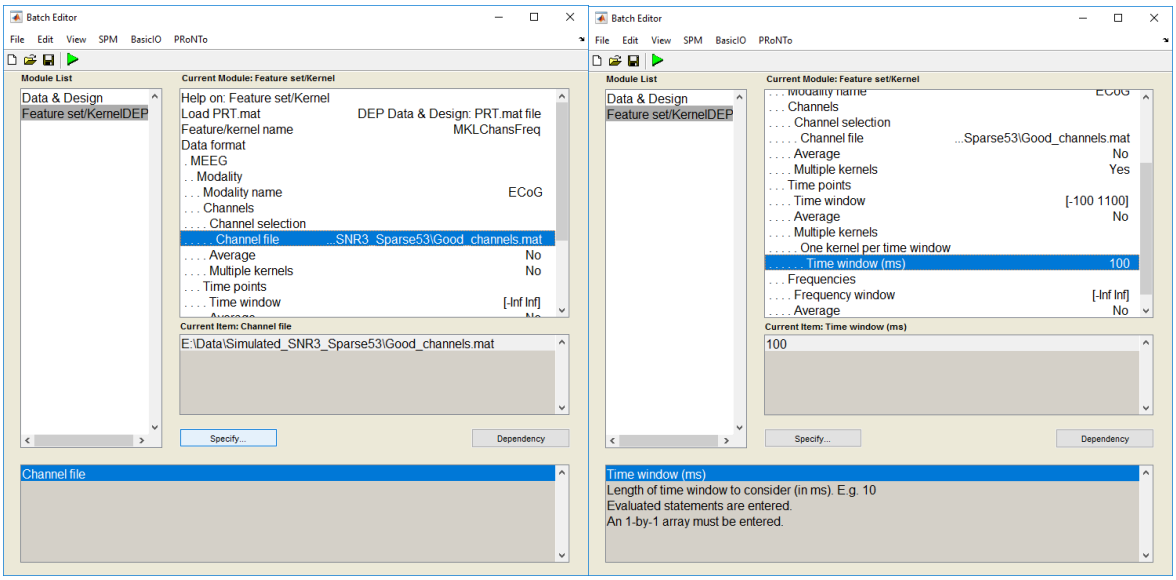


Figure 17: Feature set specification, including loading a channel file (left) and choosing multiple kernels on multiple dimensions (right).

Model specification

Specify a new model (named ‘sMKLChanstp’) that accesses your PRT.mat and specified MKLChanstp feature set. As in previous versions, define 2 classes, based on conditions A and B (subsampling to match the least represented class), and choose the L1- Multiple Kernel Learning machine. As this is a within-subject design, we can use the k-folds on Blocks per Class Out cross-validation, for hyper-parameter optimization as well as for model performance evaluation. We chose (arbitrarily) k=4 for the nested CV and k=5 for the outer CV. As the kernels do not include the same number of features, it is important to add the ‘Normalize samples’ operation.

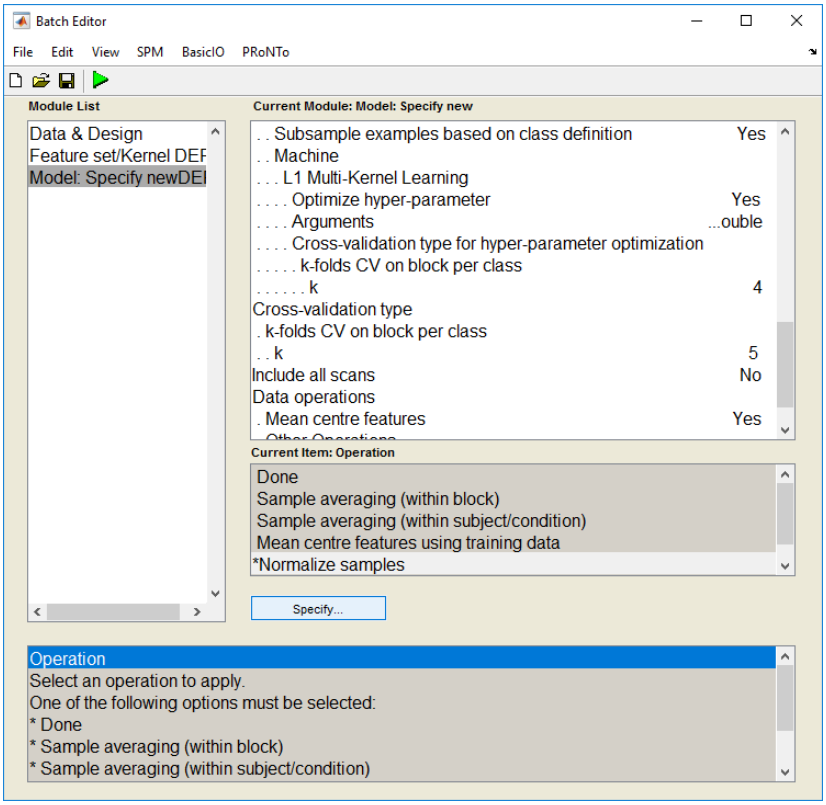


Figure 18: model specification, including 'Subsampling examples based on class definition', k-folds CV on block per class cross-validation and 'Normalize samples' operation.

Run model and results

The 'Run model' is the same as in previous PRoNTo versions. Simply load your PRT and type the name of the model as chosen in the previous step. Run your batch.

In the 'Display Results', we see that this model does not perform as well as the previous channel MKL model on average. However, the difference in model accuracy is small and the standard deviation of this model is smaller, making it more stable across folds.

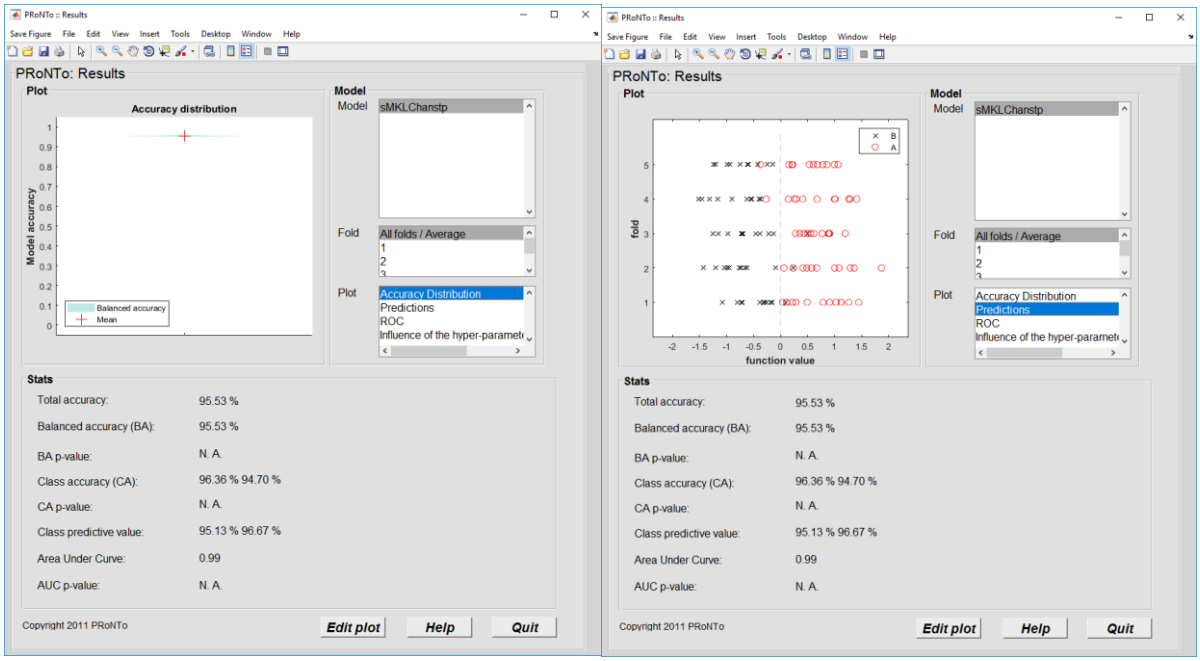


Figure 19: model results, balanced accuracy distribution (left) and fold predictions (right).

Weights

In this last (optional) step, we're interested in the kernel contributions. More specifically, is the discrimination distributed in time or not? In the present case, we know the ground truth (from the simulated signal, the discrimination is uniform in the [0 1000]ms time window). It is hence interesting to look at kernel contributions to see if this information from the simulated signal was properly recovered.

To this end, select 'Compute weights' and load the PRT.mat. Specify the model name as in II.3 and select the 'Build weights per region' option, leaving the 'Load atlas' as it is (blank). PRoNTo will automatically access the atlas defined at the feature set step if no atlas is specified. Run the batch.

In the 'Display Weights', click on the model and select 'Weights per region'. You will see the table of kernel contributions for each channel and time window (referred to by their starting time point in ms).

As we can see, the MKL model mostly identifies time windows within the [0 1000]ms window of simulated signal. However, the MKL only provides a sparse solution, i.e. it does not recover all time windows that have discrimination information in the A vs B classification.

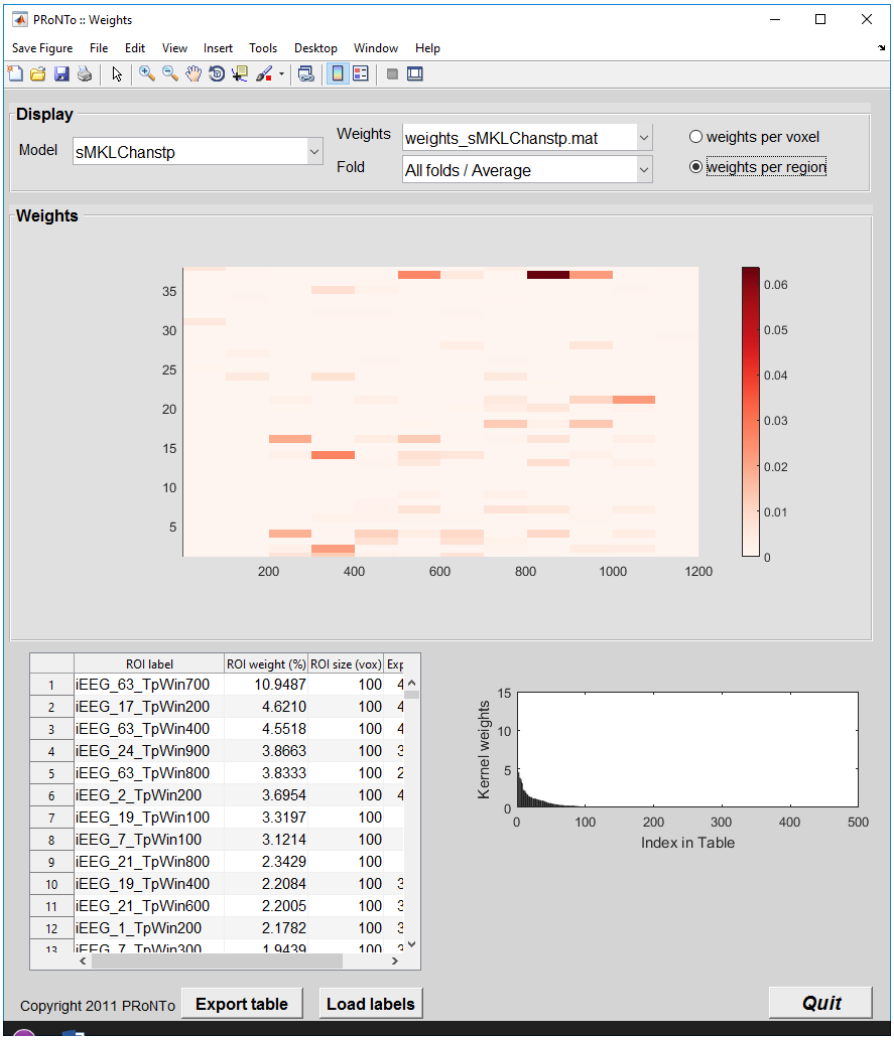


Figure 20: displaying kernel contributions.

Tutorial: Non-kernel machine

In this example, we will use a non-kernel machine to classify conditions ‘A’ and ‘B’. As non-kernel machines (also referred as ‘primal’ formulations) build the model based on the features and not based on the similarity between pairs of points, using them with data including a lot of features is not very efficient. However, some algorithms only exist in primal form and could be of interest (for example LASSO, Tibshirani, 1996).

In this tutorial, we will select a single channel of ECoG (one on which there is signal) and use the L1-SVM machine that performs feature selection during model estimation.

GUI

Feature set

We first need to build a new feature set called ‘Chan3’, selecting only channel 3 and time points [-100 to 1100]ms around onset. This leads to 1201 features for our model.

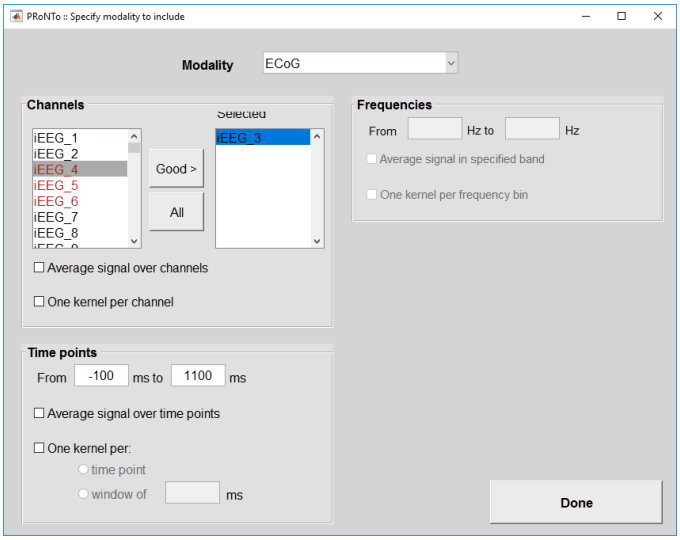


Figure 1: Feature set including only one channel.

Model

We can ‘Specify from’ and choose the ‘SVM’ model as a basis (default). We first need to remove the selected feature set and instead select our new ‘Chan3’ feature set. We can then untick the ‘Use kernels’ radio button. The ‘machine’ list will be updated and now includes Binary L2-SVM, Binary L1-SVM, Multiclass SVM, L2-Logistic Regression and L1-Logistic Regression (all interfaced from LIBLINEAR). Select ‘L1-SVM’ and optimize the hyper-parameter using a k-folds cross-validation on Blocks per Class Out (k=4). Two new options appear for non-kernel methods: ‘Normalize features’ or ‘Z-score features’. Let’s z-score.

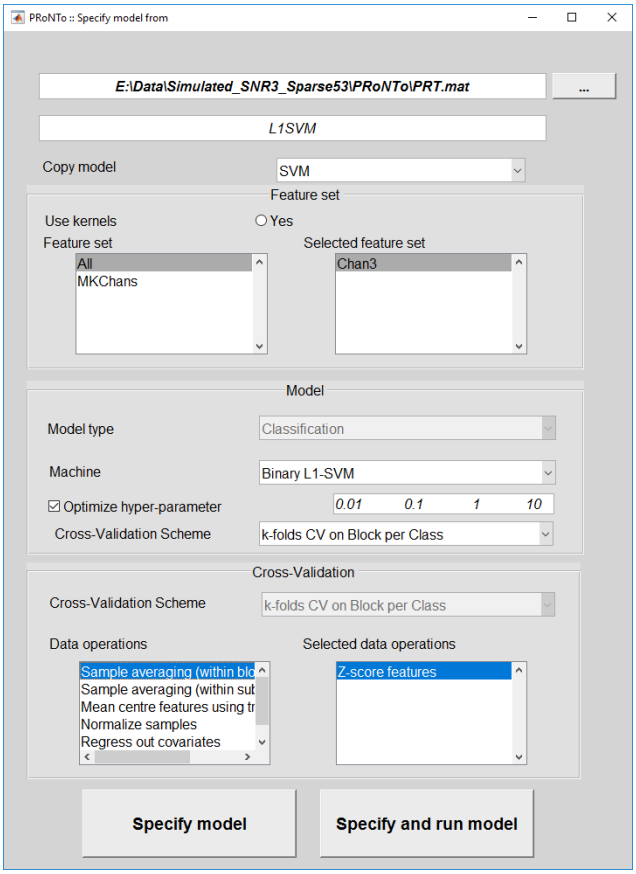


Figure 2: Specify model from the kernel SVM model, choosing a non-kernel method (L1-SVM).

The model gives 65.15% accuracy and a AUC of 0.76. We can also review the weights of this model. As this is an L1-SVM, only a few time points on channel 3 should have non-null weights. When looking at the weights across all folds, the weights will typically not be very sparse if the model was not good. This reflects that feature selection across the folds is rather unstable. It is the case here, with all features having a non-null weight across folds, but fold 1 (for example) only includes 62 features/time points.

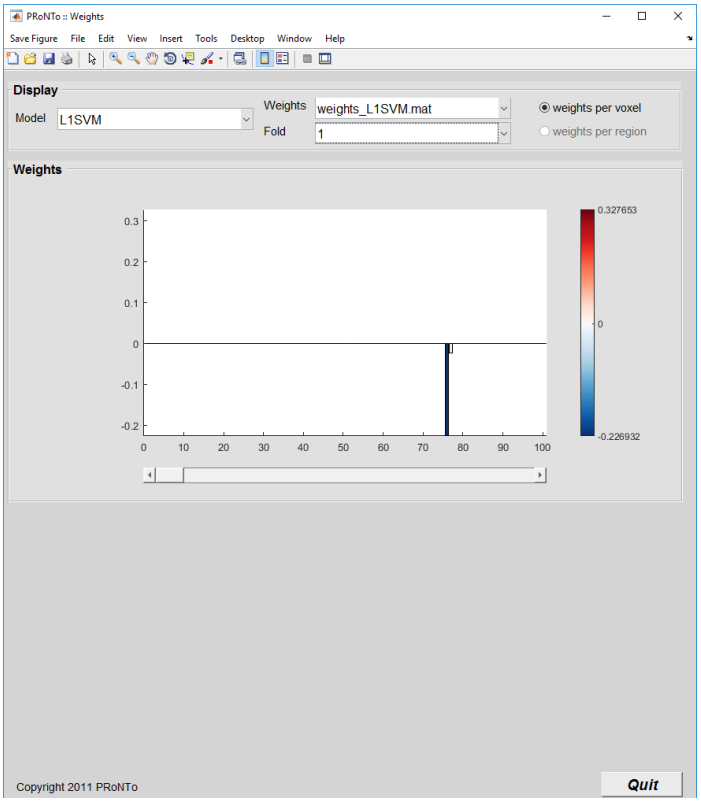


Figure 3: weights in fold 1 of the L1-SVM model.

Batch

Using the feature set Chan3 created above, we can build a non-kernel model using the batch. Open the batch and select the 'Model: Specify from' module.

Specify model

Load your PRT.mat and give a name to the model. We will try the Binary L2-SVM machine this time. We can call the model L2SVM and choose 'SVM' as the 'basis' model. We will first amend the machine, so add a new 'Model type' field in 'Fields to modify'. We then choose a classification, non-kernel machine and select the Binary L2-SVM. We then add a 'New field' to modify and change the feature set name to Chan3.

The model displays an accuracy of 69.47%, very similar to the L1-SVM. Please note that using a kernel SVM on the same data provides an accuracy of 71.36% (with only 58% for class 'A').

The weights for this model are quite smooth.

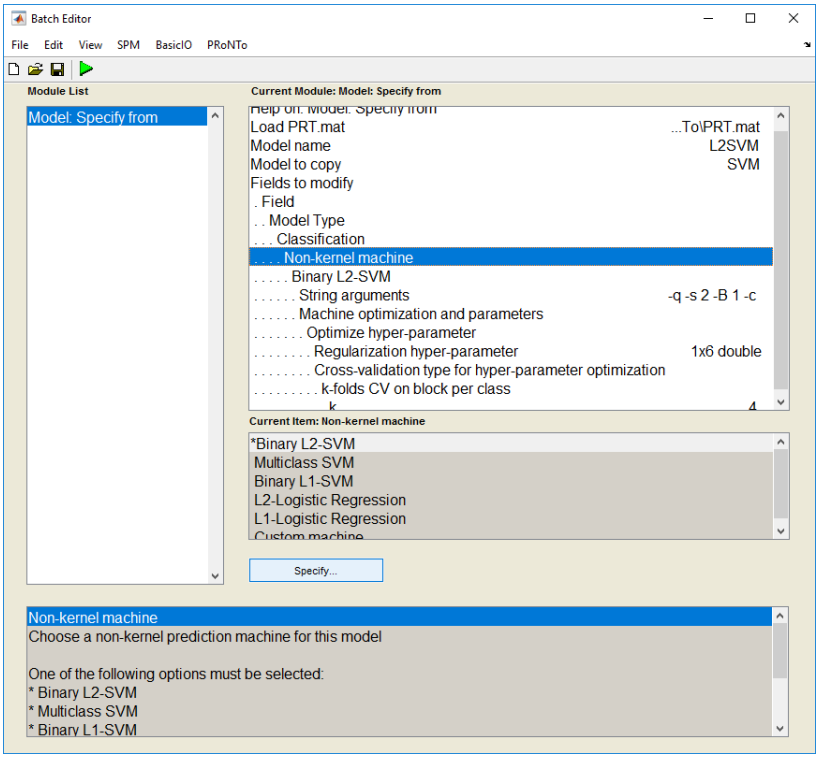


Figure 4: Choosing a non-kernel machine in the batch.

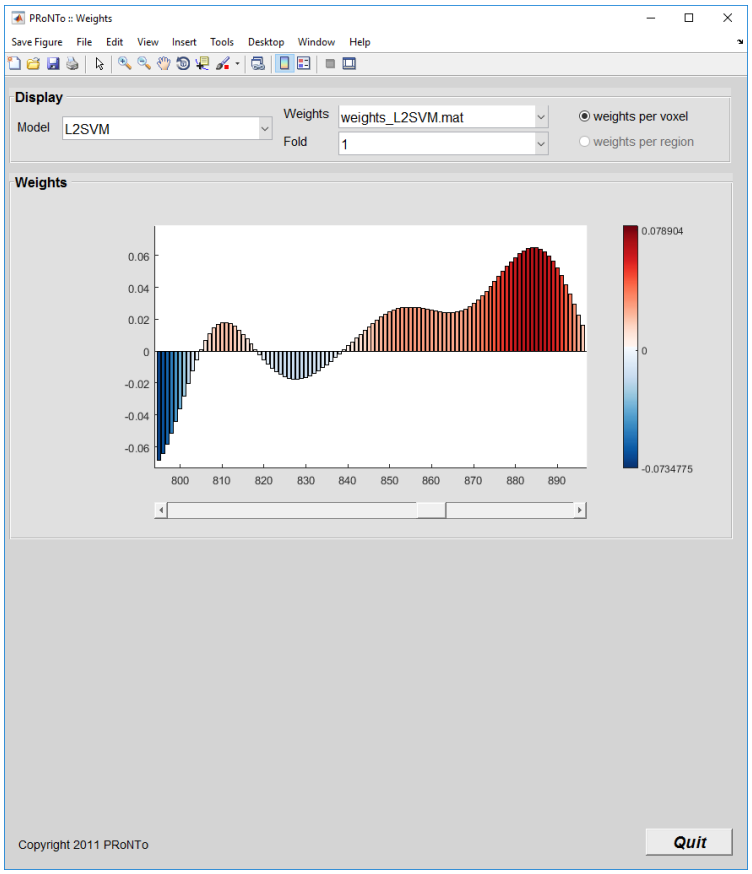


Figure 5: Weights for primal binary L2-SVM model.

Tutorial: Within-subject regression

In this example, we will use the semi-simulated ECoG data with random regression targets and covariates to illustrate within subject regression with MEEG data. Most of what follows also applies to nifti and .mat although there are small differences at the Data and Design stage.

For each condition, we will create random targets by using a fixed number (1 for condition A, 3 for condition B) to which we add uniform random noise between 0 and 1. There are 73 epochs for condition A, so:

```
>> rt_trial = ones(1,73)+rand(1,73);
```

For trials of condition 'A' and:

```
>> rt_trial = 3*ones(1,73)+rand(1,73);
```

The covariates will then model the ground truth of those targets, such that when removing confounds, we would remove all target differences between the 2 conditions. We can either simply use

```
>> R = ones(73,1); save('Confounds_A.mat','R') % for condition A
```

```
>> R = 3*ones(73,1); save('Confounds_B.mat','R') % for condition B
```

On the other hand, we could consider that this is actually a categorical covariate, i.e. being from condition A or from condition B. This needs to be one-hot encoded according to:

```
>> R = repmat([1 0],73,1); save('Confounds_A.mat','R') % for condition A
```

```
>> R = repmat([0 1],73,1); save('Confounds_B.mat','R') % for condition A
```

In the present 'dummy' application, this doesn't make much of a difference and both options correctly remove the main condition effect. In practice, we recommend using one-hot encoding of categorical variables when these are clearly unordered.

GUI

Starting as before, let's create one group 'G1' and add one subject and one modality called 'ECoG'. Choose 'MEEG' as its type and load the file. To add regression targets or covariates per subject, the design needs to be modified. To do so, click on the 'Events in file' option in the design menu. The condition names, onsets and durations have been filled. There are 2 more columns that can be edited: the 4th corresponds to regression targets per trial and the last to covariates per trial. As the design is automatically loaded from the MEEG object, these 2 fields have to be entered manually, for each file in the design. The expressions above can be entered in the corresponding table cells to obtain a design similar to (when using the value covariates):

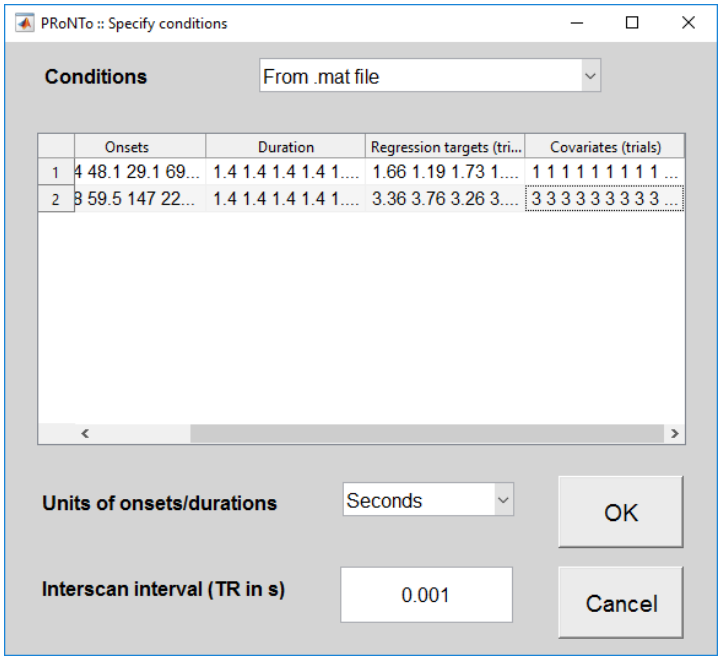


Figure 1: Specify regression targets and covariates per trial.

Important notes:

- The number of targets must correspond to the number of trials in each condition, including bad trials for MEEG data (hence the 73 instead of 60 and 56).
- Multiple covariates can be entered in the form of a # trials x # covariates matrix. As the table can only display one line, the matrix is vectorised before display. This does not affect the matrix in the PRT structure.
- For .mat or nifti formats, a file can be specified to fill in the whole table. This file should contain the 'names', 'onsets' and 'durations' variables (as in previous PRoNTo versions), as well as a 'rt_trial' and 'R' cell arrays, with one cell per condition.

Click 'OK' and 'Done', then save the PRT. You can check the values were passed correctly by typing:

```
>> load('PRT.mat')
>> PRT.group.subject.modality.design.conds(2).cov_trial
```

Feature set

You can build the feature set as in the previous tutorial, choosing one kernel per channel on the 'good' channels and time points included between -100 and 1100ms.

Model specification

We will first build a new model to perform the within-subject regression and see whether a pattern can be found for this 'artificial' regression model. We select 'Specify new' and load the PRT.mat. We call the model 'RegKRRnoCov' and choose our feature set. We indicate this is a regression problem and then click 'Select Subjects/scans'. We add subject S1 and both conditions A and B to the model (i.e. we want to include all epochs in the regression model) and click Done. We select the Kernel Ridge machine with default hyper-parameter optimization. For cross-validation, we select 'k-folds CV on Block', with k=4 for optimization (i.e. nested CV) and k=5 for the outer CV. We add the 'Mean centre' operation and Specify and run the model.

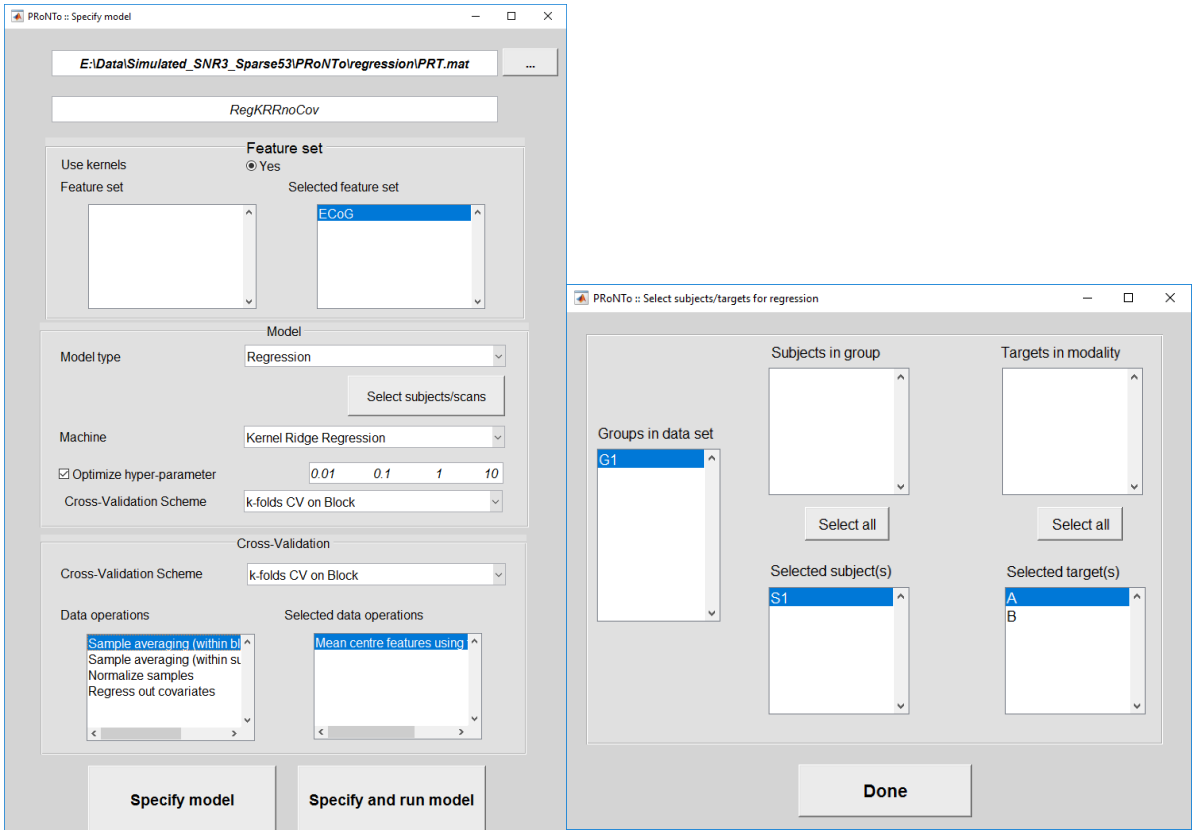


Figure 2: Specify regression model and select samples.

We can similarly create another model called 'RegKRRCov' that adds the 'Regress out covariates' operation. Please note that we could not have used the 'Specify from' option as the covariates are only loaded in the model if the 'regress out covariates' operation is selected.

Model results

The results for both models are displayed in figure 3: the first model somewhat identifies that trials from condition A have lower regression targets than epochs from condition B. We see however that the model predicts a relatively central value for all epochs, close to 2.5 (the average between all targets when taking the random noise into account).

Removing the model leads to an even worse model, that clearly predicts the target average all of the time.

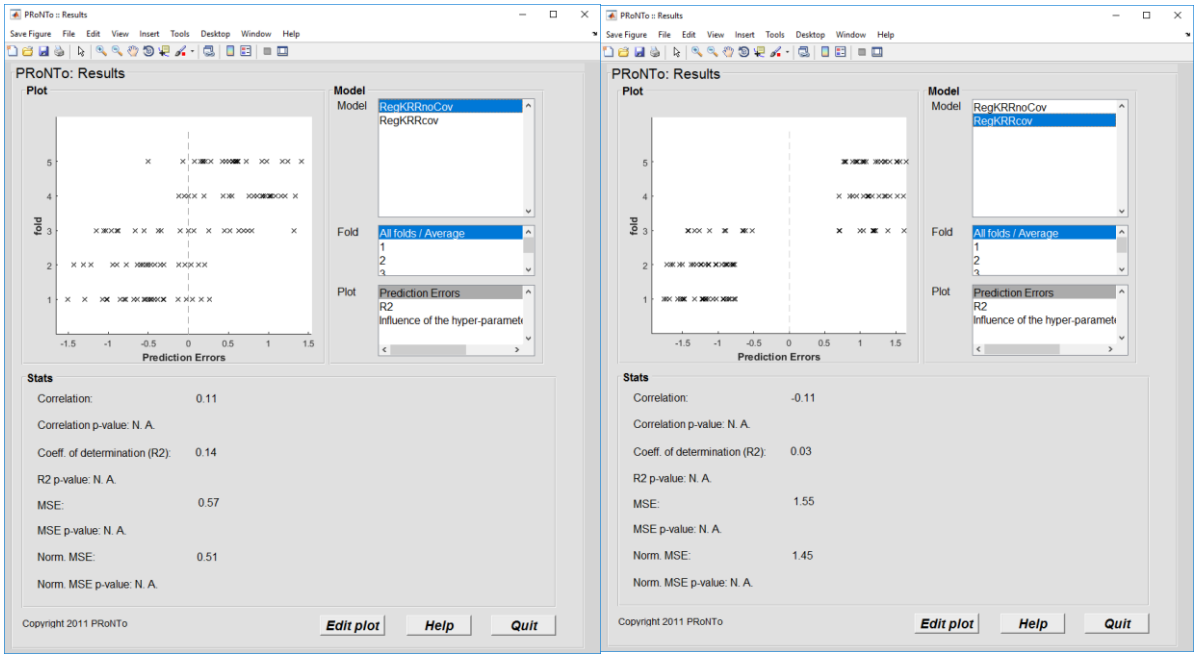


Figure 3: Model results for regression (left) and for the same regression but removing confounds that are heavily correlated with the targets (right).

As for classification, the plots have been modified according to the emphasis that 'All folds' only reflects the average across the different folds (i.e. different models estimated). A 'Prediction Errors' plot was added to reflect, for each point, the difference between the target and the predictions. A perfect regression model would lead to all points being located on the '0' vertical axis.

Similarly to the 'Accuracy distribution' plot, a 'R2' plot displays the distribution of R2 across the different folds in a violin plot. The plot is symmetric if no permutations were performed while it will contain the 'true label' R2 distribution on its left and the 'permuted label' R2 distribution on its right if permutations were estimated.

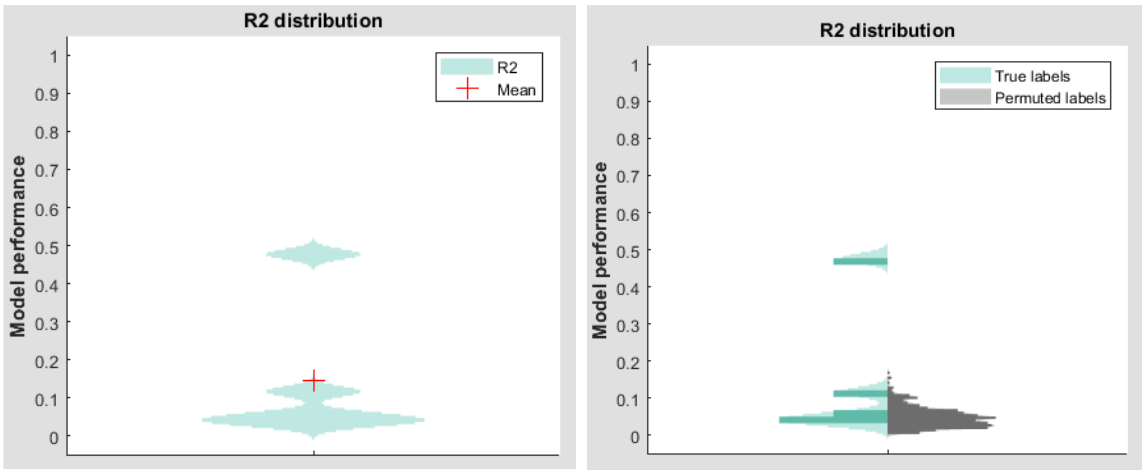


Figure 4: R2 distribution plot without (left) and with (right) permutations.

Important note: The pre-processing steps sort the 2 conditions A and B. This means that the targets are also sorted, with all the lower values in the 1st half of the samples and the higher values in the 2nd half. This clearly affects the cross-validation: in the first fold, we only leave lower values out, making the train and test sets very imbalanced. Hence, care should be taken to avoid this kind of situations and to ensure that

the cross-validation train and test sets are balanced in terms of target variances and ranges. This was not an issue during the classification as we were able to use the 'k-folds on Blocks per Class Out' cross-validation scheme, leading to balanced train and test sets.

Batch

Data and Design

As in the previous chapter, we start by adding a group, a subject and a MEEG modality to our data set. We choose the 'Events in file' design and see the option 'Add regression targets/covariates'. We click on this option and select to 'Add regression targets/covariates'. For each condition, we then need to specify the condition's name (such that PRoNTo knows which condition to relate the inputs to, e.g. 'A'), input the regression targets as an expression (a vector or expression, e.g. ones(1,73)+rand(1,73)) and finally load the covariates from a file (saved in a variable called 'R' as described in the 'Data' part of this chapter).

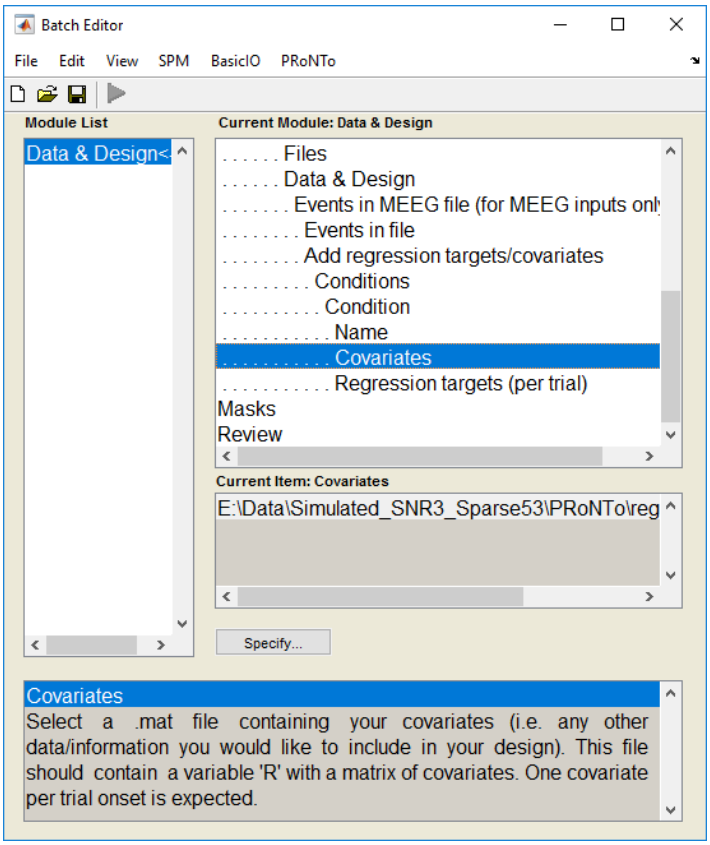


Figure 5: Data and design batch to add regression targets and/or covariates to MEEG designs.

Feature set

As for the GUI, there is nothing specific to within-subject regression at the feature set level and the instructions from the previous chapter can be followed.

Model

A new model can be specified using the built feature set. This is a 'regression' model to which we add our group 'G1', subject 1 and 'All conditions' under the 'Conditions/Samples' menu. The machine can then be a kernel machine, e.g. Gaussian Processes (with no option for hyper-parameter optimization as this is performed in a Bayesian framework) and the 'k-folds on Blocks Out' cross-validation scheme with k-5. We

can specify two versions of this model: one without additional operations and one removing the confounds. The results are similar to the GUI with KRR.

Example: between-subject regression

In PRoNTo v3, it is possible to include multiple regression targets per subject when selecting the subjects ‘per Samples’ (i.e. when there is no experimental design and when there is only one sample per subject). These different targets then appear as ‘conditions’ would when selecting subjects and targets for the model. See the examples below (data not provided).

GUI

Data and Design

In the data and design step, the editing part for regression targets has been replaced by a pull-down menu, with options ‘No targets’ (default) or ‘Specify targets’.

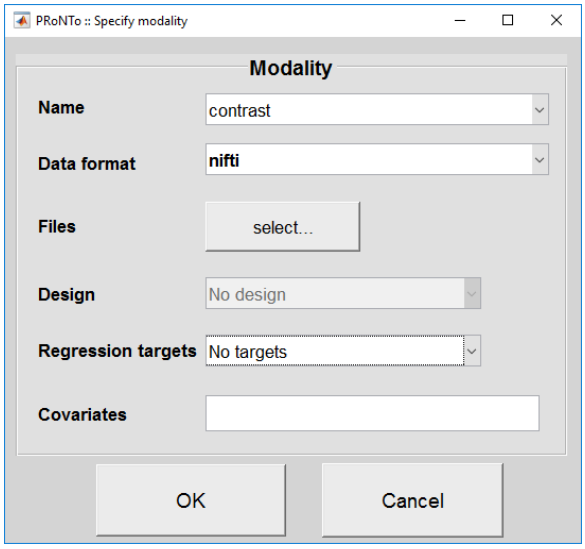


Figure 1: A menu replaces the editing part of the window for regression targets.

Select ‘Specify targets’ to input regression targets (one or multiple). A new window (similar to the one for specifying conditions in a design) appears. In this window, another menu asks you to either ‘Specify’ or ‘Load a .mat’. If you ‘Specify’, this means that you will enter the values by hand. Enter the number of regression targets you want to enter (for example, enter ‘1’ if you only want to predict age, or ‘2’ if you’d like to try age and a clinical score).

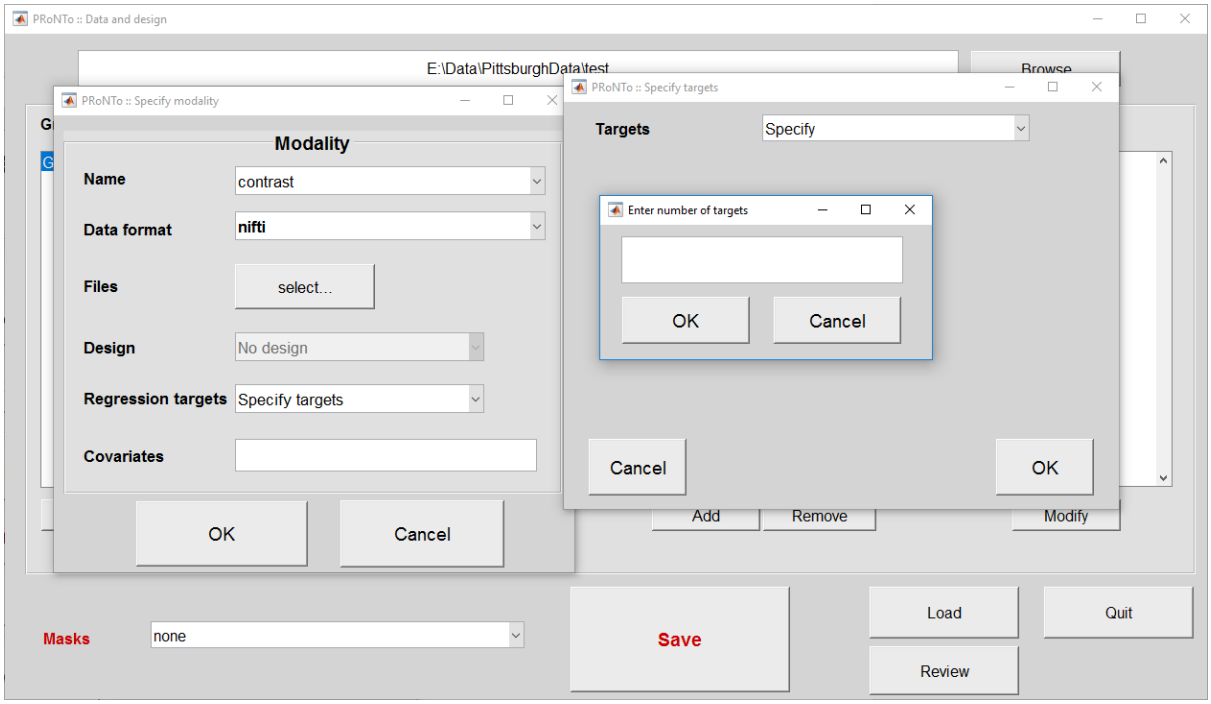


Figure 2: Select 'Specify' in the Targets menu in the new window and enter the number of targets you want to input.

Within the window, an empty table with 2 columns appears. The number of rows in the table corresponds to the number the user entered. In each row, specify first a name for the regression target (e.g. 'age' or 'clinical') and then enter the values for the corresponding regression target. Instead of specifying manually, you can load a .mat that includes 2 variables: names (cell array with the names of the targets) and rt_subj (matrix of size #samples x #targets, with the target values). There can be no missing values in the entered target values, but you can place a 'NaN' instead. Subjects with NaN target values will be excluded from the model.

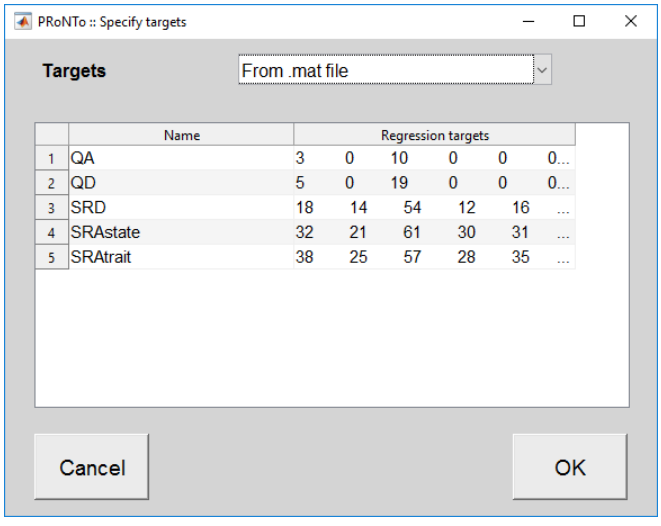


Figure 3: 5 targets were entered, with a name and one value per sample.

In all cases (batch and GUI, specifying or loading .mat), please ensure that the values are entered in the correct order and that there is one value per sample.

Specify model

When specifying a regression model at the 'Specify new' model, the window for selecting subjects is now similar to that of classification. It includes the list of groups and subjects to select from, as well as the list of available targets. Only one target can be selected per model.

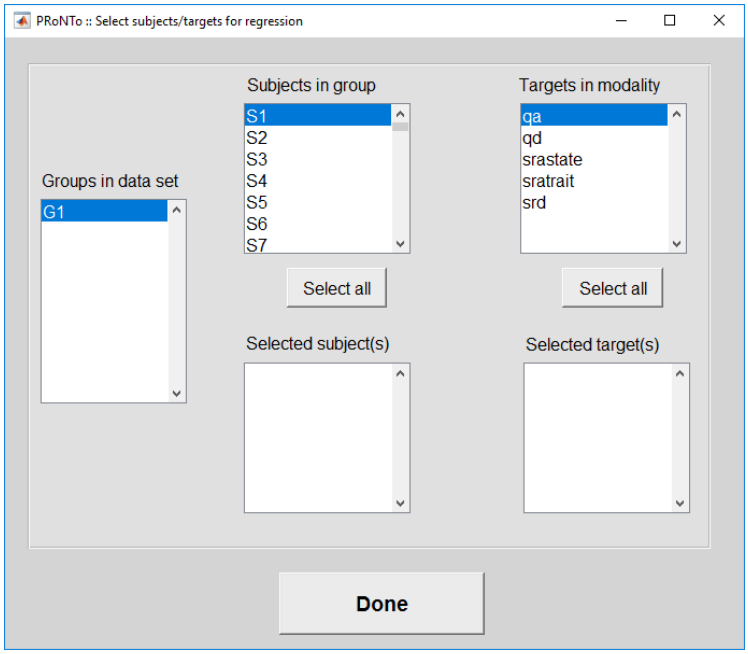


Figure 4: Specify subjects for a new regression model.

Batch

Data and Design

Create a group and select by 'Samples'. In the modality, you can see new options for the regression targets. You can manually 'Specify', adding a target and specifying its name and the corresponding values. Or you can load a file, with the same variables (names and rt_subj) as mentioned above and in the Regression target file – across subjects section.

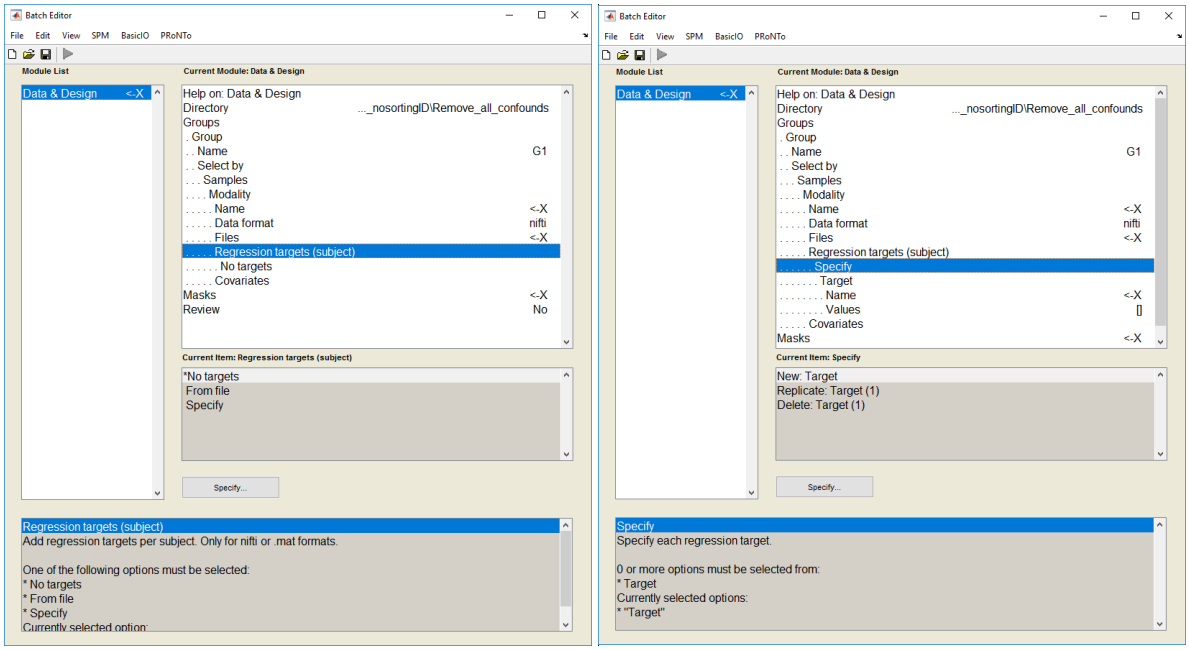


Figure 5: Batch to input multiple regression targets, either manually (right) or loading a file.

Specify model

In the 'Model: Specify new' module, select 'model type' as 'regression' and add a group. You can specify the index of the subjects to use, as well as choosing whether to use a specific condition ('Specify condition', when there is a design with regression targets per condition), all conditions, all samples, or a specific target ('Target'). Select 'Target' and enter the name (as entered in Data and Design) of the target to use. PRoNTo will 'ignore' subjects with NaN values for the chosen regression target.

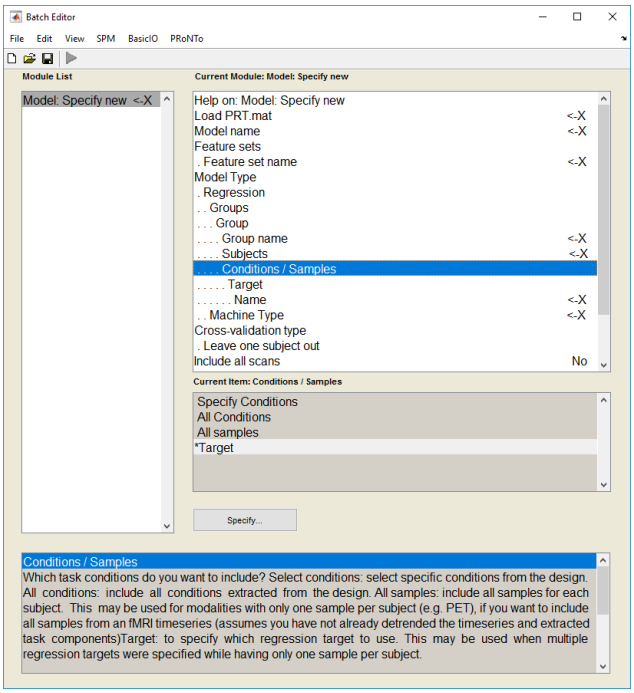


Figure 6: Select regression targets in the batch

Appendix:

One file per .mat sample

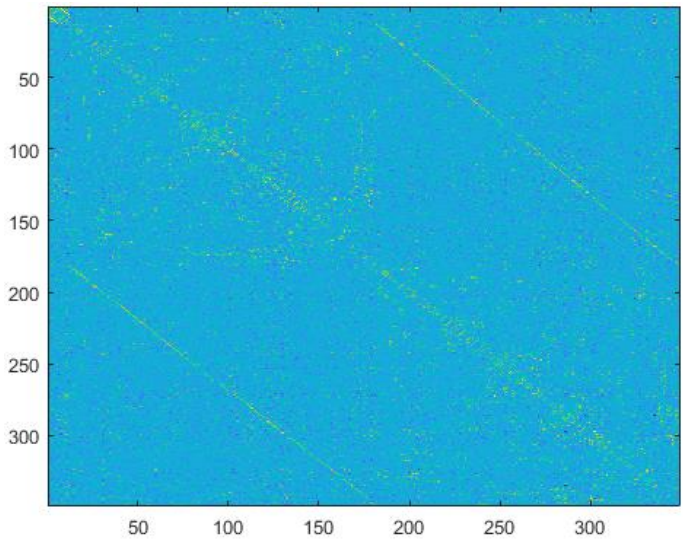
For behavioural data, it can be common to save all the data in a single file, with each row being a sample (subject or trial) and each column being a feature. PRoNTo however does not accept this format as multiple rows could arise from another input dimensionality of the data that the user wishes to keep (e.g. 2D connectivity matrix instead of vectorized).

We hence provide a script that transform a #samples x #features matrix into a series of .mat files. If the data for each subject comes from a distance measure (e.g. correlation), an additional flag allows to turn the vector back to its original 2D form.

The script takes in the matrix as well as a Boolean to decide whether to infer 2D matrix from a distance vector (1 to try to perform this operation, 0 to keep the vector, default: 0). Example: variable 'fmri' contains the connectivity vector (upper triangular only) for each of 293 subjects.

```
>> [path_files] = script_create_one_file_per_row(fmri,1);
```

The output is a new folder called 'Samples_mat' that contains files 'Samples_001.mat', …., 'Samples_293.mat' (here for 293 subjects). Each file contains one variable called 'data' that represents the data for this sample. In the present case, 'data' is a 349*349 matrix.



Script:

```
function [path_files] =
script_create_one_file_per_row(data_matrix,tosquare)
% Creates one file per row of a data matrix. The files will be saved in a
% subfolder, with the names 'Sample_' prepended. The second input
'tosquare'
% reflects whether the data should be turned into a symmetric matrix or not
% (0, default value)
%-----------------------------------------------------------------------
% Written for PRoNTo v3.0 by J. Schrouff

if nargin<1 || isempty(data_matrix)
    beep
    disp('At least one variable must be specified, the data matrix to
convert')
```

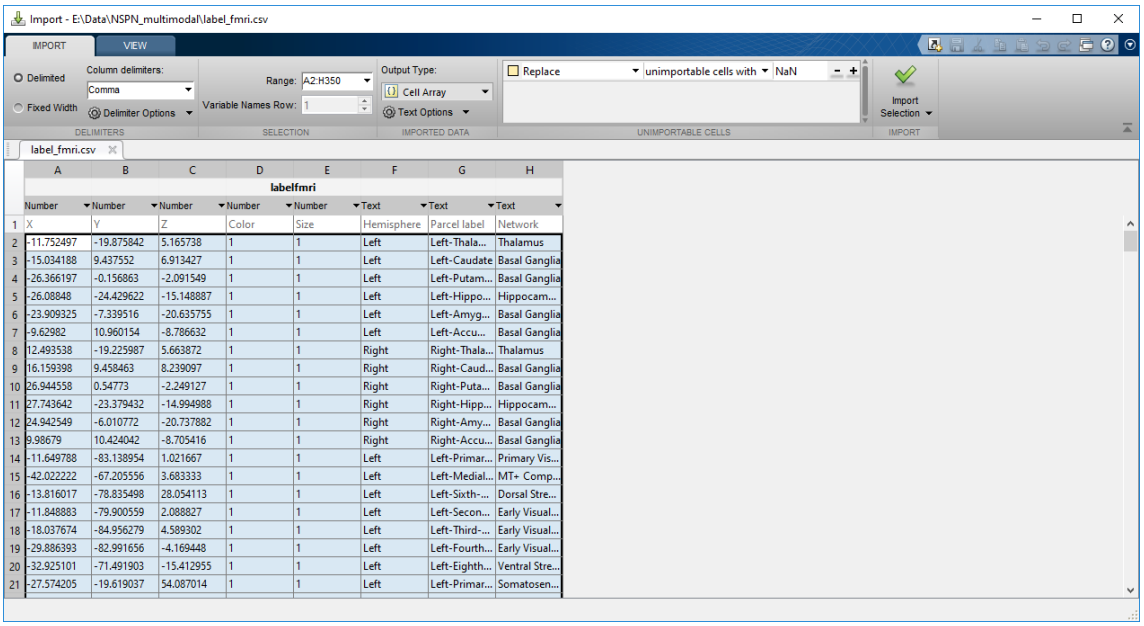
```
    return
end

if nargin<2 || isempty(tosquare)
    tosquare = 0;
end

nsamp = size(data_matrix,1);

d = dir('.');
path_files = fullfile(d(1).folder,'Samples_mat');
if ~exist(path_files,'dir')
    mkdir(path_files);
end
cd(path_files);

fprintf(['Sample (out of %d):',repmat('
',1,ceil(log10(nsamp))),'%d'],nsamp, 1);
for i = 1:nsamp
    % Subject counter
    if i>1
        for idisp = 1:ceil(log10(i)) % delete previous counter display
            fprintf('\b');
        end
        fprintf('%d',i);
    end

    % Access sample's data
    datas = data_matrix(i,:);
    if tosquare
        try
            data = squareform(datas);
        catch
            data = datas;
        end
    else
        data = datas;
    end

    prep = [];
    for idisp = 1:floor(log10(nsamp))-floor(log10(i)) % Add zeros in front
of name for easier access
        prep = [prep,'0'];
    end
    fname = ['Sample_',prep,num2str(i),'.mat'];
    save(fname,'data');
end

fprintf('\n');
```

Compute atlas for connectivity matrix

We built connectivity matrices based on time series extracted from a brain parcellation. We used 349 anatomical parcels and have associated each of them to a larger ‘network’. This has been saved in a .csv (or Excel) file.

	A	B	C	D	E	F	G	H
1	X	Y	Z	Color	Size	Hemisphe	Parcel label	Network
2	-11.7525	-19.8758	5.165738	1	1	Left	Left-Thalamus-Proper	Thalamus
3	-15.0342	9.437552	6.913427	1	1	Left	Left-Caudate	Basal Ganglia
4	-26.3662	-0.15686	-2.09155	1	1	Left	Left-Putamen	Basal Ganglia
5	-26.0885	-24.4296	-15.1489	1	1	Left	Left-Hippocampus	Hippocampus
6	-23.9093	-7.33952	-20.6358	1	1	Left	Left-Amygdala	Basal Ganglia
7	-9.62982	10.96015	-8.78663	1	1	Left	Left-Accumbens-area	Basal Ganglia
8	12.49354	-19.226	5.663872	1	1	Right	Right-Thalamus-Proper	Thalamus
9	16.1594	9.458463	8.239097	1	1	Right	Right-Caudate	Basal Ganglia
10	26.94456	0.54773	-2.24913	1	1	Right	Right-Putamen	Basal Ganglia
11	27.74364	-23.3794	-14.995	1	1	Right	Right-Hippocampus	Hippocampus
12	24.94255	-6.01077	-20.7379	1	1	Right	Right-Amygdala	Basal Ganglia
13	9.98679	10.42404	-8.70542	1	1	Right	Right-Accumbens-area	Basal Ganglia

This file is then loaded in Matlab (double-clicking or 'uiopen') as a *cell array* (you need to modify the 'Output Type' field).

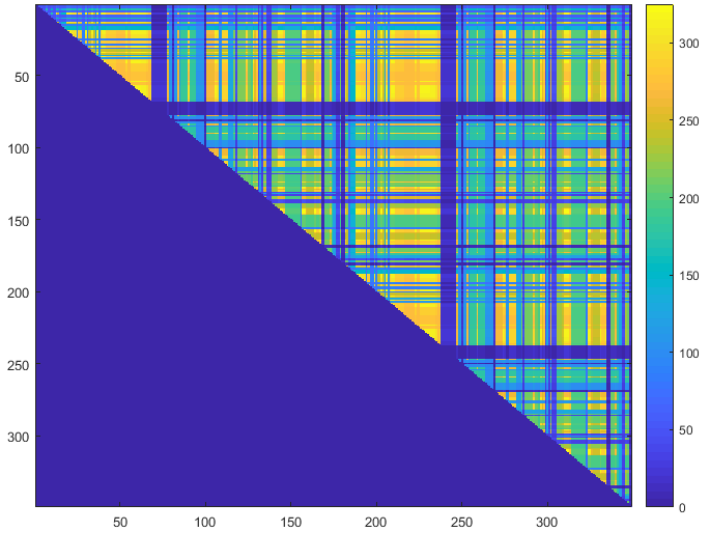


The last column of the cell array then represents the 'network' each time series belongs to. This list is passed to the script (here the variable name of the cell array is 'labelfmri' and it is the last column)

```
>> [atl_mat,ROI_names] = script_build_atlas_from_cell_array(labelfmri(:,end));
```

This outputs and saves the atlas file and its corresponding labels for use in PRoNTo. Please ensure to rename both of these files if you need to create multiple atlases (otherwise they will be over-written).

The atlas is a 2D matrix of size 349*349 that contains an integer for each 'network i – network j' interaction. Here, 25 'networks' were defined, giving 325 pairwise interactions (n*(n-1)/2, with n the number of networks). The matrix is upper diagonal to ensure to mask out redundant features (as correlations are undirected). For directed interactions, the full matrix would need to be generated (and the script modified).



Script:

```
function [atl_mat,ROI_names] =
script_build_atlas_from_cell_array(region_list)

if nargin<1 || isempty(region_list)
    beep;
    disp('Import excel file into cell array first')
    return
end

if size(region_list,2)>1
    beep
    disp('Please only provide the list of regions, no other variable')
    return
end

if ~iscell(region_list(1))
    beep
    disp('List of regions must be a cell array')
    return
end

if isstring(region_list{1})
    region_list = cellstr(region_list); % Convert to cell array of
characters
end

% Gather ROI names and labels
labs = unique(region_list);
n = numel(labs);


% Build atlas (squareform)
% ------------------------------------------------------------------------
% Initalize matrices
cnt=1;
a = zeros(numel(region_list),numel(region_list));
c = zeros(size(a));
```

```
% Loop over each pair of 'networks' and give it a unique number (i.e.
% the same unique number to all regions pertaining to the pair)
labels = cell((n*(n-1))/2 + n,1);
for i=1:n
    indi = ismember(region_list,labs{i});
    for j=i:n
        indj = ismember(region_list,labs{j});
        nroi1 = nnz(indi);
        nroi2 = nnz(indj);
        roi = cnt*ones(nroi1,nroi2);
        % Build diagonal matrix (i.e. (i,i) interactions)
        if j==i
            c(indi,indj)= roi;
        end
        a(indi,indj)= roi;
        labels{cnt}=[labs{i},'-',labs{j}];
        cnt=cnt+1;
    end
end

% Maybe regions were not sorted before we built the matrix. This will
% result in a matrix a that is not upper triangular (not symmetric). We
% correct for this with the following:
b = a+a'-c;

% b is now symmetric, and we can extract its upper triangle to match the
% inputs from the connectivity matrix
mask = triu(true(size(b)),1);
atl_mat = b.*mask;

save('atlas.mat','atl_mat');

% Save them in a file for display with the weights
ROI_names = labels;
save('Labels_atlas.mat','ROI_names')
```

Connectivity matrix from MEEG

Example script to create a connectivity matrix from averaged EEG or MEG trials (here both).

```
prepath = 'E:\Data\MultiModal_SPM_Faces\Sub';
subpath='\MEEG\';
fname = 'mapMcbdspmeeg_run_01_sss.mat';

type = {'EEG_','MEG_'};

for i = 1:16
    disp(['Computing subject S',num2str(i)])
    for t = 1:2
        if i<10
            fullp = [prepath,'0',num2str(i),subpath,type{t}];
        else
            fullp = [prepath,num2str(i),subpath,type{t}];
        end
        filename = [fullp,fname];
```

```
        D = spm_eeg_load(filename);
        dim = D.size();
        for c = 1:dim(end)
            conn_mat = corr(D(:,:,c)',D(:,:,c)');
            mask = triu(true(size(conn_mat)),1);
            conn_mat = conn_mat(mask);
            cond = D.conditions(c);
            savename = [fullp,cond{1},'_connectivity_matrix.mat'];
            save(savename,'conn_mat');
            disp(type{t})
            disp(['Size: ',num2str(size(conn_mat))])
        end
    end
end
```

Alternatively, the matrices can be saved as 2D arrays instead of taking only the upper triangular part (remove line '`conn_mat = conn_mat(mask)`'). In this case, a second-level mask should however be provided (save the defined mask in a separate file, i.e. `save('mask.mat','mask')`).

Connectivity ROI weights

Script 1: gather the atlas information (hard-coded here) and compute a summarized 'network' matrix (i.e. #networks x #networks instead of #channels x #channels).

```
% Load atlas to access the mask
load('EEG_atlas.mat')
id = find(mask);

% Atlas info for display and summarization
labs = {'OF','LF','CF','RF','LC','CC','RC','LP','CP','RP','O'};
OF = [2,4:8];
LF=[9:11,18:21];
CF=[12:14,22:24];
RF=[15:17,25:28];
LC=[29:32,41:43];
CC=[33:35,44:46];
RC=[36:39,47:49];
LP=[40,51:53,62,63];
CP=[54:58,64];
RP=[50,59,60,61,65,66];
O=[1,3,67:70];
ind = {OF,LF,CF,RF,LC,CC,RC,LP,CP,RP,O};

% load weights and access the average fold
load('ROI_weights_Conn_EEG_atlas_MKL.mat')
w_av = squeeze(weights(:,:,:,end));

% Weights, reconstructed on the original matrix
or_weights = zeros(size(mask,1),size(mask,2));
or_weights(id) = w_av;

% Summarize weights in each ROI as a single value for plotting
sum_weights = zeros(length(ind),length(ind));
w_sym = or_weights + or_weights';
```

```
for i = 1:length(ind)
    for j=1:length(ind)
        sum_weights(i,j) = max(max(w_sym(ind{i},ind{j})));
    end
end

% Imagesc matrix
cc = cbrewer('seq','Reds',200);
figure;
imagesc(sum_weights);
if min(min(sum_weights))==0 %if some weights are perfectly 0
    cc(1,:)=[0.8 0.8 0.8];
end
colormap(cc);
colorbar;
set(gca,'XTick',1:numel(labs))
set(gca,'YTick',1:numel(labs))
set(gca,'XTickLabel',labs)
set(gca,'YTickLabel',labs)


% Use modified version of schemaball to plot the summarized weights
schemaball_JS(sum_weights,labs,cc,[0 1 1])
```

Script 2: Plot schemaball of kernel contributions from a symmetric 2D matrix (first input), the network labels (a cell array, 2nd input), the colormap to plot the curves in (3rd input) and the color of the nodes to represent within-network contributions (4th input). This code was modified from the schemaball.m written by Oleg Komarov found on Matlab's file exchange.

```
function h = schemaball_JS(r, lbls, ccolor, ncolor)

% SCHEMABALL Plots correlation matrix as a schemaball
%
%   SCHEMABALL(R) R is a square numeric matrix with values in [-1,1].
%
%                 NOTE: only the off-diagonal lower triangular section of R
is
%                       considered, i.e. tril(r,-1).
%
%   SCHEMABALL(..., LBLS, CCOLOR, NCOLOR) Plot schemaball with optional
%                                         arguments (accepts empty args).
%
%       - LBLS      Plot a schemaball with custom labels at each node.
%                   LBLS is either a cellstring of length M, where
%                   M = size(r,1), or a M by N char array, where each
%                   row is a label.
%
%       - CCOLOR    Supply an RGB triplet that specifies the color of
%                   the curves. CURVECOLOR can also be a 2 by 3 matrix
%                   with the color in the first row for negative
%                   correlations and the color in the second row for
%                   positive correlations.
%
%       - NCOLOR    Change color of the nodes with an RGB triplet.
%
%
%   H = SCHEMABALL(...) Returns a structure with handles to the graphic
objects
```

```
%
%       h.l     handles to the curves (line objects), one per color shade.
%               If no curves fall into a color shade that handle will be
NaN.
%       h.s     handle  to the nodes (scattergroup object)
%       h.t     handles to the node text labels (text objects)
%
%
% Examples
%
%   % Base demo
%   schemaball
%
%   % Supply your own correlation matrix (only lower off-diagonal
triangular part is considered)
%   x = rand(10).^3;
%   x(:,3) = 1.3*mean(x,2);
%   schemaball(x)
%
%   % Supply custom labels as ['aa'; 'bb'; 'cc'; ...] or
{'Hi','how','are',...}
%   schemaball(x, repmat(('a':'j')',1,2))
%   schemaball(x, {'Hi','how','is','your','day?',
'Do','you','like','schemaballs?','NO!!'})
%
%   % Customize curve colors
%   schemaball([],[],[1,0,1;1 1 0])
%
%   % Customize node color
%   schemaball([],[],[],[0,1,0])
%
%   % Customize manually other aspects
%   h   = schemaball;
%   set(h.l(~isnan(h.l)), 'LineWidth',1.2)
%   set(h.s, 'MarkerEdgeColor','red','LineWidth',2,'SizeData',100)
%
%
% Additional features:
% - <a href="matlab:
web('http://www.mathworks.com/matlabcentral/fileexchange/42279-
schemaball','-browser')">FEX schemaball page</a>
% - <a href="matlab:
web('http://www.stackoverflow.com/questions/17038377/how-to-visualize-
correlation-matrix-as-a-schemaball-in-matlab/17111675','-browser')">Origin:
question on Stackoverflow.com</a>
% - <a href="matlab: web('https://github.com/GuntherStruyf/matlab-
tools/blob/master/schemaball.m','-browser')">Schemaball by Gunther
Struyf</a>
%
% See also: CORR, CORRPLOT

% Author: Oleg Komarov (oleg.komarov@hotmail.it)
% Tested on R2013a Win7 64 and Vista 32
% 15 jun 2013 - Created

%% Parameters
% Tweak these only

% Number of color shades/buckets (large N simply creates many perceptually
indifferent color shades)
N      = 100;
```

```
% Points in [0, 1] for bezier curves: leave space at the extremes to detach
a bit the nodes.
% Smaller step will use more points to plot the curves.
t      = (0.025: 0.05 :1)';
% Nodes edge color
% ecolor = [.25 .103922 .012745];
ecolor = [0 0 0];
% Text color
tcolor = [.3 .3 .3];

%% Checks

% Ninput
narginchk(0,4)

% Some defaults
if nargin < 1 || isempty(r);        r      = (rand(50)*2-1).^29;
end
sz = size(r);
if nargin < 2 || isempty(lbls);     lbls   = cellstr(reshape(sprintf('%-
4d',1:sz(1)),4,sz(1))');  end
if nargin < 4 || isempty(ncolor);   ncolor = [0 0 1];
end

% R
if ~isnumeric(r) || any(abs(r(:)) > 1) || sz(1) ~= sz(2) || numel(sz) > 2
|| sz(1) < 3
    error('schemaball:validR','R should be a square numeric matrix with
values in [-1, 1].')
end

% Lbls
if (~ischar(lbls) || size(lbls,1) ~= sz(1)) && (~iscellstr(lbls) ||
~isvector(lbls) || length(lbls) ~= sz(1))
    error('schemaball:validLbls','LBLS should either be an M by N char
array or a cellstring of length M, where M is size(R,1).')
end
if ischar(lbls)
    lbls = cellstr(lbls);
end

% Ccolor
if nargin < 3 || isempty(ccolor)
    ccolor = hsv2rgb([[linspace(.8333, .95, N); ones(1, N);
linspace(1,0,N)],...
                      [linspace(.03, .1666, N); ones(1, N);
linspace(0,1,N)]]');
else
    szC = size(ccolor);
    if ~isnumeric(ccolor) || szC(2) ~= 3
        error('schemaball:validCcolor','CCOLOR should be a 1 by 3 or 2 by 3
numeric matrix with RGB colors.')
    elseif szC(1) == 1
        ccolor = [ccolor; ccolor];
        ccolor = rgb2hsv(ccolor);
        ccolor = hsv2rgb([repmat(ccolor(1,1:2),N,1),
linspace(ccolor(1,end),0,N)';
                      repmat(ccolor(2,1:2),N,1),
linspace(0,ccolor(2,end),N)']);
    elseif szC(1) == 2
```

```
        ccolor = rgb2hsv(ccolor);
        ccolor = hsv2rgb([repmat(ccolor(1,1:2),N,1),
linspace(ccolor(1,end),0,N)';
                          repmat(ccolor(2,1:2),N,1),
linspace(0,ccolor(2,end),N)']);
    else
        N = floor(size(ccolor,1)/2);
    end

end

% Ncolor
szN = size(ncolor);
if ~isnumeric(ncolor) || szN(2) ~= 3  || szN(1) > 1
    error('schemaball:validNcolor','NCOLOR should be a single RGB color,
i.e. a numeric row triplet.')
end
ncolor = rgb2hsv(ncolor);
%% Engine

% Create figure
figure('renderer','zbuffer','visible','off')
axes('NextPlot','add')

% Index only low triangular matrix without main diag
tf = tril(true(sz),-1);

% Index correlations into bucketed colormap to determine plotting order
(darkest to brightest)
N2        = 2*N;
[n, isrt] = histc(r(tf), linspace(min(r(tf)),max(r(tf)) + eps(100),N2 +
1));
plotorder = reshape([N:-1:1; N+1:N2],N2,1);

% Retrieve pairings of nodes
[row,col] = find(tf);

% Use tau http://tauday.com/tau-manifesto
tau   = 2*pi;
% Positions of nodes on the circle starting from (0,-1), useful later for
label orientation
step  = tau/sz(1);
theta = -.25*tau : step : .75*tau - step;
% Get cartesian x-y coordinates of the nodes
x     = cos(theta);
y     = sin(theta);

% PLOT BEZIER CURVES
% Calculate Bx and By positions of quadratic Bezier curves with P1 at (0,0)
% B(t) = (1-t)^2*P0 + t^2*P2 where t is a vector of points in [0, 1] and
determines, i.e.
% how many points are used for each curve, and P0-P2 is the node pair with
(x,y) coordinates.
t2  = [1-t, t].^2;
s.l = NaN(N2,1);
% LOOP per color bucket
for c = 1:N2
    pos = plotorder(c);
    idx = isrt == pos;
    if nnz(idx)
```

```
        Bx     = [t2*[x(col(idx)); x(row(idx))]; NaN(1,n(pos))];
        By     = [t2*[y(col(idx)); y(row(idx))]; NaN(1,n(pos))];
        s.l(c) = plot(Bx(:),By(:),'Color',ccolor(pos,:),'LineWidth',1);
    end
end

% PLOT NODES
% Do not rely that r is symmetric and base the mean on lower triangular
part only
[row,col]  = find(tf(end:-1:1,end:-1:1) | tf);
subs       = col;
iswap      = row < col;
tmp        = row(iswap);
row(iswap) = col(iswap);
col(iswap) = tmp;
% Plot in brighter color those nodes with larger within connections
[Z,isrt]   = sort(diag(r));
Z          = (Z-min(Z)+0.01)/(max(Z)-min(Z)+0.01);
ncolor     = hsv2rgb([repmat(ncolor(1:2), sz(1),1) Z*ncolor(3)]);
s.s        = scatter(x(isrt),y(isrt),round(Z*48), ncolor,'fill');

% PLACE TEXT LABELS such that you always read 'left to right'
ipos       = x > 0;
s.t        = zeros(sz(1),1);
s.t( ipos) = text(x( ipos)*1.1, y( ipos)*1.1, lbls( ipos),'Color',tcolor);
set(s.t( ipos),{'Rotation'}, num2cell(theta(ipos)'/tau*360))
s.t(~ipos) = text(x(~ipos)*1.1, y(~ipos)*1.1, lbls(~ipos),'Color',tcolor);
set(s.t(~ipos),{'Rotation'}, num2cell(theta(~ipos)'/tau*360 -
180),'Horiz','right')

% ADJUST FIGURE height width to fit text labels
xtn        = cell2mat(get(s.t,'extent'));
post       = cell2mat(get(s.t,'pos'));
sg         = sign(post(:,2));
posfa      = cell2mat(get([gcf gca],'pos'));
% Calculate xlim and ylim in data units as x (y) position + extension along
x (y)
ylims      = post(:,2) + xtn(:,4).*sg;
ylims      = [min(ylims), max(ylims)];
xlims      = post(:,1) + xtn(:,3).*sg;
xlims      = [min(xlims), max(xlims)];
% Stretch figure
posfa(1,3) = (( diff(xlims)/2 - 1)*posfa(2,3) + 1) * posfa(1,3);
posfa(1,4) = (( diff(ylims)/2 - 1)*posfa(2,4) + 1) * posfa(1,4);
% Position it a bit lower (movegui slow)
posfa(1,2) = 100;

% Axis settings
set(gca, 'Xlim',xlims,'Ylim',ylims, 'XColor','none','YColor','none',...
         'clim',[min(r(tf)),max(r(tf))])
set(gcf, 'pos' ,posfa(1,:),'Visible','on')
axis equal

% Set colormap
colormap(gca,ccolor);

if nargout == 1
    h = s;
end
end
```